



Pashov Audit Group

Ensemble Security Review

July 20th 2026 - August 10th 2026

Contents

1. About Pashov Audit Group	4
2. Disclaimer	4
3. Risk Classification	4
4. Methodology	5
5. Executive Summary	5
6. Findings	6
Medium Severity	9
[M-01] Cross-chain deposits and redeems silently lose funds because the composer is never called	9
[M-02] Configured builder never receives fees on vault orders	11
[M-03] Inbound cross-chain messages are never checked against the configured source pool	12
[M-04] Anyone can freeze all deposits and redemptions by triggering a bogus emergency close	13
[M-05] Emergency liquidation can leave a vault with leveraged exposure	15
[M-06] CCIP transfers break because token pools use non-standard entypoints	16
[M-07] Order book spoofing can leave emergency close orders unfilled and lock the vault	18
[M-08] Keeper rewards can be farmed by round-tripping self-created buffer imbalance	19
[M-09] Emergency pause can be reverted, letting redemptions drain a distressed vault	20
[M-10] Stale rebalance cancellation can revert forever, freezing the vault	22
[M-11] Users can farm keeper rewards by redeeming and rebalancing in a loop	23
[M-12] Anyone can trigger a false emergency pause during a slow bridge recovery	24
[M-13] Dust USDC donations can freeze vault recovery and emergency withdrawals for 30 days	26
[M-14] Attackers can drain the composer's cross-chain fee reserve and stall all settlements	27
[M-15] Deposits after vault insolvency can drain the emergency recovery pool from existing holders	29
[M-16] Emergency withdrawal permanently locks bridged holders' funds	30
Low Severity	32
[L-01] Small leftover redemption deficits can block permissionless buffer refills and freeze redemptions	32
[L-02] Attacker can freeze the vault for 30 days by shrinking bridge settlement tolerance	34
[L-03] Redeemers exit at gross NAV, leaving position unwind costs to remaining holders	35
[L-04] Keeper lens can leave the vault stuck in recovery longer than needed	37
[L-05] Empty order book on an unrelated perp blocks buffer deployment and margin recovery	38
[L-06] Vault NAV is understated when a perp position is underwater, letting depositors buy in at a discount	40
[L-07] Deploying buffer into an insolvent margin account destroys protected vault funds	42
[L-08] Capital can be deployed into a vault that the keeper reward pushes below the dormant threshold	43
[L-09] NAV counts an unwithdrawable reserve, shortchanging the last redeemers	44
[L-10] Rebalance orders ignore taker fees, leaving the vault over-leveraged with a thinner margin buffer	45
[L-11] Vault can get stuck with both margin-relief paths blocked at the emergency line	46

[L-12] Core-to-EVM settlement confirms even when the bridge transfer never lands	47
[L-13] Rebalancing silently stops when the order cap hits the venue minimum	49
[L-14] A single wide order book blocks rebalancing and deleveraging for the whole vault	50
[L-15] Emergency close and rebalance orders are priced away from the market and never fill	52
[L-16] Vault can be permanently frozen by triggering a keeper reward during bridge recovery	53
[L-17] Emergency position cleanup is unreachable when VaultAdmin owns the vault	55
[L-18] Large redemptions can overleverage remaining shareholders for hours	56
[L-19] Dormant-vault guard can be skipped when Core spot balance is nonzero	57
[L-20] A single tiny dormant position can block flattening of all others	58
[L-21] Unrelated USDC transfers can prematurely clear a stalled bridge and inflate NAV	58
[L-22] Successful rebalances can be wrongly recorded as failures after market moves	59
[L-23] Rebalance orders can exceed the safety cap set by the adaptive policy	60
[L-24] Bridge recovery can freeze the vault when funds land in Core spot	61
[L-25] Keeper reward can drain the protocol's minimum reserve buffer	63
[L-26] Emergency keeper rewards can drain funds reserved for a pending bridge recovery	64
[L-27] Emergency withdrawals can strand vault USDC on HyperCore and socialize the loss	65
[L-28] Emergency withdrawals pay out assets reserved for management fees	67
[L-29] External deposits can block confirmation of a completed rebalance	68
[L-30] Failed EVM-to-Core bridge can lock the vault in recovery state	69
[L-31] Bridge withdrawals can freeze when HyperCore fees exceed the hardcoded reserve	71
[L-32] Keeper rewards can eat into USDC owed to the treasury	72
[L-33] Force-clearing a stuck bridge recovery can overstate the vault's tracked balance	73
[L-34] Management fees can be suppressed indefinitely by frequent vault interactions	75
[L-35] Management fees can be understated during drawdowns	76
[L-36] A single failed bridge can freeze the entire vault for over a week	78
[L-37] Partial bridge settlement can permanently freeze the vault	79
[L-38] Permissionless capital deployment can drain accrued fee reserves into the strategy	80
[L-39] Anyone can indefinitely block vault unpause and emergency withdrawals	82
[L-40] Keeper rewards skim management fees owed to the treasury	83
[L-41] Rebalance confirmation can freeze the vault or accept an unfilled rebalance	84
[L-42] Stale dust positions can block emergency liquidation of the whole vault	86
[L-43] Redemptions can reopen while a stale rebalance order is still live	87
[L-44] Stale rebalance cancellation can turn a successful partial rebalance into a recorded failure	88
[L-45] One unreadable price can revert the entire emergency close	90
[L-46] Residual HyperCore dust can permanently block emergency withdrawals	91
[L-47] Stuck rebalance can block all deposits and redeems until stale cancellation	93
[L-48] Unpriceable positions can permanently lock all withdrawals during emergency recovery	94
[L-49] Vault is missing standard ERC-4626 preview and conversion functions	96

1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Telegram [@pashovkrum](#).

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical

Loss or compromise of critical assets, or protocol-wide impact.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High

- leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium

- leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low

- leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High

- attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium

- only a conditionally incentivized attack vector, but still relatively likely

Low

- has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of Ensemble was conducted by the Pashov Audit Group team. This report documents 65 issues across the reviewed codebase, reported by aslanbek, ast3ros, Farman, h2134, LoopGhost, Said, Tejas, unforgiven, Valves Security, XOsauce.

FILES IN SCOPE

```
HyperCoreReadAdapter.sol
cross-chain/CcipSourceRouter.sol
cross-chain/CcipVaultComposer.sol
cross-chain/pools/BearBurnMintTokenPool.sol
cross-chain/pools/BearLockReleaseTokenPool.sol
cross-chain/pools/BullBurnMintTokenPool.sol
cross-chain/pools/BullLockReleaseTokenPool.sol
cross-chain/pools/VolBurnMintTokenPool.sol
cross-chain/pools/VolLockReleaseTokenPool.sol
cross-chain/pools/shared/VaultBurnMintTokenPool.sol
cross-chain/pools/shared/VaultLockReleaseTokenPool.sol
cross-chain/tokens/CcipBearToken.sol
cross-chain/tokens/CcipBullToken.sol
cross-chain/tokens/CcipVolToken.sol
cross-chain/tokens/shared/CcipVaultShareToken.sol
interfaces/ILeveragedVault.sol
libraries/CoreWriterLib.sol
libraries/HyperCoreBridgeConfig.sol
libraries/LeveragedVaultLib.sol
libraries/LeveragedVaultOpsLib.sol
libraries/PrecompileLib.sol
libraries/VaultAdminOpsLib.sol
libraries/VaultBridgeOpsLib.sol
libraries/VaultExecutionPolicyLib.sol
libraries/VaultPortfolioLib.sol
mocks/BatchCaller.sol
mocks/LibraryHarness.sol
mocks/MockCcipRamps.sol
mocks/MockCcipRouter.sol
mocks/MockCcipVault.sol
```

```

mocks/MockCircuitBreaker.sol
mocks/MockCoreWriter.sol
mocks/MockHyperCoreSim.sol
mocks/MockMulticall3.sol
mocks/MockPrecompile.sol
mocks/MockProjectXPool.sol
mocks/MockReentrantCoreWriter.sol
mocks/MockReentrantUSDC.sol
mocks/MockUSDC.sol
mocks/MockVaultShares18.sol
mocks/OverflowingMarginPrecompile.sol
mocks/ProxyImport.sol
mocks/RejectNativeRefundCaller.sol
periphery/CircuitBreaker.sol
periphery/VaultAdmin.sol
periphery/VaultLens.sol
probes/CoreWriterProbe.sol
probes/HyperCoreBridgeProbe.sol
vaults/BearVault.sol
vaults/BullVault.sol
vaults/VolVault.sol
vaults/shared/BaseLeveragedVault.sol
vaults/shared/VaultShareToken.sol

```

6. Findings

ID	TITLE	SEVERITY	STATUS
M-01	Cross-chain deposits and redeems silently lose funds because the composer is never called	Medium	Fixed
M-02	Configured builder never receives fees on vault orders	Medium	Fixed
M-03	Inbound cross-chain messages are never checked against the configured source pool	Medium	Fixed
M-04	Anyone can freeze all deposits and redemptions by triggering a bogus emergency close	Medium	Fixed
M-05	Emergency liquidation can leave a vault with leveraged exposure	Medium	Fixed
M-06	CCIP transfers break because token pools use non-standard entrypoints	Medium	Fixed
M-07	Order book spoofing can leave emergency close orders unfilled and lock the vault	Medium	Fixed
M-08	Keeper rewards can be farmed by round-tripping self-created buffer imbalance	Medium	Fixed
M-09	Emergency pause can be reverted, letting redemptions drain a distressed vault	Medium	Fixed

ID	TITLE	SEVERITY	STATUS
M-10	Stale rebalance cancellation can revert forever, freezing the vault	Medium	Fixed
M-11	Users can farm keeper rewards by redeeming and rebalancing in a loop	Medium	Fixed
M-12	Anyone can trigger a false emergency pause during a slow bridge recovery	Medium	Fixed
M-13	Dust USDC donations can freeze vault recovery and emergency withdrawals for 30 days	Medium	Fixed
M-14	Attackers can drain the composer's cross-chain fee reserve and stall all settlements	Medium	Fixed
M-15	Deposits after vault insolvency can drain the emergency recovery pool from existing holders	Medium	Fixed
M-16	Emergency withdrawal permanently locks bridged holders' funds	Medium	Fixed
L-01	Small leftover redemption deficits can block permissionless buffer refills and freeze redemptions	Low	Fixed
L-02	Attacker can freeze the vault for 30 days by shrinking bridge settlement tolerance	Low	Fixed
L-03	Redeemers exit at gross NAV, leaving position unwind costs to remaining holders	Low	Fixed
L-04	Keeper lens can leave the vault stuck in recovery longer than needed	Low	Fixed
L-05	Empty order book on an unrelated perp blocks buffer deployment and margin recovery	Low	Fixed
L-06	Vault NAV is understated when a perp position is underwater, letting depositors buy in at a discount	Low	Fixed
L-07	Deploying buffer into an insolvent margin account destroys protected vault funds	Low	Fixed
L-08	Capital can be deployed into a vault that the keeper reward pushes below the dormant threshold	Low	Fixed
L-09	NAV counts an unwithdrawable reserve, shortchanging the last redeemers	Low	Fixed
L-10	Rebalance orders ignore taker fees, leaving the vault over-leveraged with a thinner margin buffer	Low	Fixed
L-11	Vault can get stuck with both margin-relief paths blocked at the emergency line	Low	Fixed
L-12	Core-to-EVM settlement confirms even when the bridge transfer never lands	Low	Fixed
L-13	Rebalancing silently stops when the order cap hits the venue minimum	Low	Fixed

ID	TITLE	SEVERITY	STATUS
L-14	A single wide order book blocks rebalancing and deleveraging for the whole vault	Low	Fixed
L-15	Emergency close and rebalance orders are priced away from the market and never fill	Low	Fixed
L-16	Vault can be permanently frozen by triggering a keeper reward during bridge recovery	Low	Fixed
L-17	Emergency position cleanup is unreachable when VaultAdmin owns the vault	Low	Fixed
L-18	Large redemptions can overleverage remaining shareholders for hours	Low	Fixed
L-19	Dormant-vault guard can be skipped when Core spot balance is nonzero	Low	Fixed
L-20	A single tiny dormant position can block flattening of all others	Low	Fixed
L-21	Unrelated USDC transfers can prematurely clear a stalled bridge and inflate NAV	Low	Fixed
L-22	Successful rebalances can be wrongly recorded as failures after market moves	Low	Fixed
L-23	Rebalance orders can exceed the safety cap set by the adaptive policy	Low	Fixed
L-24	Bridge recovery can freeze the vault when funds land in Core spot	Low	Fixed
L-25	Keeper reward can drain the protocol's minimum reserve buffer	Low	Fixed
L-26	Emergency keeper rewards can drain funds reserved for a pending bridge recovery	Low	Fixed
L-27	Emergency withdrawals can strand vault USDC on HyperCore and socialize the loss	Low	Fixed
L-28	Emergency withdrawals pay out assets reserved for management fees	Low	Fixed
L-29	External deposits can block confirmation of a completed rebalance	Low	Fixed
L-30	Failed EVM-to-Core bridge can lock the vault in recovery state	Low	Fixed
L-31	Bridge withdrawals can freeze when HyperCore fees exceed the hardcoded reserve	Low	Fixed
L-32	Keeper rewards can eat into USDC owed to the treasury	Low	Fixed
L-33	Force-clearing a stuck bridge recovery can overstate the vault's tracked balance	Low	Fixed

ID	TITLE	SEVERITY	STATUS
L-34	Management fees can be suppressed indefinitely by frequent vault interactions	Low	Fixed
L-35	Management fees can be understated during drawdowns	Low	Fixed
L-36	A single failed bridge can freeze the entire vault for over a week	Low	Fixed
L-37	Partial bridge settlement can permanently freeze the vault	Low	Fixed
L-38	Permissionless capital deployment can drain accrued fee reserves into the strategy	Low	Fixed
L-39	Anyone can indefinitely block vault unpause and emergency withdrawals	Low	Fixed
L-40	Keeper rewards skim management fees owed to the treasury	Low	Fixed
L-41	Rebalance confirmation can freeze the vault or accept an unfilled rebalance	Low	Fixed
L-42	Stale dust positions can block emergency liquidation of the whole vault	Low	Fixed
L-43	Redemptions can reopen while a stale rebalance order is still live	Low	Fixed
L-44	Stale rebalance cancellation can turn a successful partial rebalance into a recorded failure	Low	Fixed
L-45	One unreadable price can revert the entire emergency close	Low	Fixed
L-46	Residual HyperCore dust can permanently block emergency withdrawals	Low	Fixed
L-47	Stuck rebalance can block all deposits and redeems until stale cancellation	Low	Fixed
L-48	Unpriceable positions can permanently lock all withdrawals during emergency recovery	Low	Fixed
L-49	Vault is missing standard ERC-4626 preview and conversion functions	Low	Fixed

Medium Severity

[M-01] Cross-chain deposits and redeems silently lose funds because the composer is never called

Medium

Fixed

SUMMARY

`CcipVaultComposer` is meant to receive CCIP messages and run the destination-chain settlement logic for cross-chain deposits and redeems. It fails to implement ERC-165 support for Chainlink's `IAny2EVMMessageReceiver` interface.

The contract inherits only a local bare interface that declares nothing but `ccipReceive`, and never implements `supportsInterface`:

```
contract CcipVaultComposer is ICcipReceiver, Ownable2Step, Pausable, ReentrancyGuardTransient {  
  
    interface ICcipReceiver {  
        function ccipReceive(CcipTypes.Any2EVMMessage calldata message) external;  
    }  
}
```

The real CCIP OffRamp delivers inbound tokens to the receiver first, then checks whether the receiver reports support for `IAny2EVMMessageReceiver` before calling `ccipReceive`:

```
releaseOrMintTokensToReceiver();  
if (!receiverSupportsIAny2EVMMessageReceiver) {  
    return;  
}  
receiver.ccipReceive(message);
```

Because the composer does not implement ERC-165, this check fails and the OffRamp returns before ever invoking `ccipReceive`. The tokens have already landed in the composer, but the settlement branch (`_handleDeposit` / `_handleRedeem`) never runs.

For example, a user sends 100 USDC cross-chain for a vault deposit. CCIP releases the 100 USDC to the composer, then skips `ccipReceive` because the support check fails, so `_handleDeposit()` is never called and the user receives no shares. The message is still marked `SUCCESS`.

IMPACT

The core destination-chain settlement logic is completely bypassed.

Cross-chain deposits do not mint or deliver vault shares, and cross-chain redeems do not redeem or deliver assets. Inbound tokens are stranded inside `CcipVaultComposer`.

Because the CCIP message is marked `SUCCESS`, it cannot be retried or manually re-executed through the normal CCIP flow, so users cannot recover their funds that way.

EXPLOIT PATH

1. A user sends a cross-chain deposit or redeem message to the composer via the real Chainlink CCIP OffRamp.
2. The OffRamp releases the inbound tokens to `CcipVaultComposer`.
3. The OffRamp checks whether the composer supports `IAny2EVMMessageReceiver`; the check fails because ERC-165 is not implemented.
4. `ccipReceive` is skipped, so settlement never runs and the message is marked `SUCCESS`.

PREREQUISITES

A cross-chain deposit or redeem message is routed to `CcipVaultComposer` on the destination chain through the real Chainlink CCIP OffRamp (which performs the ERC-165 receiver check).

RECOMMENDATION

Implement ERC-165 on `CcipVaultComposer` and report support for `IAny2EVMMessageReceiver`, matching Chainlink's expected receiver behavior. The simplest robust approach is to inherit Chainlink's `CCIPReceiver` (or otherwise implement `supportsInterface` to return true for the

`IAny2EVMMessageReceiver` interface id and for ERC-165 itself), so the OffRamp's receiver support check passes and `ccipReceive` is invoked after token delivery.

RESOLUTION

Ensemble

Status: fix in place at audit kickoff.

The composer entered the audit using Chainlink's official `CCIPReceiver`. Commit `381013fe394f81ecd10790f50c784c91a1cb0b99`, dated July 17, 2026 at 13:46:23 PT, introduced that path. The receiver exposes ERC-165, routes messages through `ccipReceive`, and accepts calls only from the configured router.

The finding-specific proof is in `775aabad0a7073c126a37ddff7391c0aacb4c7da: test/foundry/unit/CcipRoutingBehavior.t.sol::test_M01_composerReceiverDeclarationDispatchesSettlement`.

Related findings: [M-3](#) and [M-6](#).

fixed-PAG

[M-02] Configured builder never receives fees on vault orders

Medium

Fixed

SUMMARY

`adminUpdate(5)` lets the owner configure a builder for the vault. It calls `LeveragedVaultOpsLib.approveBuilderFee` (VaultAdminOpsLib.sol:107-110), which submits HyperCore action 12 and records a builder with a fee-rate cap. The intent is to compensate that builder for the vault's order flow.

Per the Hyperliquid docs, payment is decided per order, not per account:

"Once the authorization is complete, future order actions sent on behalf of the user may include an optional builder parameter: `{"b": address, "f": number}`."

The parameter is optional. An order that omits it is attributed to no builder, and there is no automatic attribution to an account-level authorized builder. The authorization only sets the fee cap; the order itself must carry the builder to be charged.

The vault never includes the parameter. Every vault order goes through `placeLimitOrder` with the same argument list (asset, direction, limit price, size, reduce-only, time-in-force, and a deterministic client order ID) and no builder field. This covers both order paths: `_submitRebalanceOrder` (LeveragedVaultLib.sol: 1286-1295) and `_placeEmergencyCloseOrder` (LeveragedVaultLib.sol:1460-1462).

IMPACT

A builder set through `adminUpdate(5)` is never paid. Because the vault's orders carry no builder parameter, HyperCore attributes none of the vault's fills to that builder, so the configured fee never accrues.

The configuration produces no payment at all: the intended compensation is silently skipped on every vault fill.

RECOMMENDATION

Make the order path deliver the fee the configuration intends. Add the builder parameter `{"b": builder, "f": feeRate}` to the `placeLimitOrder` encoding in both `_submitRebalanceOrder` and `_placeEmergencyCloseOrder`, sourced from the builder and rate recorded by `approveBuilderFee`, so the configured builder is charged on the vault's fills.

RESOLUTION

Ensemble

Commit `ddbf3f41be1ce3542bf5393ef9675ee255882bcf` fixes M-2. Xulian from Hyperliquid confirmed that an EVM contract cannot permissionlessly attach a builder code to these orders and recommended handling the fee in HyperEVM. The vault stores the builder recipient and rate on EVM, accrues fees from observed rebalance fills, and reserves each recipient's unpaid liability from treasury collection. The obsolete CoreWriter approval-only path is removed. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_M2_ownerConfiguresEvmBuilderFeeWithoutCoreWriterApproval`. Related findings: M-10, L-22, L-29, L-41, L-43, L-44, L-47.

fixed-PAG

[M-03] Inbound cross-chain messages are never checked against the configured source pool

Medium

Fixed

SUMMARY

Each pool stores a per-lane `remotePool` (the expected token pool on the source chain) via `configureRemotePool`. This value is meant to authenticate where an inbound message came from, but it is never read to gate an inbound `releaseTo` / `mintTo`. The only place `remotePool` is read is a `.length != 0` "is this lane initialized?" probe.

Inbound authentication therefore collapses to two checks: `msg.sender == router` and `remote.enabled`. Any message the router relays for an enabled lane settles regardless of which source pool actually produced it.

```
function releaseTo(address receiver, uint64 sourceChainSelector, uint256 amount) external {
    if (msg.sender != router) revert NotRouter(msg.sender);
    RemotePool storage remote = remotePools[sourceChainSelector];
    if (!remote.enabled) revert BadRoute(sourceChainSelector);
    ...
    token.safeTransfer(receiver, amount);
}
```

`mintTo` on the burn/mint pool is structurally identical, minting instead of transferring. The configured `remotePool` / `remoteToken` are write-only state, set and emitted by `configureRemotePool`, and read only by the initialization probe:

```
function _routeInitialized(RemotePool storage remote, RateLimitState storage state) internal view returns (bool) {
    return state.lastUpdated != 0 || remote.enabled || remote.remotePool.length != 0
        || remote.remoteToken.length != 0 || remote.outboundCapacity != 0 ||
        remote.inboundCapacity != 0;
}
```

Chainlink's reference `TokenPool` reverts `InvalidSourcePoolAddress` in exactly this case. These pools cannot: the hook never even receives the source pool address, so it is unable to perform the check in principle. The source-pool binding is dead configuration.

IMPACT

For every enabled lane, the source-pool authentication the design explicitly provisions provides no protection. The only remaining barrier is the `router` sender check plus the `enabled` flag.

The lock/release pool can have its full locked share balance released to an arbitrary receiver, and the burn/mint pool can have arbitrary new shares minted to an arbitrary receiver, for any message the router relays on an enabled lane.

How far this is exploitable depends on the trust model of `router`. If `router` is a fully trusted CCIP off-ramp that already binds each enabled lane to exactly one source pool, this degrades toward a defense-in-depth gap. Even so, the stored `remotePool` is presented as the source-authentication control yet is completely inert, making this a real missing-validation issue.

RECOMMENDATION

Pass the source pool address into the inbound hooks and validate it against the stored binding, matching Chainlink's reference behavior. The hook signatures must be updated and the router integration updated to supply the value:

```
-function releaseTo(address receiver, uint64 sourceChainSelector, uint256 amount) external {
+function releaseTo(address receiver, uint64 sourceChainSelector, bytes calldata sourcePoolAddress,
uint256 amount) external {
    if (msg.sender != router) revert NotRouter(msg.sender);
    RemotePool storage remote = remotePools[sourceChainSelector];
    if (!remote.enabled) revert BadRoute(sourceChainSelector);
+    if (keccak256(sourcePoolAddress) != keccak256(remote.remotePool)) revert InvalidSourcePoolAddress();
    ...
}
```

Apply the same change to `mintTo`, validating `sourcePoolAddress` against `remote.remotePool`.

RESOLUTION

Ensemble

Status: fix in place at audit kickoff.

Both shared vault pools used Chainlink's official implementations before the audit started. Commit `381013fe394f81ecd10790f50c784c91a1cb0b99` introduced the source-pool authentication relevant to M-3. The inbound validation rejects unregistered source pools.

The finding-specific proof is in `8e3836352f525c600069e6c2825ec894645996bc`: `test/foundry/unit/CcipRoutingBehavior.t.sol::test_M03_releaseRejectsSpoofedSourcePoolBeforeSettlement`.

Related findings: [M-1](#) and [M-6](#).

fixed-PAG

[M-04] Anyone can freeze all deposits and redemptions by triggering a bogus emergency close

Medium

Fixed

SUMMARY

`emergencyCloseStatus` treats `IERC20(usdc).balanceOf(vault) < accountedBalance` as conclusive proof that vault USDC has been lost, and uses that single check to authorize a permissionless state transition into `Paused` (`LeveragedVaultLib.sol:663-670`).

The protocol's own bridge accounting deliberately breaks that equality during recovery. For a to-Core bridge that HyperCore never credited, `reverseBridgeAccounting()` restores `accountedBalance` to its pre-bridge value even though `depositFor` has already moved the USDC out of the vault (`VaultBridgeOpsLib.sol:137-148`). This is the correct recovery state, NAV is exact and nothing is lost, yet it is indistinguishable from a real loss under this test.

Every other NAV-sensitive entrypoint guards against this by refusing to run while `bridgePendingAmount != 0`. `emergencyClose` does not, and `emergencyCloseStatus` is never passed any bridge state, no bridge parameter appears in its signature at all.

IMPACT

An attacker pays one transaction's gas and collects the full `keeperReward` for zero work on a vault that has lost nothing. The bookkeeping loss itself is small and bounded.

The material harm is the recovery-profile downgrade. Without the call, the bridge-recovery state can be cleared by anyone through the permissionless, state-gate-free `reconfirmBridge`, and deposits and redemptions resume in the same block with no owner involvement.

With the call, the owner must run `reconfirmBridge`, then repeated admin bridge cycles to drain 100% of HyperCore perp equity, then `confirmBridge`, then wait out a 24-hour cooldown, then `unpause`. Throughout that window every holder's `redeem` hard-reverts with `EmergencyCloseActive`, and the vault cannot deploy, withdraw, rebalance, or collect fees. On a vault with meaningful AUM the drain alone spans many bridge round trips.

PREREQUISITES

The attacker cannot induce the precondition. It requires a genuine non-settlement past the confirm timeout with a flat, zero-equity Core account, such as at launch or immediately after an unpause. This makes the issue opportunistic escalation rather than on-demand griefing. Because the precondition guarantees zero leveraged exposure at the moment of the freeze, users are locked out of a flat USDC position with no price risk, no funds are lost, and NAV stays exact throughout.

RECOMMENDATION

Pass the recovery amount into `emergencyCloseStatus` and net it out in a direction-aware way. Only a to-Core recovery inflates `accountedBalance` above the physical buffer; a to-EVM recovery restores exact parity, so netting must not apply there or it would mask a real loss.

Compute `inFlightToCore = bridgeToCore ? bridgePendingAmount : 0` and compare `currentBuffer` against `accountedBalance - inFlightToCore` instead of `accountedBalance`, then update the caller to forward `bridgePendingAmount` and `bridgeToCore`.

Apply the identical change to the lens mirror so keeper telemetry stops advertising the action; both fields live in slot 5, readable as `bridgePendingAmount = uint64(slot5)` and `bridgeToCore = uint8(slot5 >> 232) != 0`.

As defence in depth, add a bridge-recovery gate to the permissionless branch of `emergencyClose` so it cannot latch `lastEmergencyTimestamp` while a resolvable recovery is outstanding, while keeping the

owner-cleanup branch and the genuine negative-equity, non-zero-margin and free-margin branches reachable.

RESOLUTION

Ensemble

This shares the M-12 fix in [4375341](#). The emergency loss check now reads the Core-only transfer marker and leaves the expected EVM buffer intact during retained spot-to-perps recovery. The regression test creates an EVM shortfall in that state and verifies that recovery no longer masks it. The Lens uses the same helper, and ordinary bridge-principal adjustments are unchanged.

fixed-PAG

[M-05] Emergency liquidation can leave a vault with leveraged exposure

Medium

Fixed

LOCATION

`BaseLeveragedVault:emergencyClose:approximately lines 470-525`

SUMMARY

The emergency workflow is meant to cancel any outstanding rebalance orders and then flatten every configured perpetual position, so a paused vault has no way to acquire fresh exposure.

The implementation clears `rebalancePending` and submits both the cancellation and the close actions in a single EVM transaction. CoreWriter operations settle asynchronously on HyperCore, so the cancellation is not confirmed before `closeAllPositions` submits its reduce-only orders.

Because of this, a previously submitted expansion order can still be live and fill after the close sequence runs. Clearing local rebalance state is treated as proof the async order was cancelled, but it is not.

The reduce-only close orders are built from the pre-settlement position snapshot, so they cannot cover exposure that only appears after a later fill.

IMPACT

A vault undergoing emergency liquidation can retain or reacquire leveraged exposure after its close orders are submitted, while the emergency accounting assumes the liquidation sequence is authoritative.

Subsequent adverse price movement can reduce the collateral available for emergency withdrawals, creating additional shareholder losses or bad debt during the paused recovery period.

EXPLOIT PATH

1. A pending rebalance contains an expansion order and the vault later meets the emergency-close condition.
1. `emergencyClose()` clears the pending rebalance, submits `cancelOrdersByAssets()`, and immediately submits `closeAllPositions()`.
1. HyperCore processes the close order before the old cancellation, or processes the old expansion after the close has filled.
1. The old expansion fills against the flat or partially flat account, reopening leveraged exposure and increasing losses while the vault is paused.

RECOMMENDATION

Track cancellation as a separate asynchronous phase and block close submission until the cancellation has actually been observed:

```
if (rebalancePending) {
    _requestRebalanceCancellation(cancelNonce);
    emergencyClosePending = true;
    return;
}

if (emergencyClosePending && _cancellationsConfirmed()) {
    _submitEmergencyCloseOrders();
}
```

Additionally, emergency recovery should require a fresh position read showing zero size for every configured asset before declaring liquidation complete. Any nonzero position should keep the vault paused and permit another close attempt, since fills visible only after the first close may otherwise go uncovered.

RESOLUTION**Ensemble**

Commit [f24f94c6b9e27fc6523636b7d4b5300f08009b37](#) fixes M-5. Emergency close first persists the pause and submits cancellations. It waits for a later block and the configured confirmation timeout before reading fresh positions and placing reduce-only close orders. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_M5_emergencyCloseCancelsBeforeFreshPositionClose](#). Related findings: M-7, M-9.

fixed-PAG

[M-06] CCIP transfers break because token pools use non-standard endpoints

Medium

Fixed

LOCATION

[VaultBurnMintTokenPool:burnFrom/mintTo:18-45](#)

SUMMARY

Each vault token pool is meant to be driven by the configured Chainlink CCIP router as a standard burn/mint or lock/release pool. The router escrows or burns tokens on the source chain and mints or releases the matching amount on the destination chain using the standard CCIP pool ABI.

Instead, the pools expose bespoke hooks:

```
function burnFrom(address sender, uint64 chain, uint128 amount) external;
function mintTo(address receiver, uint64 chain, uint128 amount) external;
function lockFrom(address sender, uint64 chain, uint256 amount) external;
function releaseTo(address receiver, uint64 chain, uint256 amount) external;
```

These selectors and signatures do not match the standard Chainlink pool interface (such as [lockOrBurn](#) / [releaseOrMint](#)). Router authorization inside these custom functions cannot compensate for the ABI the

external integration actually calls. Any CCIP lane configured to use these contracts as token pools therefore cannot complete transfers through the real router/off-ramp.

IMPACT

Cross-chain deposits, redeems, outbound result transfers, and refunds routed through these pools can fail or remain unexecutable.

This is an availability and integration failure rather than an unauthorized-funds vulnerability. Recovery requires deploying and configuring pool contracts that expose the standard interface.

EXPLOIT PATH

1. The owner registers BullBurnMintTokenPool or BullLockReleaseTokenPool as the CCIP pool for BULL on a source or destination lane.
1. A user calls `CcipSourceRouter.deposit` or `redeem`, causing the router to process a message with a BULL/USDC token amount.
1. The CCIP router attempts the standard pool call (`lock0rBurn` or `release0rMint`) against the custom pool.
1. No matching function exists, the interaction reverts, and the cross-chain transfer cannot settle.

RECOMMENDATION

Implement the standard Chainlink pool interface directly and delegate to the existing accounting logic:

```
function lock0rBurn(Lock0rBurnInV1 calldata input)
    external
    onlyRouter
    returns (bytes memory destPoolData, bytes memory destTokenData)
{
    // Validate route/capacity and lock or burn input.amount.
    // Return the exact CCIP-required metadata.
}
```

```
function release0rMint(Release0rMintInV1 calldata input)
    external
    onlyRouter
    returns (uint256 amount)
{
    // Validate route/capacity and release or mint input.amount.
    return input.amount;
}
```

Use the exact structs and interface version required by the deployed Chainlink CCIP router rather than approximating them with custom hooks.

RESOLUTION

Ensemble

`375e3ca` adds shutdown handling to the remote pool and separates stopping new transfers from sealing the lane. Preparation blocks outbound transfers at both pools while allowing messages already in flight to settle. Operators must then verify that both directions have drained, refresh the supply and backing commitments, seal the remote pools, and finally seal the canonical pool while enabling withdrawals.

Preparation cannot be undone, and recovery does not reopen the lane. Tests cover blocked outbound transfers, inbound settlement before sealing, stale commitments, and attempts to reopen. Existing remote burn/mint pools need to be replaced because they are not upgradeable.

fixed-PAG

[M-07] Order book spoofing can leave emergency close orders unfilled and lock the vault

Medium

Fixed

LOCATION

`LeveragedVaultLib:_tryEmergencyCloseLimitPx:1458-1488`

SUMMARY

Emergency close reads the best bid/offer (BBO) and clamps only expensive buys and cheap sells against the reference price. A low offer used to buy back a short, or a high bid used to sell a long, passes through unchanged because it looks favorable.

The resulting reduce-only IOC is submitted asynchronously through CoreWriter with no same-transaction fill confirmation. An attacker can present favorable liquidity during the EVM read and remove it before HyperCore processes the order, so the order is non-marketable by the time it executes.

Because the price is not bounded against a trusted mark or oracle, a stale or manipulated order book can prevent liquidation of the vault's positions.

IMPACT

An attacker who can influence and then remove the top-of-book quote can cause an emergency close to never fill while the vault has already transitioned to Paused with its position still open.

Recovery checks prevent unpause and emergency withdrawal until a close succeeds, so the vault can be locked. The attacker can repeat the quote-and-remove sequence to keep it stuck.

EXPLOIT PATH

1. A vault with a short BTC position reaches the emergency margin condition; the attacker places a far-below-mark best offer and calls `emergencyClose`.
1. `_tryEmergencyCloseLimitPx` reads the offer, leaves it unchanged because it is favorable for a buy, and submits a reduce-only IOC buy through CoreWriter while changing the vault to Paused.
1. Before asynchronous HyperCore execution, the attacker cancels the offer, so the IOC cannot fill at the stale low limit and the short position remains open.
1. The owner attempts unpause or emergency withdrawal, but `ensurePositionsClosed` (or equivalent) reverts because the position is still open; the attacker repeats the sequence.

RECOMMENDATION

Clamp both sides of the reference price to the trusted anchor band so a favorable-side quote cannot pass through unbounded:

```
uint256 lower = uint256(anchorPx) * (p.bpsDenominator - p.emergencyMaxSpreadBps) / p.bpsDenominator;
uint256 upper = uint256(anchorPx) * (p.bpsDenominator + p.emergencyMaxSpreadBps) / p.bpsDenominator;
if (rawReferencePx < lower) rawReferencePx = lower.toUint64();
if (rawReferencePx > upper) rawReferencePx = upper.toUint64();
```

Also require a later position-state confirmation before treating emergency closure as complete; an accepted CoreWriter submission should not by itself imply the position was closed.

RESOLUTION

Ensemble

Commit `facd884286ce680d4e9d6001761e3dd0e18ac160` fixes M-7. Emergency IOC orders use the worst acceptable price in the mark/oracle-anchored band. The close proceeds only when the live book is marketable within that band; otherwise the asset remains available for retry. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_M7_emergencyCloseUsesWorstAcceptableBandForFavorableSellBook`. Related findings: M-5, M-9, L-15.

fixed-PAG

[M-08] Keeper rewards can be farmed by round-tripping self-created buffer imbalance

Medium

Fixed

LOCATION

`BaseLeveragedVault:deployCapital:329-359`

SUMMARY

Keeper rewards are meant to compensate permissionless actors for maintaining the vault's buffer and Core collateral. The current flow lets a single actor manufacture the imbalance, perform the maintenance, reverse it, and collect the reward.

`deployCapital` pays `_payKeeperReward` before checking whether the excess arose solely from a recent deposit. After the deposited amount is bridged to Core, the same user can redeem shares from the EVM buffer, creating a deficit. `withdrawCapital` then bridges collateral back, and `confirmBridge` pays `_payBridgeKeeperReward` without checking the origin of that deficit.

The result is a repeatable, holder-funded reward loop under ordinary deployable configurations with nonzero keeper rewards. The capital is largely round-tripped, so net gain is positive whenever the rewards exceed deposit, withdrawal, and execution costs.

If chunked Core-to-EVM refills are allowed, the pattern is worse: one large deficit can be split into many separately rewarded bridge cycles.

IMPACT

Existing vault holders subsidize a caller who round-trips self-created capital movements while collecting one or two keeper rewards per cycle.

Under profitable parameters, repeated cycles directly drain tracked USDC from the vault.

EXPLOIT PATH

1. Deposit an amount `A` through `CircuitBreaker.deposit` large enough to exceed the target buffer by at least `minDeployAmount` plus the deploy reward.
2. Call `deployCapital`; the vault pays the caller `keeperReward`, pre-debits accounting, and bridges the excess to Core.
3. After settlement, `redeem` the shares from step 1; the vault pays from its remaining EVM buffer, dropping it below target while the capital sits in Core.
4. Call `withdrawCapital` and later `confirmBridge`; the Core-to-EVM confirmation pays a second keeper reward, and the cycle repeats while the vault stays liquid.

PREREQUISITES

The attacker needs capital to seed the initial deposit and must tolerate bridge confirmation delays. Keeper rewards must be nonzero, which holds under ordinary deployable configurations.

RECOMMENDATION

Gate rewards on an independent maintenance epoch or state transition rather than merely on a callable buffer operation, so a reward can be paid at most once per genuine maintenance need:

```
if (lastRewardEpoch == currentEpoch) revert RewardAlreadyPaid();
if (!independentMaintenanceNeeded()) revert NoRewardableMaintenance();
lastRewardEpoch = currentEpoch;
_payKeeperReward();
```

Also track the origin of buffer changes and refuse a reward when the pending imbalance was created by the same account's recent deposit or redeem:

```
if (msg.sender == lastFlowAccount && block.timestamp < lastFlowTimestamp + COOLDOWN) {
    revert SelfCreatedImbalance();
}
```

A single-lifecycle reward marker is important because repeated bridge-phase rewards amplify the value of self-created buffer oscillation.

RESOLUTION

Ensemble

Commit `c19fad84387fe5ce2785fceb5143425e0982f631` fixes M-8. Anyone may call keeper actions, but ordinary rewards can be paid only to the governance-configured recipient stored in reserved slot 21. Proof case: `test/foundry/shared/VaultLifecycleIntegrationShared.t.sol::test_M8_bridgeRewardPaysConfiguredRecipientOnlyAfterSettlement`. Related findings: M-11, L-8.

fixed-PAG

[M-09] Emergency pause can be reverted, letting redemptions drain a distressed vault

Medium

Fixed

LOCATION

`src/vaults/shared/BaseLeveragedVault.sol:emergencyClose:503-560`

SUMMARY

The emergency flow is meant to pause a vault once it falls below its emergency margin threshold and hold that paused state while positions are closed. In practice the pause write and the position reads happen in the same transaction, so a single failed read can roll back the pause.

`emergencyCloseStatus()` may authorize the operation, but `closeAllPositions()` then performs a strict position read for every configured asset and bubbles up `PrecompileCallFailed()` if any one read fails. Because the tentative pause write and the reads share one transaction, that revert erases the pause and all emergency markers.

`redeem()` checks only stored state, so after the rollback it sees `vaultState` as `Normal` and no `lastEmergencyTimestamp`, and proceeds even though the live account is still distressed. This opens a bank-run window whenever account-health reads remain available but one position read is unavailable.

IMPACT

During a position-read outage affecting a configured market, a distressed vault can stay in `Normal` state and keep paying redemptions from its EVM USDC buffer while leveraged positions remain open.

Early redeemers can withdraw the buffered liquid assets before liquidation succeeds, leaving the remaining holders with open leveraged exposure and fewer liquid assets.

EXPLOIT PATH

1. A vault's free margin falls below its emergency threshold while the EVM buffer still holds USDC.
1. One configured asset's position precompile read reverts, so `emergencyClose()` reverts inside `closeAllPositions()` after its tentative pause write.
1. The rollback leaves `vaultState` as `Normal` and `lastEmergencyTimestamp` unset, so a holder redeems successfully because `redeem()` sees no stored emergency flag.
1. Multiple holders repeat redemptions while the read outage persists, draining the buffer before the position can finally be closed.

RECOMMENDATION

Persist the pause before attempting to close positions, so a later revert cannot undo it:

```
function emergencyClose() external nonReentrant {
    _authorizeEmergency();
    if (vaultState == VaultState.Normal) {
        vaultState = VaultState.Paused;
        lastEmergencyTimestamp = block.timestamp.toUint40();
    }
    _submitCloseOrdersBestEffort();
}
```

Also isolate failures per asset so one unavailable position does not revert all close submissions:

```
try this.readPosition(asset) returns (int64 size) {
    _submitCloseIfNeeded(asset, size);
} catch {
    emit EmergencyCloseAssetSkipped(asset, true);
}
```

RESOLUTION

Ensemble

Commit `2bf0b90223f828180f206d2f6ff523b0815cdf0c` fixes M-9. Emergency close commits the pause and cancellation state before any best-effort position read or close submission. A failed asset close is recorded for retry instead of reverting the pause, and no reward is paid while exposure remains. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_M9_emergencyClosePausesWhenCloseSubmissionFails`.

Related findings: M-5, M-7, L-45.

fixed-PAG

[M-10] Stale rebalance cancellation can revert forever, freezing the vault

Medium

Fixed

LOCATION

`BaseLeveragedVault:cancelStaleRebalance:592-620`

SUMMARY

When a portfolio transition completes, only the final configured markets should remain in the active basket, and any residual position below the minimum should be handled separately. Instead, `applyTransition` keeps removed sub-minimum positions in `_activeAssets` with zero weight, and those assets get copied into every future rebalance parameter set.

When a rebalance times out, `cancelStaleRebalance` measures progress before clearing its state, and `positionSizeProgressNotional` performs strict position reads for every active asset. If the read for the retained dust asset fails, the cancellation reverts before the pending flag is cleared.

The obsolete market is no longer economically relevant to the strategy, yet its unavailable position read now controls whether the vault can ever exit a timed-out rebalance. Because `rebalancePending` stays true, all NAV-sensitive operations keep reverting.

IMPACT

A retained zero-weight dust market whose position read fails causes every stale-cancel attempt to revert, leaving `rebalancePending` set.

With the pending flag stuck, deposits, redemptions, bridging, and further rebalances all remain blocked until the read failure is resolved externally or another recovery path intervenes.

EXPLOIT PATH

1. A removed market has a residual position worth less than 10 USDC, so `applyTransition` carries it into `_perpAssets` with weight zero.
1. A later `rebalance()` submits an IOC cycle and sets `rebalancePending = true`; the orders receive no fills.
1. After the timeout, `cancelStaleRebalance()` calls `positionSizeProgressNotional`, whose strict position read for the retained market fails.
1. The cancellation reverts before clearing state, so deposits, redemptions, bridges, and rebalances stay blocked.

RECOMMENDATION

Exclude removed dust assets from the active basket and retain them in a separate dust-only record:

```
(uint32[] memory appliedAssets, uint16[] memory appliedWeights) =
    _appliedBasket(new uint32[](0), finalAssets, finalWeights);
_replace(_activeAssets(), _activeWeights(), appliedAssets, appliedWeights);
```

Additionally, clear pending state when progress reads fail so a cancellation can always proceed:

```
uint256 achievedProgress;
try this.positionSizeProgressNotional(params, submittedSizes) returns (uint256 p) {
    achievedProgress = p;
} catch {
    _clearPendingRebalance();
    LeveragedVaultLib.cancelOrdersByAssets(coreWriterTarget, _perpAssets, cancelNonce);
    return;
}
```

RESOLUTION**Ensemble**

Commit [6d17568147066abb7bb6ba2cec2242ffee52d7ef](#) fixes M-10. Stale cancellation submits every queued cancellation even when the live position-progress read fails, so a precompile outage cannot leave the rebalance permanently pending. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_M10_positionReadOutageSubmitsCancellationThenClearsWithoutCredit](#). Related findings: [L-43](#), [L-44](#).

fixed-PAG

[M-11] Users can farm keeper rewards by redeeming and rebalancing in a loop

Medium

Fixed

LOCATION

[BaseLeveragedVault.sol:confirmRebalance:494-522](#)

SUMMARY

Permissionless rebalancing is meant to restore the vault's target exposure and pay a keeper for useful corrective work. The problem is that target exposure is derived from the current `totalNav`, while the `HyperCore` position is read independently.

A redemption lowers `totalNav` immediately but leaves the asynchronous `HyperCore` position unchanged. This leaves the position temporarily larger than the new target, so the next rebalance enters the reductions-only phase and later pays out a reward:

```
reductionsOnly = _hasActionableReduction(p, totalTargetNotional, address(this));
...
if (achievedProgress < MIN_REBALANCE_PROGRESS_NOTIONAL) revert RebalanceNotSettled();
_payKeeperRewardUpTo(achievedProgress);
```

The reward is paid from accounted vault USDC, and nothing records who caused the NAV shrink. So the same user who redeemed can trigger the reduction, collect the reward, and redeposit.

IMPACT

An attacker can repeatedly transfer tracked vault USDC from the remaining shareholders to themselves.

Each redeem, rebalance, confirm, redeposit cycle nets a profit whenever the keeper reward exceeds the round-trip redemption/deposit fees and execution costs.

EXPLOIT PATH

1. Redeem shares for amount A via `CircuitBreaker.redeem`, reducing `accountedBalance` and `totalNav` while the HyperCore position stays unchanged.
1. Call `rebalance`; the lowered target makes the unchanged position overexposed, so a reduce-only order is submitted.
1. After the order fills, call `confirmRebalance` to satisfy the progress floor; `_payKeeperRewardUpTo` transfers reward R to the caller.
1. Redeposit the recovered USDC and repeat while R exceeds fees and execution costs.

RECOMMENDATION

Record the source of material NAV changes and suppress keeper compensation for a reduction attributable to the same caller's recent redemption. For example:

```
mapping(address => uint256) public lastRedeemBlock;
// in the routed redemption path
lastRedeemBlock[msg.sender] = block.number;
// before paying a rebalance reward
if (lastRedeemBlock[msg.sender] == rebalanceSubmitBlock) return 0;
```

A stronger design is to pay rewards only for progress caused by an independently observed position/risk event, excluding user-induced target changes from the rewardable progress calculation.

RESOLUTION

Ensemble

Commits `c19fad84387fe5ce2785fceb5143425e0982f631` and `fc11c2c33f7b218ab03e0d7e11a9f8cba2ae233a` fix M-11. The M-8 reward rule removes caller-directed compensation from every maintenance path. A redeeming shareholder may trigger a needed permissionless correction but cannot receive its reward. Proof case: [test/foundry/integration/BullVaultLifecycleIntegration.t.sol::test_M11_configuredRecipientPreventsRedeemRewardFarming](#). Related findings: M-8.

fixed-PAG

[M-12] Anyone can trigger a false emergency pause during a slow bridge recovery

Medium

Fixed

LOCATION

`BaseLeveragedVault:cancelStaleBridge/emergencyClose` + `LeveragedVaultLib:emergencyCloseStatus`:
309-333, 439-494, 648-672

SUMMARY

The EVM-to-Core bridge path optimistically removes USDC from the tracked EVM buffer before asynchronous HyperCore settlement becomes visible. If settlement is not observed by the timeout, stale cancellation reverses the accounting debit and records bridge recovery state, but the physical USDC has not yet returned to the vault.

The emergency condition treats this expected temporary mismatch as evidence of a loss. In the zero-equity case, `emergencyCloseStatus` only checks:

```
if (accountValue == 0) {
    if (marginUsed != 0) return (true, accountValue, marginUsed);
    uint256 currentBuffer = IERC20(usdcToken).balanceOf(address(this));
    return (currentBuffer < accountedBalance || continuation, accountValue, marginUsed);
}
```

Because `bridgePendingAmount` is never considered, the restored `accountedBalance` exceeds the current physical buffer while the bridge is still in flight, so the check reports an emergency even though nothing has actually been lost.

IMPACT

A slow but ultimately successful EVM-to-Core bridge can be turned into a permissionless false emergency pause.

This blocks normal vault operations and forces emergency recovery plus owner intervention, causing temporary vault-wide disruption even when the delayed bridge later settles correctly.

EXPLOIT PATH

1. `deployCapital()` starts a to-Core bridge; USDC leaves the EVM vault and `accountedBalance` is reduced.
1. Settlement stays invisible until the stale timeout, and `cancelStaleBridge()` restores `accountedBalance` while leaving bridge recovery pending.
1. The vault now shows zero Core equity, no margin used, and a physical USDC balance below the restored `accountedBalance`.
1. A permissionless `emergencyClose()` call passes `emergencyCloseStatus` and pauses the vault, forcing manual recovery.

RECOMMENDATION

Reject `emergencyClose` while bridge recovery is unresolved:

```
if (bridgePendingAmount != 0) {
    revert BridgeRecoveryPending();
}
```

Alternatively, pass bridge state into `emergencyCloseStatus` and exclude the EVM-buffer mismatch from emergency authorization whenever a to-Core bridge or its recovery record is active. In that state, emergency handling should require an independent margin or loss condition.

RESOLUTION

Ensemble

We added the missing `preBridgeEvmBalance == type(uint64).max` check in `4375341`. A Core spot-to-perps transfer now leaves the expected EVM balance unchanged during retained recovery because those funds never left the EVM wallet. The vault, Lens, and fee-liquidity calculations all pass the marker to the shared helper.

Tests verify that healthy recovery does not trigger a false loss check, a real EVM shortfall remains detectable, and ordinary bridge accounting is unchanged. This is also the fix for the M-04 follow-up.

fixed-PAG

[M-13] Dust USDC donations can freeze vault recovery and emergency withdrawals for 30 days

Medium

Fixed

LOCATION

`VaultBridgeOpsLib.prepareRejectedEmergencyBridgeRetry:20-45`

SUMMARY

When a Core-to-EVM bridge goes stale, the emergency retry helper is meant to tell a genuine late bridge receipt apart from unrelated USDC sitting in the vault, then clear stale bridge state so recovery can proceed. Instead it treats any raw balance increase as evidence of a bridge receipt.

After a stale bridge, any balance above `_preBridgeEvmBalance` makes `prepareRejectedEmergencyBridgeRetry` revert:

```
if (IERC20(usdcToken).balanceOf(address(this)) > _preBridgeEvmBalance()) {
    revert ILeveragedVaultTypes.BridgeRecoveryPending();
}
```

A direct token donation is indistinguishable from a partial late bridge under this check, and a single unit of USDC is enough to trigger the revert. The revert fires before the `withdrawable()` test that would otherwise show the bridge failed and funds are still recoverable on Core, and `clearBridgeState()` never executes on this branch. The positive balance delta never self-resolves, so the stale amount stays set until a genuine late settlement or the time-gated force-clear path.

IMPACT

A negligible-cost dust USDC donation can pin a vault in bridge-recovery state, blocking normal owner recovery and NAV-sensitive operations until the hard-coded 30-day force-clear escape hatch becomes available.

Emergency withdrawals stay disabled the entire time: `_enableEmergencyWithdraw` rejects a nonzero `bridgePendingAmount`, so pro-rata exits after a rejected bridge cannot be activated.

`BaseLeveragedVault._requireNormalNoPending` also remains blocked while the stale amount is uncleared.

The same blocked retry path is reached through `VaultAdminOpsLib._recoverEmergencyAssets`, so a dust donation also prevents ordinary emergency bridge-recovery retries. The net effect is that all vault holders can be temporarily frozen through emergency handling for up to the configured 30-day delay.

EXPLOIT PATH

1. A stale Core-to-EVM bridge is cancelled via `cancelStaleBridge()`, leaving a nonzero `bridgePendingAmount` in recovery state.
2. An attacker transfers one unit of the configured USDC directly to the vault, without touching any bridge state.
3. The owner invokes emergency recovery, but the raw-balance check reverts with `BridgeRecoveryPending`.
4. Emergency withdrawals cannot be enabled while the pending amount remains, so all users stay locked in the recovery flow until force-clear becomes available after the delay.

RECOMMENDATION

Separate verified bridge receipts from arbitrary token transfers before deciding whether recovery may clear. Let recovery clear stale state when the bridge outcome is not proven, while leaving any unsolicited surplus outside tracked accounting:

```
uint256 current = IERC20(usdcToken).balanceOf(address(this));
uint256 baseline = _preBridgeEvmBalance();
uint256 attributed = current > baseline ? current - baseline : 0;
attributed = Math.min(attributed, rejectedAmount);
clearBridgeState();
```

Reconcile only `attributed` and do not revert solely on an excess balance. Alternatively, add an owner-only emergency acknowledgement that records the excess as unsolicited funds and then clears recovery state. Either way, the excess must not be credited to NAV or allowed to block emergency withdrawal.

RESOLUTION

Ensemble

Commit [9245e7af5ff15d24c0e11016929a79dba79a8e56](#) fixes M-13. If the full recorded principal remains withdrawable on Core, rejected Core-to-EVM recovery ignores unrelated EVM balance changes. A dust donation can no longer block the retry path. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_M13_dustDonationDoesNotFreezeRejectedEmergencyBridgeRetry](#). Related findings: L-4, L-21.

fixed-PAG

[M-14] Attackers can drain the composer's cross-chain fee reserve and stall all settlements

Medium

Fixed

LOCATION

`CcipVaultComposer::handleDeposit:164-183`

SUMMARY

Cross-chain deposits and redeems settle on HyperEVM and send the resulting shares or assets back to the source chain. If settlement fails, the composer refunds the inbound token, but it pays the outbound CCIP fee from its own balance rather than from the request initiator.

The source router forwards user-selected minimum-output values without validating them against realistic settlement bounds. An impossibly large `minShares` or `minAmountOut` makes the settlement self-

call revert with `MinimumAmountOutNotMet`, and the composer then enters its refund branch. That branch calls `_sendToken()`, which charges the outbound fee from `address(this)`:

```
uint256 refundFee = ROUTER.getFee(destinationChainSelector, refundMessage);
```

The refund always returns the full inbound amount, so the caller recovers their principal while the shared fee reserve absorbs the send cost. This same fee-externalization path exists for both deposit and redeem settlement failures.

Because both success and failure responses are subsidized from the same composer balance, even ordinary permissionless low-value traffic erodes the reserve. A related timing angle makes it worse: the destination-to-source fee is quoted only at delivery time, so already-accepted messages can become unserviceable later under the same shared-reserve design.

IMPACT

A permissionless user can consume the composer's outbound native or ERC20 fee reserve without bearing the refund execution cost.

Once the reserve is exhausted, both refund sends and legitimate result sends revert with `InsufficientFeeBalance`. CCIP retries cannot progress while the composer stays underfunded, so the disruption is route-wide and persists until an operator replenishes the balance.

EXPLOIT PATH

1. The attacker calls `CcipSourceRouter.deposit(100 USDC, receiver, minShares)` with an impossibly large `minShares`.
2. `ccipReceive()` invokes `settleDepositFor()`, which reverts on `MinimumAmountOutNotMet` and enters the refund branch.
3. `_sendToken()` refunds the full USDC amount while charging the outbound fee from the composer's reserve; the attacker repeats this across many messages.
4. The reserve is depleted, so a legitimate settlement's outbound send reverts with `InsufficientFeeBalance` and stays pending until manual replenishment.

RECOMMENDATION

Require the initiating message to fund the possible refund fee, for example by including a response-fee reserve in the payload and transferring it into the composer before settlement:

```
uint256 refundFee = ROUTER.getFee(destinationChainSelector, refundMessage);
if (msg.value < refundFee) revert InsufficientFeeBalance(address(0), refundFee, msg.value);
```

Alternatively, reject requests whose minimum output cannot realistically be met before sending the source message, so intentional failures cannot be crafted:

```
if (minShares > previewDeposit/assets)) revert InvalidMinimumOut();
```

Either approach should be paired with treating caller-funded response fees as a prerequisite, deferring or rejecting refunds when they are unavailable.

RESOLUTION

Ensemble

The deducted skim now stays reserved for that message's executor in `ddd4c58`. The owner cannot sweep it. When the executor pays the current route fee and completes the return, they receive the reserved USDC skim. This also applies when a pending share return settles through emergency withdrawal.

If the send fails, the claim, skim reservation, and executor funds roll back together. Tests exercise native and ERC-20 fees, owner sweep attempts, failed sends, and emergency settlement, including checks that one return cannot spend another claim's funds.

fixed-PAG

[M-15] Deposits after vault insolvency can drain the emergency recovery pool from existing holders

Medium

Fixed

LOCATION

`BaseLeveragedVault:deposit:248-283`

SUMMARY

When the vault's real net asset value has fallen to zero, deposits are still accepted using a virtual-only price. `_navWithFees` continues to return a nonzero `navPerShare` derived from the virtual asset offset, and `deposit` uses it without any solvency check:

```
(uint256 grossNav, uint256 navPerShare) = _accrueAndGetNavPerShare();
shares = LeveragedVaultLib.computeDepositShares(assets, feeAmount, navPerShare, SHARE_SCALE);
accountedBalance = (uint256(accountedBalance) + assets).toUint64();
_mint(receiver, shares);
```

The virtual offset protects ordinary share conversion from first-depositor inflation, but it is not a physical asset available for emergency distribution. A tiny deposit made while `grossNav == 0` therefore mints a large share balance relative to the real assets backing them.

Those freshly minted shares are then treated identically to older shares by `_enableEmergencyWithdraw`, which snapshots the full physical USDC balance and the full total supply. Because the virtual offset inflates the attacker's shares but is never added to the physical emergency balance, a small post-insolvency deposit captures a disproportionate slice of the recovery pool.

IMPACT

A small deposit made after real NAV reaches zero can capture a substantial fraction of the vault's remaining emergency USDC, economically transferring recovery proceeds away from existing shareholders.

The same inflated post-insolvency share supply also dilutes future upside if the leveraged position later recovers, since those shares participate in the rebound.

EXPLOIT PATH

1. Existing shareholders hold a vault whose HyperCore deficit exceeds all positive assets, so `grossNav == 0`, while roughly 100 USDC is still physically available or recoverable.

2. While the vault is still `Normal`, the attacker deposits 1 USDC and receives approximately $1e18$ -equivalent shares against the 1 USDC virtual offset.
3. Emergency recovery completes and `_enableEmergencyWithdraw` snapshots roughly 101 USDC against the old supply plus the attacker's new shares.
4. The attacker calls `emergencyWithdraw` and receives approximately half of the snapshot pool despite contributing only 1 USDC.

RECOMMENDATION

Reject deposits when the real NAV is zero or otherwise insolvent:

```
(uint256 grossNav, uint256 navPerShare) = _accrueAndGetNavPerShare();
if (grossNav == 0) revert VaultDormant(grossNav, 1);
```

A stronger fix is to use a real-asset-based conversion for deposits while distressed and segregate recapitalization claims, so that new deposits made during distress cannot participate in the emergency pool on the same terms as pre-existing shares.

RESOLUTION

Ensemble

Commit `052d169f149c043baa7530c3727b6b338558c2dd` fixes M-15. Deposits are rejected when existing supply has zero real gross NAV or live Core equity is negative. A legitimate launch with zero supply remains valid. Proof case: [test/foundry/integration/VaultLensIntegration.t.sol::test_M15_existingSupplyAtZeroGrossNavRejectsDepositsEverywhere](#). Related findings: L-49.

fixed-PAG

[M-16] Emergency withdrawal permanently locks bridged holders' funds

Medium

Fixed

LOCATION

`BaseLeveragedVault:_update:748-756; BaseLeveragedVault:emergencyWithdraw:558-579;`
`VaultLockReleaseTokenPool:releaseTo:38-49`

SUMMARY

The emergency flow lets every canonical share holder exchange shares for their pro-rata portion of recovered USDC. Shares held by a lock/release pool represent remote (bridged) users, so they must stay claimable or be distributable through the pool.

The transfer hook freezes all ordinary transfers once the emergency snapshot is taken, including the pool's release transfer:

```
if (emergencyTotalShares != 0 && from != address(0) && to != address(0)) {
    revert EmergencyWithdrawAlreadyEnabled();
}
```

A remote user holds no canonical balance after bridging, so `emergencyWithdraw` rejects them directly. The pool's `releaseTo` cannot restore the balance either, because its `safeTransfer` reverts in `_update` (both addresses are nonzero). The pool has no emergency claim or distribution method of its own, and its

shares remain in `totalSupply`, so the snapshot denominator counts backing that can never be redeemed against.

Both canonical CCIP directions fail after the snapshot: remote holders cannot receive released canonical shares, and canonical holders cannot bridge shares out through the pool.

IMPACT

Remote CCIP holders whose canonical shares sit in a HyperEVM lock/release pool permanently lose access to their proportional emergency withdrawal value.

The stranded pool-held shares also keep total supply above zero, blocking emergency cleanup.

EXPLOIT PATH

1. A remote user bridges 1,000 shares; the lock/release pool receives the canonical shares and the user gets remote representations.
1. Emergency withdrawals are enabled, snapshotting total supply (including pool shares) and freezing transfers.
1. The remote user attempts a return transfer; `releaseTo` reaches `token.safeTransfer` but the vault's `_update` reverts because both addresses are nonzero.
1. Direct `emergencyWithdraw` also reverts since the remote user's canonical balance is zero, leaving the pool-held claim inaccessible.

RECOMMENDATION

Add a pool-specific emergency claim function that burns the pool-held shares and sends the proportional USDC to the authenticated remote recipient, for example:

```
function emergencyRelease(address receiver, uint256 shares) external onlyRouter {
    vault.emergencyWithdrawFor(address(this), receiver, shares);
}
```

Alternatively, allow only the configured lock/release pool to transfer after the snapshot, ensuring each such transfer is paired with a corresponding emergency claim accounting step rather than permitting unrestricted secondary-market transfers. A further option is to snapshot only locally claimable supply and segregate locked backing, instead of redeeming against raw `totalSupply`.

RESOLUTION

Ensemble

We reproduced the one-wei donation case and fixed it in `28ce63e`. When sealing, the pool includes any donated shares in the final balance and recalculates the commitment ID. `VaultAdmin` records that ID and enables withdrawals in the same transaction. A balance decrease or mismatched commitment still reverts, and the sealed balance must match exactly. The regression tests verify that the donation no longer blocks enablement and that the refreshed commitment accounts for the donated shares.

fixed-PAG

Low Severity

[L-01] Small leftover redemption deficits can block permissionless buffer refills and freeze redemptions

Low

Fixed

SUMMARY

VOL keeps 45% of NAV in its EVM redemption buffer and the rest on HyperCore. The only Normal-state permissionless Core-to-EVM refill is `withdrawCapital`. That function computes the buffer deficit, clamps the caller's request down to that deficit (and to the margin-safe capacity), and only then checks the result against `minDeployAmount`.

Because of the clamp, the value checked against the floor can never exceed the complete deficit. Writing `D` for the deficit, `M` for `minDeployAmount`, `C` for safe capacity, and `R` for the request, the guard receives `A = min(R, D, C)`, so `A <= D` always. When `0 < D < M`, every call reverts, including `withdrawCapital(type(uint256).max)` even when `C >= D`.

```
amount = LeveragedVaultLib.bridgeToEvmAmount(amount, needed, precompileTarget, BPS_DENOMINATOR, _emergencyMarginBps());
if (amount < minDeployAmount) revert InsufficientWithdrawable(needed, amount);
```

```
amount = requestedAmount > needed ? needed : requestedAmount;
if (amount > capacity) { amount = capacity; }
```

Redemptions can only be paid from tracked EVM liquidity, so a blocked refill directly blocks exits:

```
if (spendableBuffer < assets) revert InsufficientBuffer(assets, spendableBuffer);
```

The shares stay fully backed by Core equity, but that backing cannot satisfy `redeem` until it is bridged to the EVM.

With an empty buffer the affected zone is bounded: `D = 45% * residualNAV` and `D < $25` gives `residualNAV < $55.555555`, which sits inside VOL's dormant range below `minViableAum = $50`. A sub-`$25` deficit can also arise at larger NAV whenever the buffer is close to target. The enforced relation `minDeployAmount <= minViableAum` does not prevent the contradiction between the 45% buffer target and the \$25 floor.

A read-only sample returned `minDeployAmount = $25`, `minViableAum = $50`, roughly \$118 of NAV, an EVM balance of \$54.473701, and a 94.664586% dominant shareholder. That concentration makes the dead zone reachable through ordinary redemption and refill rounds even though sampled NAV starts above it.

IMPACT

Valid principal can be left without any permissionless recovery. In the reproduced trace a third party's \$5.976018 redemption reverts `InsufficientBuffer(5976018, 293434)` while \$36.140174 remains on Core and the EVM buffer holds only \$0.293434.

Recovery is neither permissionless nor automatic. It requires the owner to lower `minDeployAmount` (or invoke a privileged recovery workflow), after which the same public refill bridges the funds and the redemption succeeds.

The blocked call is exactly the remedy the protocol documentation prescribes: it states redemptions draw only on the EVM buffer, that an insufficient buffer reverts, and that a keeper must then call `withdrawCapital()` to bring funds back from HyperCore. `minDeployAmount` is documented as a guard against tiny uneconomic deployments, but the implementation reuses it as a hard floor after clamping Core-to-EVM withdrawals, disabling the documented redemption recovery path in the sub-floor deficit range.

The triggering actions need not be profitable and need not touch anyone else's shares. A dominant holder's ordinary exit or the tail of a redemption wave is enough. Absolute exposure is capped by the \$55.555555 empty-buffer residual bound, and a concrete trace may affect less because EVM dust remains and the triggering holder can keep residual shares.

EXPLOIT PATH

1. A dominant holder and an independent holder both hold VOL shares while the vault is in Normal state.
2. The dominant holder redeems against the EVM buffer; public `rebalance` resizes to the lower NAV, creating a \$28.812926 deficit that exceeds the floor, so public `withdrawCapital` refills normally.
3. The dominant holder redeems again, leaving NAV at \$36.070173 and only \$0.293434 of EVM liquidity. NAV is now below `minViableAum`, so `rebalance` takes the dormant-flatten branch and the full Core balance becomes withdrawable.
4. The independent holder's \$5.976018 redemption reverts `InsufficientBuffer(5976018, 293434)`.
5. `withdrawCapital(type(uint256).max)` reverts `InsufficientWithdrawable(15938143, 15938143)`: the clamped amount equals the complete deficit and capacity is larger, so the \$25 floor is the only blocker. No request bypasses it, and neither `deployCapital`, a settled dormant close, donations, nor new deposits can restore aggregate exit capacity.

RECOMMENDATION

Treat `minDeployAmount` as a dust guard for partial requests, not as a prohibition on clearing the complete remaining deficit:

```
-   if (amount < minDeployAmount) revert InsufficientWithdrawable(needed, amount);
+   if (amount == 0) revert InsufficientWithdrawable(needed, amount);
+   if (amount < minDeployAmount && amount < needed) {
+       revert InsufficientWithdrawable(needed, amount);
+   }
```

This permits a sub-minimum bridge only when it clears the complete deficit and safe capacity is sufficient. If the policy also requires monotonic recovery when capacity is below the deficit, extend the sizing function to identify and exempt full safe-capacity withdrawals.

Also enforce a launch invariant such as `bufferBps * minViableAum / 10_000 >= minDeployAmount`. This blocks the dormant-range contradiction, though it is only defense in depth because NAV can still fall below `minViableAum`.

The regression property to keep: `needed > 0 and capacity >= needed` must let `withdrawCapital(type(uint256).max)` start a full-deficit refill. The fix requires upgrading the vault implementation but changes no storage layout.

RESOLUTION

Ensemble

Commit `cfb4b2f0e0afd1df555f5e4e6f48caed81edfd9f` fixes L-1. The linked withdrawal-sizing operation accepts a sub-minimum amount only when that amount clears the entire deficit. Zero amounts and caller- or capacity-limited partial refills below the minimum still revert. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L1_withdrawCapitalBridgesAnExactSubMinimumBufferDeficit`.

fixed-PAG

[L-02] Attacker can freeze the vault for 30 days by shrinking bridge settlement tolerance

Low

Fixed

SUMMARY

`deployCapital` is permissionless, and its first branch bridges whatever is sitting in the vault's HyperCore spot balance, with no upper or lower shaping beyond `minDeployAmount`.

HyperCore spot balances can be credited by anyone via a permissionless `spotSend`, so an attacker fully controls `bridgePendingAmount` for that cycle. The recovery branch takes that attacker-controlled balance directly as the bridge amount, before any sizing policy or dormancy gate:

```
uint256 spotAmount = LeveragedVaultLib.coreSpotUsdc(precompileTarget, address(this));
if (spotAmount >= minDeployAmount) {
    _startCoreSpotRecovery(spotAmount);
    emit BridgeToCoreSubmitted(spotAmount, block.number);
    return;
}
```

`maxDeployAmount` is applied only on the regular deploy path further down, never here.

The settlement tolerance is a relative 1% of the bridged amount, so choosing the smallest admissible amount (`minDeployAmount`) shrinks the tolerance to `0.01 * minDeployAmount`, a few dollars. Meanwhile the vault's ambient `rawUsd` noise (hourly funding on the full leveraged notional) is orders of magnitude larger. With `minDeployAmount = 100e6`, tolerance is `1e6` (\$1), while a vault holding \$255k of notional pays far more than \$1 in a single funding tick, so ordinary market movement causes confirmation to miss the threshold.

Once that happens, the to-Core recovery path has no practical retry. `reconfirmBridge` reuses the same frozen `rawUsd` target, emergency retry rejects to-Core recoveries, and force-clear requires Emergency state plus a 30-day delay. The `type(uint64).max` sentinel stored by `startCoreSpotRecovery` short-circuits only the accounting branch of `reconfirmBridgeAccountedBalance`, and only after `_verifyBridgeOutcome` has already passed, so it provides no alternative evidence path.

IMPACT

An attacker can brick the vault for a minimum of 30 days at a cost of `minDeployAmount` (\$100 at the reference config) plus gas.

While frozen, all deposits, redemptions, fee collection, capital movement, and rebalancing are blocked for LPs. The freeze persists because the recovery predicate keeps using the stale absolute `rawUsd` target and the emergency retry path refuses to handle to-Core recovery, so the owner must eventually use the force-clear path. The attack is repeatable after each force-clear.

EXPLOIT PATH

1. Wait for the vault to be in `Normal` state with no pending bridge or rebalance.
2. `spotSend` exactly `minDeployAmount` USDC to the vault's HyperCore spot balance (permissionless; the donation is retained by the vault).
3. Call `deployCapital()`, ideally timed just before an hourly funding boundary, so the recovery bridges the minimal amount with a \$1 tolerance.
4. Ambient `rawUsd` funding movement exceeds the tiny tolerance, confirmation fails, and the vault is stuck until a 30-day force-clear.

PREREQUISITES

The vault must be in `Normal` state with no pending bridge or rebalance. Both `spotSend` and `deployCapital` are permissionless, so no privileged role is required.

RECOMMENDATION

Apply the same sizing policy to the Core spot recovery branch that governs the regular deploy path, so the bridged amount cannot be driven down to `minDeployAmount` by a donated balance. In particular, clamp the recovery amount with `maxDeployAmount` (or size the bridge against the vault's own accounting rather than the raw spot balance).

Also reconsider the relative 1% settlement tolerance: enforce a sensible absolute floor on tolerance so ordinary funding noise on the vault's notional cannot exceed it, and provide a viable retry path for to-Core recoveries that does not depend on a stale frozen `rawUsd` target or require a 30-day force-clear.

RESOLUTION

Ensemble

Commit `e3a17a425afeaf6d2412d081ba35b778073f44dd` fixes L-2. Settlement tolerance is calculated from tracked principal, and its absolute allowance is capped at half of the request. An unrelated donation therefore cannot shrink the tolerance or prolong recovery. Proof case: `test/foundry/unit/BridgeSettlementFloor.t.sol::test_L2_bullToCoreSettlementFloorIsSharedByWriteAndLens`. Related findings: L-30, L-33, L-36, L-37.

fixed-PAG

[L-03] Redeemers exit at gross NAV, leaving position unwind costs to remaining holders

Low

Fixed

SUMMARY

Share price is derived from HyperCore's gross `accountValue`, which counts collateral, unrealized PnL, and settled funding. NAV reserves nothing for the cost of later resizing or flattening the leveraged perp position, such as taker fees and execution slippage.

```
function _grossNav(address precompileTarget, uint256 accountedBalance) private view returns
(uint128 grossNav) {
    (int64 accountValue,,, ) = PrecompileLib.accountMarginSummary(precompileTarget, 0, address(this))
;
    ...
}
```

That gross value flows into `navPerShare` with only treasury and management fee adjustments, and `redeem` pays the user from the EVM buffer at that price while the Core notional is left unchanged in the same transaction. The exiting user therefore never bears their share of the future unwind cost.

After a user redeems R dollars, the next rebalance must reduce target notional by roughly $R \times L$ (where L is realized leverage), so the deferred unwind cost left to remaining holders is about $R \times L \times \text{costRate}$, or $L \times \text{costRate}$ per redeemed dollar. At the $6x$ target and a 3.5 bps taker cost that is around 21 bps per dollar; under stress with $7.46x$ leverage and 25 bps execution cost it rises to about 186.5 bps per dollar.

Redemption is not blocked just because emergency thresholds are already met. It is only blocked once `emergencyClose()` has actually run and `lastEmergencyTimestamp` is set:

```
if (vaultState == VaultState.Emergency) revert InvalidState(vaultState, VaultState.Normal);
if (lastEmergencyTimestamp != 0) revert EmergencyCloseActive();
```

So in a stressed state where NAV is about $\$826k$ against $\$6.17M$ notional (roughly $7.46x$ leverage and 18.4% free margin, already below the 20% emergency threshold), `emergencyClose()` is eligible but `redeem` still succeeds until someone calls it. With a 45% EVM buffer, a large holder can pull out a meaningful amount and leave the full Core position and its unwind cost behind.

IMPACT

Remaining LPs absorb HyperCore unwind costs for capital that has already exited the vault.

Under stress, a large holder can redeem against gross NAV before the vault deleverages, pushing the cost of resizing or flattening the Core position onto the holders who stay.

The gain is bounded by the available EVM buffer and the exiting holder's pro-rata share, so it is not an unbounded drain. It does create a clear first-mover advantage for anyone who redeems before Core exposure is reduced.

EXPLOIT PATH

1. Wait for a stressed state where realized leverage is elevated and free margin has fallen below the emergency threshold, but `emergencyClose()` has not yet been called (`lastEmergencyTimestamp == 0`).
2. Ensure no bridge or rebalance is pending and the vault is in `Normal` state.
3. Call `redeem` for a large share amount; receive assets at gross NAV from the EVM buffer while the Core notional stays unchanged.
4. Remaining holders later bear the taker fees and slippage when the position is resized or flattened.

PREREQUISITES

VolVault in `Normal` state, no bridge or rebalance pending, `lastEmergencyTimestamp == 0`, an open Core position with meaningful notional, and an EVM buffer large enough to service the redemption. The stressed variant additionally requires elevated realized leverage and redeeming before `rebalancePending` is set or `emergencyClose()` closes the window.

RECOMMENDATION

Reserve an estimated unwind cost when computing NAV so redeemers bear their pro-rata share of the flatten/resize cost. Derive the reserve from current notional and realized leverage times an expected taker-fee/slippage rate, and subtract it from `grossNav` before converting to `navPerShare`.

Alternatively, block or throttle redemptions once emergency thresholds are met (free margin below the emergency level), rather than only after `emergencyClose()` has been called, so exits cannot front-run a deleverage.

RESOLUTION

Ensemble

The original guard only covered stressed exits. In [225ae09](#), we applied the report's primary recommendation: deduct the estimated unwind cost from shareholder NAV using current HyperCore notional and the configured execution-fee and slippage rates. Redeemers now bear their share of that estimate even outside the stressed band.

Deposit caps still use physical gross NAV. In Standard mode, the reserve cannot exceed positive Core equity, so it cannot consume separately held EVM or spot assets. Tests check the pro-rata cost as a position unwinds, default and configured rates, deposit solvency, and the Standard-mode equity limits. The stressed-exit guard is unchanged.

fixed-PAG

[L-04] Keeper lens can leave the vault stuck in recovery longer than needed

Low

Fixed

SUMMARY

For the Core-to-EVM case, the lens uses a two-sided test that checks both the Core side decreased and the EVM side increased:

```
bool coreDecreased = int256(currentRawUsd) <= int256(preBridgeRawUsd) - int256(expectedAssets);
bool evmIncreased = currentEvmBalance >= uint256(preBridgeEvmBalance) + expectedAssets;
return coreDecreased && evmIncreased;
```

The write path, `reconfirmBridgeAccountedBalance`, clears the recovery once the observed EVM receipt reaches the required fraction of the pending amount. It does not re-check the Core-side value for this branch.

The Core margin value moves with funding and PnL, so after a delayed bridge it can rise back above the threshold while the EVM funds have already arrived. In that state the lens returns `needsReconfirmBridge = false`, yet a direct `reconfirmBridge()` call would succeed and clear the recovery.

IMPACT

Keeper automation that follows the lens can leave the vault in recovery longer than necessary, which blocks deposits, redemptions, and capital operations.

The reconfirmation function is permissionless, so any caller can clear the state directly. This limits the impact to automation and liveness.

RECOMMENDATION

Have the lens apply the same condition the write path uses for the Core-to-EVM case, or surface a separate flag that distinguishes strict settlement verification from the receipt-backed reconfirmation the write path already accepts.

RESOLUTION**Ensemble**

Commits `6a437ea089c58842614c1a4cbe6860590935560b` and `b91afe1fdcfee2d03a49bf542089057497d065f2` fix L-4. Keeper readiness and recovery execution use the same Core-spot and EVM-receipt checks, so the Lens cannot keep advertising recovery after the executable condition has cleared. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L4_coreToEvmRecoveryReadinessTracksExecutionDespitePositiveCashDrift`. Related findings: L-21, M-13.

fixed-PAG

[L-05] Empty order book on an unrelated perp blocks buffer deployment and margin recovery

Low Fixed

SUMMARY

`deployCapital()` is the permissionless bridge that moves excess EVM buffer USDC into the vault's HyperCore margin account. It places no orders and needs no order-book data, yet it reverts in full whenever either side of any configured perp's book is empty.

The sizing path is the culprit. `capitalDeployAmount` (`src/libraries/LeveragedVaultLib.sol:L419-L433`) always calls the margin recovery helper:

```
deployAmount = _bufferDeployAmount(...);
uint256 marginRecovery = _marginRecoveryDeployAmount(usdcToken, accountedBalance, minDeployAmount,
p, account);
deployAmount = Math.max(deployAmount, marginRecovery);
```

`_marginRecoveryDeployAmount` (L469-L493) takes a full rebalance margin preview at L479 ahead of its own early exits. `_rebalanceMarginPreview` (L1233-L1256) walks every configured perp via `_independentlyFillableIncreaseMargin` (L1264-L1279), calling `_getAssetDelta(..., false)` on each. The trailing `false` (`enforceDriftThreshold`) routes into the cap helper, which reads that perp's top of book:

```
uint256 spreadBps_ =
    p.maxRebalanceSpreadBps == 0 ? 0 : VaultExecutionPolicyLib.spreadBps(p.precompileTarget,
assetId);
```

`spreadBps` forwards the raw BBO to `_spreadBps` (`src/libraries/VaultExecutionPolicyLib.sol:L287-L292`), which reverts rather than degrading:

```
if (bid == 0) revert ILeveragedVaultTypes.MissingPrice(assetId);
if (ask == 0) revert ILeveragedVaultTypes.MissingPrice(assetId);
```

Crucially, the spread value read this way is then discarded. `_adaptiveMaxRebalanceOrderNotional` (L201-L248) returns before its spread block whenever the vault is uncapped:

```
uint256 aumCap = autoRebalanceCapBps == 0 ? 0 : (totalTargetNotional * autoRebalanceCapBps) /
BPS_DENOMINATOR;
if (maxRebalanceOrderNotional == 0 && aumCap == 0) return (0, LIMIT_NONE);
```

No execution-policy value switches the read off. `_maxRebalanceSpreadBps()` rewrites a stored zero to the 100 bps default, and every permitted nonzero value keeps the read armed, so an operator cannot disable it through configuration. On the observed staging vaults (block 41,409,356) both cap fields decoded to zero, meaning the spread that could revert the call was never consumed in the result.

With production BEAR parameters in a Normal state holding `$398.532478` NAV and an `$85.085129` deployable surplus, an empty SOL bid or offer makes `deployCapital()` revert `MissingPrice(5)`, even though SOL is neither source nor destination of the bridge and BTC and ETH remain readable. Restoring the SOL book lets the same call bridge the full `$85.085129`, and the amount is identical at a 50 bps and a 5000 bps SOL spread, confirming the spread never enters the result.

IMPACT

This is a conditional liveness defect, not theft. Principal stays solvent and fully accounted, share accounting is untouched, and deposits and redemptions remain available. The damage is denial of the vault's buffer deployment and margin-recovery sizing while an unrelated perp's book is unreadable.

Four consequences follow:

- **Idle collateral.** The surplus stays on the EVM side, so the strategy runs below target exposure while holding more buffer than its own policy calls for. The blocked amount scales with NAV and `bufferBps`, bounded by funds available for deployment.
- **Margin recovery denied.** `_marginRecoveryDeployAmount` exists to bridge enough USDC for a projected rebalance to clear the 30% free-margin floor. The same unconditional book traversal runs first, so a margin-deficient vault cannot bridge the collateral that would restore it. This follows from the shared source path, not a separate stressed-margin run.
- **Keeper observability lost.** `VaultLens.getKeeperActions` (`src/periphery/VaultLens.sol:L1108-L1121`) reaches the same sizing, so the read-only view a keeper polls reverts identically. Automation cannot tell an outstanding deploy apart from a vault that needs nothing.
- **No execution-policy opt-out.** Since no permitted `maxRebalanceSpreadBps` value disables the read, recovery requires the book to become readable, a portfolio transition that safely removes the affected asset, or a code upgrade.

The coupling also sits outside the documented scope of the control: BBO spread checks are documented inside the rebalance execution-policy stack, while `deployCapital` is documented purely as permissionless buffer movement with no order-book dependency.

Related finding M-119 covers the same strict spread helper on the `rebalance()` order-construction path, which legitimately needs a valid book. The path here builds no order and the uncapped branch returns before consuming the spread, so the entry point, the discarded read, and the minimal fix are distinct. A broad fix that filters unavailable books could close both.

EXPLOIT PATH

1. A vault is in a Normal state with a deployable buffer surplus (or a margin deficit requiring recovery).
2. One configured perp's book has an empty bid or offer (a naturally empty side, a sweep that leaves one side empty, or an attacker controlling the remaining side).
3. The BBO precompile returns zero for the absent side.
4. Any call to `deployCapital()`, and the keeper's `getKeeperActions` view, reverts `MissingPrice` deterministically until the book becomes two-sided.

The local PoC reaches the condition by cancelling the only resting SOL offer and then the only resting SOL bid. This establishes the vault branch but does not demonstrate that an arbitrary attacker can remove better orders owned by honest makers on a liquid live book.

PREREQUISITES

The behavior is conditional on the HyperEVM BBO reader returning zero for a missing book side. Hyperliquid documents empty books as a supported market state (with `allMids` falling back to last trade price), but does not specify how the BBO precompile encodes a missing side; the source treats a zero bid or offer as `MissingPrice`. No crossed-book state is required. Production reachability is bounded by the ability to leave one side of a configured perp's book empty; arbitrary control of a liquid live book is not demonstrated.

RECOMMENDATION

Read the spread lazily. `_adaptiveCapForOrder` should evaluate the uncapped early return before calling `spreadBps`, so a book is consulted only when an order cap actually consumes the result. `capitalDeployAmount` may additionally skip the margin preview when it cannot affect buffer sizing.

Alternatively, treat an unreadable asset as unavailable within the preview rather than reverting the whole call. Keep `enforceRebalanceSpread` strict on the order-construction path, where a valid book is genuinely required.

RESOLUTION

Ensemble

Commit `22a04ff7a60eb0afd97fef4c5ea5e8fffcbbff4e7` fixes L-5. The shared policy first determines whether an asset needs an order and reads that asset's book only then. An empty book for an unrelated asset no longer blocks capital deployment. Proof case: `test/foundry/unit/LibraryBehavior.t.sol::test_L5_unrelatedMissingBookDoesNotBlockCapitalDeploySizing`. Related findings: L-14.

fixed-PAG

[L-06] Vault NAV is understated when a perp position is underwater, letting depositors buy in at a discount

Low

Fixed

SUMMARY

`LeveragedVaultLib._grossNav` treats a negative `accountValue` reported by HyperCore's perp margin account as a real liability and nets the deficit against the vault's positive assets (`spotUsdc + accountedBalance`):

```
if (accountValue >= 0) {
    grossNav = (positiveAssets + uint256(uint64(accountValue))).toUint128();
} else {
    uint256 deficit = uint256(-int256(accountValue));
    grossNav = positiveAssets > deficit ? (positiveAssets - deficit).toUint128() : 0;
}
```

Under HyperCore's margin architecture this deduction is not a claim the protocol will ever enforce against the vault's assets. The perp clearinghouse and the spot clearinghouse are independent ledgers with no

cross-clearinghouse recourse, and `accountValue` reflects only the perp-side account (margin plus unrealized PnL).

A deficit beyond the account's margin is absorbed by the protocol backstop (insurance fund, then auto-deleveraging); an account liquidated past insolvency settles with zero equity and is never charged against its spot balance or against HyperEVM state, which no Core liquidation path can reach.

The negative branch therefore over-deducts: computed NAV falls below the `accountedBalance + spotUsdc` floor that a settlement can never actually go below, and NAV then jumps upward once a losing position is settled, which is self-contradictory.

The same incorrect branch is duplicated in `VaultLens._grossNav` and in `CircuitBreaker`, so all NAV consumers inherit it.

IMPACT

While `accountValue < 0`, NAV is understated and shares are priced below the true settlement floor (`accountedBalance + spotUsdc`).

Redemptions are underpriced, and a depositor can enter at a discount and profit when NAV normalizes after the perp position settles. The gain is transferred from existing holders.

In the worked example, existing holders' value drops from 100 to 75 while a fresh 50 USDC deposit returns roughly 50% profit.

EXPLOIT PATH

1. State: `accountedBalance = 100`, `spotUsdc = 0`, open leveraged position with unrealized PnL `= -250`, giving `accountValue = -50`. `vaultState == Normal`, so `deposit` is callable and `_grossNav` computes `max(100 + 0 - 50, 0) = 50`.
2. Assuming the deposit fits active caps, zero deposit/redeem fee, and a vault large enough that virtual NAV offsets are negligible, the depositor sends 50 USDC at `navPerShare = 50·1e18/S` and receives `~ S` shares against pre-existing supply `S`.
3. HyperCore backstop-liquidates the position: realized loss exceeds the perp margin, the excess deficit is absorbed by the protocol backstop, and the account's equity settles at zero. No `emergencyClose` was called, so `vaultState` stays `Normal` and the depositor can redeem directly.
4. `accountedBalance` is now 150 and `accountValue = 0`; NAV normalizes to `150` against supply `2S`, so `navPerShare = 75·1e18/S` and the depositor's `~ S` shares are worth about 75 USDC.

RECOMMENDATION

Treat negative Core perp equity as zero rather than as a liability against other assets, keeping EVM USDC unaffected, equivalently `grossNav = accountedBalance + spotUsdc + max(accountValue, 0)`:

```

    if (accountValue >= 0) {
        grossNav = (positiveAssets + uint256(uint64(accountValue))).toUint128();
    } else {
-       uint256 deficit = uint256(-int256(accountValue));
-       grossNav = positiveAssets > deficit ? (positiveAssets - deficit).toUint128() : 0;
+       grossNav = positiveAssets.toUint128();
    }

```

Apply the same correction at all three sites: `LeveragedVaultLib._grossNav`, `VaultLens._grossNav`, and `CircuitBreaker`.

RESOLUTION

Ensemble

Commit `460389773034a3f04335c975ab66d2f818a1ea56` fixes L-6. While paused and free of pending work, the owner requests Standard account mode and confirms it in a later block after the Info API reports `userAbstraction == "disabled"`. Until confirmation, NAV-priced deposits, redemptions, fees, and capacity views fail closed. Once confirmed, negative perp equity cannot offset separately accounted EVM or Core spot assets. Proof case: `test/foundry/unit/StandardAccountMode.t.sol::test_L6_unconfirmedVaultRejectsNavPricedDepositsAndRedemptions`.

fixed-PAG

[L-07] Deploying buffer into an insolvent margin account destroys protected vault funds

Low

Fixed

SUMMARY

`deployCapital` moves idle EVM buffer USDC into the HyperCore perp margin account. It takes the larger of two deploy amounts, and the buffer excess amount has no account-health check:

```
deployAmount = _bufferDeployAmount(...);
uint256 marginRecovery = _marginRecoveryDeployAmount(...);
deployAmount = Math.max(deployAmount, marginRecovery);
```

`_marginRecoveryDeployAmount` deliberately refuses to cure an insolvent account (`preview.accountValue < 0` returns `0`), but the `Math.max` keeps the buffer excess anyway. So `deployCapital` still bridges `accountedBalance totalNav·bufferBps` into a negative-equity margin account.

Under HyperCore's margin architecture the perp clearinghouse has no recourse to HyperEVM state. A deficit beyond the account's margin is absorbed by the protocol backstop (insurance fund, then ADL), and a liquidated account settles with zero equity. The EVM buffer is unreachable by any Core liquidation path, so bridging it into an account with `accountValue < 0` converts protected funds into margin that the imminent backstop liquidation consumes.

IMPACT

The bridged buffer is destroyed value. Example with `accountedBalance = 150`, `accountValue = -100`, `bufferBps = 15%`:

- Bridging: `grossNav = max(150100, 0) = 50`, `targetBuffer = 7.5`, `excess = 142.5` bridged, leaving `accountedBalance = 7.5` and `accountValue = +42.5`. The account is liquidated and the margin is consumed, leaving vault value **7.5**.
- Not bridging: the account is liquidated at zero equity, the deficit is backstopped, and the EVM buffer is untouched, leaving vault value **150**.

RECOMMENDATION

Skip the deploy when the margin preview reports an insolvent account. `_marginRecoveryDeployAmount` already computes this signal, so gate the buffer deploy on the same `accountValue < 0` condition rather than letting `Math.max` override it.

RESOLUTION

Ensemble

Commit [a6b39a64ec8ab80292fb6432264ee146225f1221](#) fixes L-7. The combined deploy calculation returns zero whenever live Core equity is negative, including valid configurations whose rounded target notional is zero. Buffer capital therefore cannot be sent into an insolvent margin account. Proof case: [test/foundry/unit/NegativeEquityDeploy.t.sol::test_L7_negativeCoreEquityBlocksBufferDeploymentAcrossBullBearVol](#). Related findings: L-19.

fixed-PAG

[L-08] Capital can be deployed into a vault that the keeper reward pushes below the dormant threshold

Low Fixed

SUMMARY

`deployCapital` checks the dormant gate on the NAV snapshot taken before the keeper reward is paid, but the reward debits `accountedBalance`, which is a component of NAV.

```
(uint256 totalNav,) = computeNav();
if (totalNav < minViableAum) revert VaultDormant(totalNav, minViableAum);

_payKeeperReward();
```

`_payKeeperReward` settles through `keeperRewardStatus`, which subtracts the reward from `accountedBalance`. Since `totalNav = spotUsdc + accountedBalance + accountValue - accruedFees`, the real NAV falls by exactly the reward amount after the gate has already passed.

The deploy is then sized from the pre-reward `totalNav` while `accountedBalance` reflects the post-reward value, mixing inputs from two different points in the flow.

IMPACT

When the pre-reward NAV lies in the range `[minViableAum, minViableAum + keeperReward)`, paying the reward drops the actual NAV below `minViableAum`, yet `deployCapital` still bridges the buffer excess into HyperCore.

This violates the design intent that a vault below `minViableAum` is dormant and may only deleverage: `rebalanceMultiAsset` flattens reduce-only and refuses any expansion, while `deployCapital` here injects fresh capital into a vault that should be treated as non-viable.

RECOMMENDATION

Pay the keeper reward first, then recompute NAV and re-check the dormant gate, so the deploy only proceeds when the post-reward vault is genuinely viable:

```
+ _payKeeperReward();

(uint256 totalNav,) = computeNav();
if (totalNav < minViableAum) revert VaultDormant(totalNav, minViableAum);
```

```
- _payKeeperReward();
```

RESOLUTION

Ensemble

Commits `c19fad84387fe5ce2785fceb5143425e0982f631` and `2124fe2ba44ce536d16a7953a14bb95d504ec5be` fix L-8. M-8 removes the reward for deployment submission, so viability checks, accounting, and sizing all use the same pre-settlement balance. A reward cannot push the vault below the dormant threshold after an otherwise valid deploy. Proof case: `test/foundry/integration/BullVaultLifecycleIntegration.t.sol::test_L8_dormantBoundaryDeployKeepsRewardOutOfPrincipal`. Related findings: M-8.

fixed-PAG

[L-09] NAV counts an unwithdrawable reserve, shortchanging the last redeemers

Low Fixed

SUMMARY

`_grossNav` prices the vault's net asset value including `USDC_SPOT_RESERVE` (2.1 USDC), the Core->EVM bridge-gas reserve. This reserve is not always on Core spot: `deployCapital`'s spot recovery moves it into perp margin (`BaseLeveragedVault.sol:415-419`), and NAV counts it in either location, via `spotUsdc` while on spot or via perp account value once in margin:

```
uint128 spotUsdc =
    (PrecompileLib.spotBalance(precompileTarget, address(this), HYPERCORE_USDC_TOKEN) / 100).toUint128();
uint256 positiveAssets = uint256(spotUsdc) + accountedBalance;
```

Every extraction path treats that same 2.1 USDC as unavailable. `_safeBridgeToEvmCapacity` subtracts the reserve deficit from the Core withdrawable cap (`LeveragedVaultLib.sol:240-243`), `_submitBridgeToEvm` tops Core spot back up to the reserve after each `sendAsset` (`LeveragedVaultOpsLib.sol:112-117`), and `_enableEmergencyWithdraw` snapshots only the EVM token balance (`VaultAdminOpsLib.sol:312`). The reserve is therefore included in `navPerShare` but can never be bridged back to the EVM or enter any withdrawal pool.

IMPACT

The vault's realizable assets are permanently `totalNav 2.1 USDC`, while every `navPerShare`-priced withdrawal assumes the full `totalNav`. When a vault is fully withdrawn, the 2.1 USDC gap is absorbed by whoever exits last.

Let $N = \text{totalNav}$ (includes the 2.1 reserve), S = total supply, holder A exits with a shares before an emergency close via `redeem`, and holder B exits with the remaining b shares ($a + b = S$) via `emergencyWithdraw`:

- A pays `navPerShare = N/S` and receives `a·N/S`, of which `2.1·a/S` is the inflated reserve.
- The emergency snapshot holds `N 2.1 a·N/S` USDC (EVM balance only), so B receives `N 2.1 a·N/S`, which is `2.1` below the `b·N/S` the share price promised. Of that gap, `2.1·a/S` was transferred to A and the remaining `2.1·b/S` is stranded in Core with no claim path.

The same skew occurs without any emergency: once earlier redeemers have taken their full share-priced amount, the remaining buffer is `2.1` short of what the last holder's shares are worth, so the last redemption reverts `InsufficientBuffer` (`BaseLeveragedVault.sol:395-397`) or pays `2.1` less. Value is structurally transferred from the last exit to the first, and the stranded fraction has no recovery path.

The absolute amount is fixed at 2.1 USDC and cannot be amplified, so the relative loss scales inversely with vault size: negligible for large vaults, notable for small ones (about 1% at a 210 USDC NAV).

RECOMMENDATION

Exclude the unwithdrawable reserve from NAV so pricing matches realizable assets. Track the reserve as a locked asset that is neither priced into shares nor swept by recovery.

RESOLUTION

Ensemble

Commit `2c31a289773c515d3e65ef94f5118e6b3bc37b9d` fixes L-9.

`HyperCoreBridgeConfig.reserveAwareGrossNav` subtracts `min(effectiveReserve, coreSpot + positivePerpEquity)` from gross NAV. Only the funded, unwithdrawable reserve is excluded; an unfunded reserve does not reduce shareholder assets. Proof case: `test/foundry/unit/HyperCoreBridgeConfig.t.sol::test_L9_reserveAwareGrossNavExcludesOnlyTheFundedReserve`. Related findings: L-31, L-46.

fixed-PAG

[L-10] Rebalance orders ignore taker fees, leaving the vault over-leveraged with a thinner margin buffer

Low

Fixed

SUMMARY

Rebalance order sizing and the margin preview both model a fill as costless. `_buildOrderPlacement` sizes an order so that `size * limitPx / 1e10 <= absDelta` (`LeveragedVaultLib.sol:1322-1346`), and `_rebalanceMarginPreview` projects the post-trade free-margin floor (`LeveragedVaultLib.sol:1233-1257`). Neither reserves the execution fee.

Every rebalance fill is a taker execution. The limit is derived from the best offer/bid, so an in-band order crosses the spread immediately. HyperCore then deducts the taker fee (0.045% of fill notional at the base tier per the Hyperliquid fee schedule) from the account's cash on every fill, reducing `accountValue`, which is exactly the figure `_grossNav` prices into NAV (`LeveragedVaultLib.sol:1543-1557`).

The target, the order size, and the margin preview are all computed from the pre-fee state, so the state reached after the fill never matches what any of them assumed.

IMPACT

Every taker-filled rebalance lands in a state that differs from what each computation assumed, in two directional ways:

Resulting notional is higher than target. Orders are sized to bring the position to `totalTargetNotional = totalNav * targetLeverage` (`LeveragedVaultLib.sol:1203-1209`). When the order fills, the fee lowers `accountValue`, so actual NAV is below the NAV used to size the order. The position lands at the old target while the correct target has moved down by the fee, leaving `fee * leverage` of excess exposure. This

residue sits below the drift threshold, so the cycle confirms as settled and the excess is never corrected. The vault runs persistently above its target leverage, and the gap is recreated by every subsequent rebalance.

Free margin is below the preview projection. `_rebalanceMarginPreview` computes `minFreeMargin` from the pre-fee `accountValue` and adds projected margin without the fee. The fill spends `fee` more than the preview assumes, so actual free margin ends up `fee × notional` below the projected safety floor on every expansion. The shortfall is directional: it appears on every fill, accumulates across consecutive rebalances, and never recovers. The vault's real buffer is thinner than the safety check believed, and `deployCapital`'s margin-recovery only responds after the damage has occurred.

Both effects stem from the same recurring cash outflow that no order-size or margin computation reserves. It scales with fill notional and leverage and compounds with rebalance frequency.

RECOMMENDATION

Reserve the fee before sizing and in the margin preview: size orders against `absDelta` net of the expected taker fee, and include the fee in `_rebalanceMarginPreview`'s projected margin. This makes the post-fill state match the pre-trade projection so the safety check evaluates the state the vault will actually reach.

RESOLUTION

Ensemble

Commit `f7d6f2738abb3395c79e6036f12eb4e4234de496` fixes L-10. Governance sets a bounded conservative execution-fee rate. Shared target sizing and margin previews reserve that fee for every final order that can actually be placed, including fee-induced reduction legs. Proof case: `test/foundry/unit/LibraryBehavior.t.sol::test_L10_buyOrderReservesDefaultExecutionFee`. Related findings: L-13, L-23.

fixed-PAG

[L-11] Vault can get stuck with both margin-relief paths blocked at the emergency line

Low

Fixed

SUMMARY

Two predicates share the same `emergencyMarginBps` threshold but treat the boundary point differently.

`_safeBridgeToEvmCapacity` blocks all Core → EVM withdrawal once free margin reaches the threshold:

```
uint256 minFreeMargin = (av * emergencyMarginBps) / bpsDenominator;
if (freeMargin <= minFreeMargin) return 0;
```

`emergencyCloseStatus` authorizes emergency close only strictly below the threshold:

```
authorized = freeMargin < threshold || continuation;
```

The first treats "at the line" as unsafe and blocks withdrawal, the second treats "at the line" as not yet an emergency and does not authorize close. The two guardrails disagree exactly at `freeMargin == threshold`, each assuming the other covers the boundary.

IMPACT

At `freeMargin == threshold` the vault is pinned between the two guardrails: the path that relieves margin pressure by pulling capital out of Core, and the path that relieves it by flattening positions, are both closed.

Consider a vault with `accountValue` = \$1,000, `emergencyMarginBps` = 2000 (20%), so `threshold` = \$200, and `freeMargin` = \$200 exactly, sitting on the emergency line.

- `withdrawCapital` with a buffer deficit: `_safeBridgeToEvmCapacity` returns 0 (`freeMargin` 200 <= `minFreeMargin` 200), so `bridgeToEvmAmount` returns 0, `amount(0) < minDeployAmount`, and the call reverts with `InsufficientWithdrawable`.
- `emergencyClose`: `emergencyCloseStatus` returns not authorized because `freeMargin` 200 < `threshold` 200 is false.

A vault that reaches the emergency line while its EVM buffer needs a refill is therefore stuck: it cannot bridge capital back from Core to replenish the buffer, and it cannot trigger emergency close to flatten its positions. Redemptions keep draining the buffer until they revert `InsufficientBuffer`, with no way to replenish and no way to exit via emergency. The state that most needs a relief path is the one where both relief paths are closed.

RECOMMENDATION

Unify the boundary semantics so the vault always has an exit at the line. Since the capacity guard already treats the boundary as unsafe by blocking withdrawal, emergency close should also be authorized at the boundary:

```
-   authorized = freeMargin < threshold || continuation;
+   authorized = freeMargin <= threshold || continuation;
```

RESOLUTION

Ensemble

Commit [2c3fbdda7eb3b77a7f9490b8d09455add5c06652](#) fixes L-11. At the exact emergency margin boundary, Core-to-EVM deployment capacity is zero while both Lens and execution authorize the exposure-reducing emergency close. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L11_emergencyCloseExecutesAtMarginThreshold](#). Related findings: [M-5](#), [M-9](#).

fixed-PAG

[L-12] Core-to-EVM settlement confirms even when the bridge transfer never lands

Low Fixed

SUMMARY

The Core-to-EVM settlement predicate is meant to prove two independent facts: that Core-side principal left the vault, and that USDC arrived on EVM. On the normal path the `coreDecreased` term is always true before the bridge leg is proven, so it adds no settlement information and the check silently degrades to a single ERC-20 balance comparison.

`_submitBridgeToEvm` emits two independent `CoreWriter` actions. Action 7 (`transferUsdClass`, perps to spot) is the only one that moves perps `rawUsd`; action 13 (`sendAsset`, spot to the EVM system address) is the actual bridge and moves only spot to EVM.

```
if (currentSpot < requiredSpot) {
    CoreWriterLib.transferUsdClass(coreWriterTarget, (requiredSpot - currentSpot).toUint64(), false)
;
}
CoreWriterLib.sendAsset(/* spot -> HYPERCORE_USDC_SYSTEM_ADDRESS, wei = amount * 100 */);
```

`preBridgeRawUsd` snapshots the perps summary, so the measured drop is produced by the class transfer, in the same transaction, before any bridging happens, and regardless of whether the `sendAsset` leg ever lands.

```
bool coreDecreased = int256(currentCoreRawUsd) <= int256(preBridgeRawUsd) - int256(expectedAssets);
```

In normal operation `currentSpot` sits in `[0, minDeployAmount)` (a settled to-EVM bridge leaves exactly `USDC_SPOT_RESERVE`, and `deployCapital` only sweeps spot at `>= minDeployAmount`), while `requiredSpot = amount + USDC_SPOT_RESERVE` with `amount >= minDeployAmount`. So `currentSpot < requiredSpot` always holds, the class transfer always fires and always moves `>= amount`, which always satisfies `<= preBridgeRawUsd - 0.99 * amount`.

What remains is `balanceOf(vault) >= preBridgeEvmBalance + expectedAssets`, which the codebase itself twice declares insufficient: "ERC20 balance changes have no bridge-specific provenance" (`VaultBridgeOpsLib.sol:42-43`) and "EVM balance changes have no bridge provenance and can be masked by donations" (`VaultLens.sol:721-723`).

The root problem is that the verifier measures perps `rawUsd`, while the actual bridge consumes Core spot through `sendAsset`. If the class transfer moves too little, a valid EVM receipt can fail confirmation; if it moves enough, the Core-side predicate becomes near-tautological even when `sendAsset` never lands. The fix is to attribute the bridge leg, not to tune the threshold.

IMPACT

The stated Core-to-EVM settlement proof is vacuous, so a failed `sendAsset` is indistinguishable from a settled one.

`confirmBridge` finalizes, `reconcileBridgeEvmBalance` debits only the 1% shortfall, bridge locks clear, and a keeper reward is paid, while principal remains stranded in the vault's Core spot balance.

Direct theft is not demonstrated (the donor funds the EVM side), so this is an invariant break plus forced-state-transition griefing rather than a value extraction path.

EXPLOIT PATH

1. A Core-to-EVM bridge is submitted through the normal path (`currentSpot < amount + USDC_SPOT_RESERVE`), firing the perps-to-spot class transfer that satisfies `coreDecreased`.
2. The `sendAsset` leg does not land (dropped by Core or still in flight), leaving principal in the vault's Core spot balance.
3. The EVM balance rises by `>= expectedAssets` for any reason (a donation or an unrelated inbound transfer).
4. `confirmBridge` passes both terms and finalizes; only the 1% shortfall is reconciled and a keeper reward is paid, while principal stays stranded on Core.

PREREQUISITES

Requires a pending Core-to-EVM bridge on the default path, the `sendAsset` leg failing to land, and an independent EVM balance increase of at least `expectedAssets`.

RECOMMENDATION

Replace `rawUsd` attribution with attribution of the actual bridge leg. Measure the Core spot balance consumed by `sendAsset` (the spot-to-EVM system-address transfer) rather than the perps summary drop caused by the `transferUsdClass` class transfer, so that `coreDecreased` reflects principal actually leaving via the bridge rather than an intra-vault class transfer that fires unconditionally.

RESOLUTION

Ensemble

Commit `8c3915b4e95e1207d1b680858c32ee87128d04c9` fixes L-12. A Core-to-EVM submission records the pre-bridge Core spot balance. Normal confirmation requires the live source balance to reach the deterministic post-bridge baseline within the 1% delivery tolerance, so an unrelated EVM credit cannot confirm a failed send. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L12_coreToEvmRejectsFailedSendAcrossSpotFundingRegions`. Related findings: L-4, L-21.

fixed-PAG

[L-13] Rebalancing silently stops when the order cap hits the venue minimum

Low

Fixed

SUMMARY

The vault uses `MIN_ORDER_NOTIONAL = 10e6` (10 USDC) both as a floor that rounds up a too-small order cap and as the gate that admits an order. The problem is that the two measure the same 10 USDC in different units, so a cap sized exactly at the floor produces an order the gate always rejects.

`_minExecutableCap` rounds a too-small per-order cap up to 10 USDC measured at the **mark price**:

```
function _minExecutableCap(uint256 cap) private pure returns (uint256) {
    if (cap != 0 && cap < MIN_ORDER_NOTIONAL) return MIN_ORDER_NOTIONAL;
    return cap;
}
```

But `_orderSizeMeetsMinimum` admits an order only if it clears 10 USDC measured at the **limit price after lot-rounding**:

```
function _orderSizeMeetsMinimum(OrderPlacement memory placement) private pure returns (bool) {
    return placement.size != 0 && Math.mulDiv(placement.size, placement.limitPx, 1e10) >=
    MIN_ORDER_NOTIONAL;
}
```

Between the two, the order is repriced by the 0.2% slippage haircut (`DEFAULT_REBALANCE_SLIPPAGE_BPS = 20`) in `_buildOrderPlacement` and floored to the asset's lot, so the limit-priced value is always smaller. An order sized at the floor rests at roughly `mark * (1 - 0.2%) ~ $9.98`, below 10 USDC. Every such asset is skipped in `_countPlaceableRebalanceOrders`, leaving zero placeable orders, so `rebalance()` collapses to a silent `RebalanceNoOp` and `VaultLens.getKeeperActions()` reports `needsRebalance = false`.

The effective per-order cap can reach `MIN_ORDER_NOTIONAL` three realistic ways: the admin setter accepts `maxRebalanceOrderNotional == MIN_ORDER_NOTIONAL` (rejecting only values strictly below it); on a smaller vault the adaptive `aumCap = totalTargetNotional * autoRebalanceCapBps / 10000` computes below 10 USDC and gets floored back up; or the per-failure `backoffCap` / wide-spread `spreadCap` shrinks the cap below 10 USDC and is likewise floored back up.

IMPACT

Automated rebalancing is silently disabled. Affected assets stop tracking their target and drift accumulates with no on-chain signal, since `rebalance()` emits only a routine `RebalanceNoOp` and monitoring reports the vault as healthy. This is protocol breakage and tracking error, not theft.

The backoff path is self-latching. The only reset, `resetRebalanceFailures()`, runs inside `confirmRebalance()` / `cancelStaleRebalance()`, both of which require `rebalancePending == true`. A no-op cycle returns before that flag is set, and backoff decays only with the failure count, never with time. Once backoff drives the cap to the floor the vault cannot place, cannot confirm, and cannot reset, so only an admin cap change breaks the loop.

RECOMMENDATION

Express the floor in `_minExecutableCap` in the same units the placement gate uses. Gross `MIN_ORDER_NOTIONAL` up by the slippage haircut and add one lot, e.g. `Math.ceilDiv(MIN_ORDER_NOTIONAL * BPS_DENOMINATOR, BPS_DENOMINATOR - slippageBps) + lotNotional`, so that an order sized at the floor still clears `_orderSizeMeetsMinimum` at its limit price.

Alternatively, gate on mark-priced notional to match the floor, or reject the degenerate cap in `setExecutionLimits` so it can never be configured to the dead value.

RESOLUTION

Ensemble

Commit `d55d854b50af0c8bd47b7f28a6741cf059e832e4` fixes L-13. Configuration rejects static execution caps that cannot produce a venue-valid order. At runtime, a positive dynamic cap below the rounded venue minimum is classified as unplaceable instead of being promoted above its safety bound. Proof case: `test/foundry/unit/LibraryBehavior.t.sol::test_L13_exactMinimumCapsFailClosedWithTheirSourceReason`. Related findings: L-10, L-23.

fixed-PAG

[L-14] A single wide order book blocks rebalancing and deleveraging for the whole vault

Low Fixed

SUMMARY

A failure to price or trade one asset should not block the risk-reducing rebalance of every other asset. The codebase enforces that isolation elsewhere: `_closePosition` skips an unpriceable asset per-asset, and `_adaptiveMaxRebalanceOrderNotional` degrades a wide-spread asset to `LIMIT_SPREAD_BLOCKED` so it is skipped. Neither isolation applies on the rebalance path.

`_buildOrderPlacement` unconditionally routes through `_rebalanceLimitPx8`, whose first statement is `VaultExecutionPolicyLib.enforceRebalanceSpread(...)`. That helper reverts:

`RebalanceLiquidityUnavailable` when the spread exceeds `maxRebalanceSpreadBps`, `MissingPrice` when

either BBO side is zero, and `InvalidConfig` when `ask < bid`. The gate cannot be disabled, because `BaseLeveragedVault._maxRebalanceSpreadBps()` maps a configured `0` back to the 100 bps default.

Two reachable configurations hit that hard revert instead of a per-asset skip.

Default, uncapped configuration. `maxRebalanceOrderNotional` and `autoRebalanceCapBps` are both zero on a freshly initialised vault: `_initialize` never writes slot 11, neither field exists in `InitParams`, and only admin ops 16 and 18 set them. `_adaptiveMaxRebalanceOrderNotional` therefore early-returns before the spread branch:

```
if (maxRebalanceOrderNotional == 0 && aumCap == 0) return (0, LIMIT_NONE);
```

So `_getAssetDelta` applies no clamp, `shouldPlace` stays true, and `_countPlaceableRebalanceOrders` calls `_buildOrderPlacement`, which reverts. Because `rebalance()` calls `countRebalanceOrders(params)`, the whole transaction reverts before any state is written.

Dormant flatten, under any configuration. `_validateDormantFlattenOrders` bypasses `_getAssetDelta` entirely and calls `_requireOrderSize(order, _buildOrderPlacement(p, order))` per asset, so one wide book reverts the flatten of the whole basket.

With caps configured the spread branch does run, but it zeroes the delta for reduce-only orders identically to expansions, so deleveraging is silently skipped rather than reverted. Either way the vault cannot reduce exposure on that asset.

Keeper telemetry fails at the same moment. `VaultLens.getKeeperActions` reverts on the same call chain (`_navKeeperActions` -> `_rebalancePreview` -> `previewRebalanceOrderCountForAccount`) with no `try/catch` anywhere in `VaultLens`. Because `_navKeeperActions` runs before `needsEmergencyClose` is computed, keeper bots also lose the emergency signal.

IMPACT

The vault's routine de-risking mechanism becomes unavailable in exactly the market conditions that require it. No attacker is needed: a momentarily wide or one-sided book on any one of the basket's assets is sufficient.

Drift then compounds unmanaged until free margin crosses the emergency floor, escalating to `emergencyClose`. That flattens every position at emergency slippage (100 bps default versus 20 bps for a routine rebalance), pauses the vault, latches `lastEmergencyTimestamp`, and blocks redemptions for at least the 24-hour cooldown plus a complete HyperCore drain.

Keeper operators are simultaneously blind, because the lens that would tell them an emergency close is needed reverts on the same condition.

RECOMMENDATION

Skip the offending asset instead of reverting. Move the spread check out of `_buildOrderPlacement` into a per-asset predicate that maps a blocked asset to `shouldPlace = false` on every path, including the dormant flatten and the uncapped configuration.

Exempt reduce-only orders from the spread throttle entirely: a risk-reducing fill in a wide book is strictly better than no fill, and it cannot increase exposure.

Wrap the lens preview so keeper telemetry degrades gracefully instead of reverting.

RESOLUTION**Ensemble**

Commit [ddac5286ea58a83a2a0ef09b811c56a6a0663064](#) fixes L-14. Each asset's book is read without reverting the full rebalance. An unreadable, crossed, or policy-blocked market is skipped while valid sibling orders continue. Proof case: [test/foundry/unit/LibraryBehavior.t.sol::test_L14_uncappedRebalanceSkipsOneSidedBookAndPlacesSafeSibling](#). Related findings: L-5, L-15.

fixed-PAG

[L-15] Emergency close and rebalance orders are priced away from the market and never fill

Low

Fixed

SUMMARY

`_emergencyAnchorPx` deliberately picks the worst member of the `{markPx, oraclePx}` pair for the trade direction (`min` for a buy, `max` for a sell), then clamps the live BBO reference back inside `anchor * (1 ± spreadBps)` in the unfavourable direction:

```
uint256 boundedReference = isBuy
    ? (uint256(anchorPx) * (p.bpsDenominator + p.emergencyMaxSpreadBps)) / p.bpsDenominator
    : (uint256(anchorPx) * (p.bpsDenominator - p.emergencyMaxSpreadBps)) / p.bpsDenominator;
if (isBuy && rawReferencePx > boundedReference) return boundedReference.toUint64();
if (!isBuy && rawReferencePx < boundedReference) return boundedReference.toUint64();
```

For a sell, a real best bid below the band is raised up to `max(mark, oracle) * (1 - spread)`. The resulting reduce-only IOC order (`tif = 3`) is priced above the book and does not fill.

With shipped defaults (`DEFAULT_EMERGENCY_CLOSE_MAX_SPREAD_BPS = 100`, `DEFAULT_EMERGENCY_CLOSE_SLIPPAGE_BPS = 100`, `DEFAULT_MAX_REBALANCE_SPREAD_BPS = 100`, `DEFAULT_REBALANCE_SLIPPAGE_BPS = 20`), the vault can only sell down to `max(mark, oracle) * 0.99 * 0.99` and only buy up to `min(mark, oracle) * 1.01 * 1.01`. Because the anchor already takes the far side of the pair, the perp-versus-index basis consumes the entire band before any book width is allowed, so once oracle and mark diverge by more than roughly 1% the band is exhausted and the close cannot cross.

Nothing detects this. `_tryEmergencyCloseLimitPx` still reports `priced == true`, so the per-asset skip never engages and the caller believes a close was placed. This is a deliberate fail-closed choice against an adversarial book placing the BBO far from fair value, but the no-fill failure it selects is worse than the manipulation it prevents.

IMPACT

Both de-risking mechanisms, routine rebalancing and emergency close, become inoperative whenever the HyperCore BBO sits more than roughly 1% plus slippage away from the mark/oracle anchor. That is precisely the dislocation that triggers them.

By the time this bites, the vault has already transitioned to `Paused` and latched `lastEmergencyTimestamp`, so all redemptions are frozen while the positions the close was meant to flatten stay open and keep moving against the vault. On a 6x profile, the difference between a controlled close at a 2% dislocation and a venue liquidation is most of the remaining maintenance-margin buffer.

The silent failure makes it worse. Because `priced == true` is still reported, no `EmergencyCloseAssetSkipped` event fires, `emergencyRewardPending` records that the close happened, and operators have no signal that nothing was actually executed.

RECOMMENDATION

Widen or waive the anchor band for reduce-only orders. A reduce-only fill can never increase exposure, so the manipulation risk the band guards against does not apply.

Alternatively, escalate: if the clamped limit is not marketable against the observed BBO, fall back to a wider emergency band rather than emitting an order that cannot fill.

At an absolute minimum, detect non-marketable and report `priced == false`, so the per-asset skip engages, the `EmergencyCloseAssetSkipped` event fires, and `emergencyRewardPending` correctly records that the close did not happen.

RESOLUTION

Ensemble

Commit `4701053efe7deecdb91eddbbecad43b57438ad86` fixes L-15. Rebalance and emergency IOC orders set their limit to the worst acceptable oracle/mark-anchored price. Placement also requires the live close-side book to be marketable within that limit, so an IOC is not submitted at a price that cannot fill. Proof case:

`test/foundry/unit/`

`LibraryBehavior.t.sol::test_L15_rebalanceIocPlacesAtMarketableBoundAndSkipsOutsideIt`. Related findings: M-7, L-14.

fixed-PAG

[L-16] Vault can be permanently frozen by triggering a keeper reward during bridge recovery

Low

Fixed

SUMMARY

A Core->EVM bridge freezes the vault's EVM USDC balance as `preBridgeEvmBalance` at submission, and settlement for that direction is judged purely as a delta from that frozen snapshot, with no absorption for unrelated outflows. While that baseline is live, no other code path may move USDC out of the vault's EVM balance.

The protocol encodes this invariant as:

```
bool unsafeToReward = bridgePending && !bridgeToCore;
```

But the baseline stays live in a second state the predicate does not cover: bridge-recovery-pending, where `bridgePending` is false but `bridgePendingAmount != 0`. `markBridgeRecoveryPending` clears the `bridgePending` bit but does not clear the slot holding `preBridgeEvmBalance`, so the suppressor is inert in exactly the window where the snapshot still governs settlement.

An `emergencyClose` in that window pays a keeper reward out of the vault's EVM USDC, dropping the balance below the frozen baseline. Because `_lateBridgeEvmReceipt` has no downward absorption for unrelated outflows, the Core->EVM settlement delta can never be reached again.

IMPACT

Every value-moving flow reverts on `bridgePendingAmount != 0`: `deposit`, `redeem`, `collectFees`, `deployCapital`, `withdrawCapital`, `rebalance`, and `startBridge`. Owner `_unpause` is also gated on the same condition through `_requireNoPendingOperation`, and that gate is independent of `vaultState`, so the owner cannot return the vault to Normal either. The vault is stuck in `Paused` with no user exit and 100% of TVL frozen.

The duration is indefinite in code. The only sanctioned exit destroys the product: the owner must call the irreversible `permanentEmergencyShutdown`, wait 30 days for `forceClearBridgeRecovery`, after which the vault can never return to Normal and users can only claim pro rata. The cost to trigger is one `emergencyClose` transaction, from which the caller profits by the keeper reward.

An undocumented cure exists but is not discoverable in practice. `_lateBridgeEvmReceipt` reads a live `balanceOf`, so an out-of-band USDC donation restores the delta. The donation needed is only the reward minus 1% of the bridged amount, on the order of a single dollar at the reference configuration; after it, anyone can call `reconfirmBridge`, the received amount clamps at `bridgePendingAmount`, the state fully unlatches, and `accountedBalance` is conservative. But that cure appears nowhere in the code, comments, tests, or recovery machinery, and is not deducible from a bare `BridgeNotSettled()` revert, so an operator following the contract's own recovery surface is steered to the irreversible shutdown and the 30-day force-clear.

EXPLOIT PATH

1. The vault enters bridge-recovery-pending: `markBridgeRecoveryPending` clears `bridgePending` but leaves `bridgePendingAmount != 0` and `preBridgeEvmBalance` set.
2. An attacker calls `emergencyClose` in this window; because `unsafeToReward` is false, a keeper reward is paid out of the vault's EVM USDC.
3. The EVM balance drops below the frozen `preBridgeEvmBalance`, so the Core->EVM settlement delta can never be satisfied.
4. All value-moving flows and owner `_unpause` now revert on `bridgePendingAmount != 0`; the vault is frozen with the attacker profiting from the reward.

PREREQUISITES

Requires a non-zero `keeperReward`. Deployments shipping `keeperReward = 0` are inert because the reward path returns early on zero. However `_setKeeperReward` is bounded only by `keeperReward <= minDeployAmount`, so any non-zero reward set later puts small bridges inside the break window. The break window is `bridgeAmount < 100 × keeperReward`, while the on-chain bound permits a reward up to 100 times wider than that.

RECOMMENDATION

Extend both copies of the predicate to cover the recovery state. The condition "a to-EVM settlement baseline is live" is `!bridgeToCore && bridgePendingAmount != 0`, which holds in both the pending and recovery-pending states:

```
bool unsafeToReward = !bridgeToCore && (bridgePending || bridgePendingAmount != 0);
```

Do not fix this by re-snapshotting the EVM balance inside `markBridgeRecoveryPending`. In this sequence the reward is paid after the recovery mark, so a fresh snapshot taken at mark time is corrupted by the very same outflow.

As defence in depth, give the to-EVM receipt path the same absorption its to-Core sibling already has, by decrementing `preBridgeEvmBalance` on any sanctioned outflow that occurs while a to-EVM baseline is live: a single helper called from the keeper-reward transfer wrapper and from `collectFees`. That makes the settlement test robust to any future permitted outflow, not just the one currently known.

RESOLUTION

Ensemble

Commit `ef96f9e1d856e71e70f6a5d4980728aa8b319439` fixes L-16. Ordinary keeper rewards reserve Core-to-EVM principal during both an active bridge and retained recovery. Saturating subtraction handles a recorded principal larger than the accounted balance without exposing that principal to payment. Proof case: `test/foundry/unit/LibraryBehavior.t.sol::test_L16_keeperRewardStatusReservesToEvmRecoveryPrincipal`. Related findings: L-25, L-26, L-32.

fixed-PAG

[L-17] Emergency position cleanup is unreachable when VaultAdmin owns the vault

Low

Fixed

SUMMARY

After `permanentEmergencyShutdown()`, the vault enters the `Emergency` state. If positions are still open, `emergencyClose()` only allows its owner-cleanup path when the caller is the vault owner:

```
bool ownerCleanup = vaultState == VaultState.Emergency && msg.sender == owner();
if (!ownerCleanup && vaultState != VaultState.Normal && vaultState != VaultState.Paused) {
    revert InvalidState(vaultState, VaultState.Normal);
}
```

Deployments are set up so the `VaultAdmin` proxy is the vault owner. But `VaultAdmin` has no way to reach this function: `IVaultAdminTarget` does not expose `emergencyClose()`, no wrapper exists, and `forwardVaultAdminUpdate()` can only forward `adminUpdate(...)`.

IMPACT

The owner-cleanup path becomes unreachable in practice. The `VaultAdmin` owner/multisig is not itself the vault owner, so it cannot call `emergencyClose()` directly, and the `VaultAdmin` proxy that is the owner has no path to invoke it.

This stalls the rest of the recovery flow, since both `recoverAllEmergencyFunds()` and `enableEmergencyWithdraw()` require all positions to be closed first.

RECOMMENDATION

Add an owner-only `VaultAdmin.emergencyClose(address vault)` wrapper and expose `emergencyClose()` through `IVaultAdminTarget`, so the `VaultAdmin` proxy can execute the vault's owner-cleanup path directly.

RESOLUTION**Ensemble**

Commit `714df8905ffaf20d4feabc7fceb5e90a747314` fixes L-17. A typed VaultAdmin wrapper lets its owner invoke the vault's existing owner-only emergency cleanup. Unauthorized callers remain blocked by the same ownership checks. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L17_vaultAdminOwnerCanReachEmergencyCleanup`.

fixed-PAG

[L-18] Large redemptions can overleverage remaining shareholders for hours

Low

Fixed

SUMMARY

`redeem()` burns shares and pays USDC from the EVM buffer without proportionally reducing the vault's HyperCore positions. The same position notional is then borne by a smaller NAV, increasing leverage for the remaining shareholders.

For a redemption of fraction `f` immediately after a rebalance:

```
effective leverage = L / (1 - f)
drift              = f / (1 - f)
```

For example, a BULL vault at \$100 NAV and 3x leverage holds \$300 notional. A 15% redemption reduces shareholder NAV to about \$85 while leaving the \$300 position unchanged, pushing effective leverage to `3.53x` and creating `17.65%` drift.

BULL/BEAR use a 10% drift threshold with a 4-hour rebalance cooldown, and the inside-cooldown bypass requires 2x the threshold (20% drift). So a full-buffer redemption can leave the vault overleveraged for up to four hours even though it already exceeds the normal 10% rebalance threshold.

The 0.3% withdrawal fee does not compensate remaining holders because it accrues to protocol fees rather than shareholder NAV. CircuitBreaker redemption caps can mitigate this, but they are configurable and `0` means unlimited, so the vault itself does not enforce a safe redemption bound.

IMPACT

Remaining shareholders are left materially overleveraged for up to four hours, bearing increased PnL sensitivity and the later trading costs required to deleverage back to target.

RECOMMENDATION

Allow redemption-induced deleveraging to bypass the cooldown once drift exceeds the normal threshold, or cap instant redemptions so a single redemption cannot create excessive drift.

RESOLUTION

Ensemble

Commit `f7048742f9334a87c7cc4c757a6a187980993aa3` fixes L-18. During cooldown, normal-drift correction may proceed only as reductions-only work. Every order must still pass the adaptive-cap, book, price-band, lot, and minimum-notional checks, so a large redemption cannot leave the remaining holders overleveraged.

Proof case: `test/foundry/unit/`

`VaultLifecycle.t.sol::test_L18_redemptionOverexposureDuringCooldownPlacesOnlyReductions`.

Related findings: L-3.

fixed-PAG

[L-19] Dormant-vault guard can be skipped when Core spot balance is nonzero

Low

Fixed

SUMMARY

`deployCapital()` processes any existing Core spot USDC before it checks whether the vault is dormant.

```
uint256 spotAmount = LeveragedVaultLib.coreSpotUsdc(...);

if (spotAmount >= minDeployAmount) {
    _startCoreSpotRecovery(spotAmount);
    return;
}

(uint256 totalNav,) = computeNav();
if (totalNav < minViableAum) revert VaultDormant(...);
```

Because the spot-recovery branch returns early, the `totalNav >= minViableAum` check is never reached when `coreSpot >= minDeployAmount`. In that case the call succeeds and moves Core spot into perp margin even though `totalNav < minViableAum` would otherwise mark the vault dormant.

IMPACT

The dormant-vault policy is bypassed: capital gets deployed into perp margin for a vault whose NAV is below the minimum viable threshold, when it should have reverted.

RECOMMENDATION

Compute NAV and enforce `totalNav >= minViableAum` before handling Core spot recovery, so the dormant check applies regardless of the Core spot balance.

RESOLUTION

Ensemble

Commit `f174fa2cfb94bbfc9a1d9b05f7225c51f6fe2c67` fixes L-19. Before moving attributed Core spot, capital deployment accrues fees and enforces the existing minimum viable AUM. Core spot can no longer bypass the dormant-vault guard. Proof case: `test/foundry/unit/`

`VaultLifecycle.t.sol::test_L19_dormantVaultRejectsCoreSpotRecovery`. Related findings: L-7.

fixed-PAG

[L-20] A single tiny dormant position can block flattening of all others

Low Fixed

SUMMARY

Dormant positions are treated as actionable once their mark-priced notional is at least `MIN_ORDER_NOTIONAL`. The problem is that `_validateDormantFlattenOrders` then builds the actual lot-rounded order at its limit price and reverts if that order's notional falls below the minimum.

A position can pass the initial notional threshold yet produce an unplaceable order after slippage and lot-size rounding are applied. For example, a \$10 long position may yield a sell order worth less than \$10, so validation reverts with `OrderSizeTooSmall`.

```
uint256 absNotional = _absNotional(currentNotional);
if (absNotional < MIN_ORDER_NOTIONAL) continue;
...
_requireOrderSize(order, _buildOrderPlacement(p, order));
ordersPlaced++;
```

Because the loop reverts instead of skipping the asset that cannot be placed, one borderline position aborts the entire flattening call.

IMPACT

A single borderline dormant position prevents all other dormant positions from being flattened, leaving larger exposure open when it should have been closed.

RECOMMENDATION

Apply the same minimum-size reduce-only oversizing used by emergency closes to dormant flattening, or skip individually unplaceable dust positions instead of reverting the whole operation.

RESOLUTION

Ensemble

L-15's per-asset placement filter in `4701053efe7deecdb91eddbbecad43b57438ad86` also fixes L-20. A repriced dormant leg that falls below the venue minimum is skipped by itself, so it cannot revert and roll back a valid sibling flatten order. The finding-specific proof is committed in

`1b1fe5427d44ba9274b0528ce38346cefa4ec72f`: `test/foundry/unit/LibraryBehavior.t.sol::test_L20_L15SkipsRepricedSubminimumDormantOrderAndFlattensSibling`.

Related finding: L-15.

fixed-PAG

[L-21] Unrelated USDC transfers can prematurely clear a stalled bridge and inflate NAV

Low Fixed

SUMMARY

For Core-to-EVM bridges, `_verifyBridgeOutcome` requires two conditions: HyperCore `rawUsd` must decrease and the EVM USDC balance must increase.

The reconfirmation path in `reconfirmBridgeAccountedBalance` only checks the EVM balance increase, not the HyperCore decrease. After a stale bridge is cancelled, any unrelated USDC transfer into the vault can satisfy `reconfirmBridge()` before the original HyperCore bridge actually executes.

This clears the bridge recovery state early and counts the unrelated USDC as the bridge receipt, even though the corresponding HyperCore balance has not yet decreased.

IMPACT

NAV is temporarily overstated because the incoming USDC is credited while the HyperCore side has not been debited.

If the original bridge later executes, NAV returns to the correct level. But any deposits or redemptions performed during the interim are priced against the inflated NAV, allowing unfair profit at the expense of other users.

RECOMMENDATION

Require the Core-to-EVM reconfirmation path to verify both `coreDecreased` and `evmIncreased`, consistent with `_verifyBridgeOutcome`, before clearing the bridge recovery state.

RESOLUTION

Ensemble

Commit `6a437ea089c58842614c1a4cbe6860590935560b` fixes L-21. Before crediting a capped late receipt, recovery reconfirmation applies the same Core-spot and EVM-receipt checks as normal confirmation. Unrelated EVM USDC cannot clear recovery without source settlement. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L21_coreToEvmRecoveryRejectsUnrelatedEvmDonationWithoutSourceSettlement`. Related findings: L-4, L-12, M-13.

fixed-PAG

[L-22] Successful rebalances can be wrongly recorded as failures after market moves

Low Fixed

SUMMARY

`cancelStaleRebalance()` decides whether a stale rebalance succeeded by checking whether the **current** rebalance drift is zero:

```
if (LeveragedVaultLib.rebalanceProgressNotional(params) == 0) {
    // success
} else {
    VaultExecutionPolicyLib.recordRebalanceFailure();
}
```

The problem is that `params.totalNav` and the position notionals are recomputed from the latest NAV and mark prices. So even when every submitted order was fully filled, subsequent market movement can shift NAV and the target notional enough to reintroduce drift.

When that happens, a completed rebalance is treated as a failure and `rebalanceFailureCount` is incremented.

IMPACT

A rising failure count reduces the effective notional cap of later rebalances through the configured backoff policy, and after enough failures it can temporarily block routine rebalances.

Because ordinary market movement alone can trigger these false failures, successful executions can progressively degrade the vault's rebalance capacity, leaving it further from its intended exposure and increasing market and margin risk.

RECOMMENDATION

Determine settlement from execution attributable to the pending rebalance, such as the snapshotted position sizes and submitted order amounts, rather than comparing current positions against a target recomputed from the latest NAV.

RESOLUTION

Ensemble

Commit `c909a4b7f40ed2ef39b0e2b4bd8ad655d5df4b63` fixes L-22. Rebalance submission records the signed position-size delta for every order, including the actual size of a dormant flatten. Confirmation measures fills against those submitted deltas instead of a mark-priced target that can move after submission. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L22_fullSubmittedSizesConfirmAfterMarkMovement`. Related findings: L-41.

fixed-PAG

[L-23] Rebalance orders can exceed the safety cap set by the adaptive policy

Low

Fixed

LOCATION

`VaultExecutionPolicyLib::_minExecutableCap:1-5`

SUMMARY

Adaptive execution caps limit the notional of each rebalance order based on vault AUM, backoff state, and market conditions. A cap below HyperCore's minimum executable order size should block that order, not authorize a larger one.

Instead, `_minExecutableCap` promotes any positive sub-minimum cap up to the floor:

```
function _minExecutableCap(uint256 cap) private pure returns (uint256) {
    // returns MIN_ORDER_NOTIONAL whenever cap is nonzero but < MIN_ORDER_NOTIONAL
}
```

So an AUM-derived cap of 100000 becomes 10e6, and `_getAssetDelta` then uses the larger value as the effective cap. This breaks the invariant that a submitted order must not exceed the cap chosen by the adaptive policy.

The same upward rounding defeats several distinct cap sources before `_getAssetDelta` applies them:

- **AUM caps** - a low-AUM per-asset cap is inflated to the minimum order size.
- `autoRebalanceCapBps` - a configured percentage of total target notional that permits only a smaller trade still yields a minimum-sized order.

- **Spread-shrink branch** - near the configured spread ceiling the proportional spread cap can fall well below the executable minimum, where it should throttle rebalance size toward zero, but normalization restores the full minimum order and defeats the throttle.

IMPACT

Rebalance orders can be larger than the AUM-based, percentage-based, or spread-reduced cap the execution policy intended.

The excess is bounded per order to the minimum order size, but it can push exposure beyond the configured risk budget and compound across repeated rebalances.

EXPLOIT PATH

1. Configure a nonzero `autoRebalanceCapBps` and reach a low-AUM state where the calculated per-asset cap is 100000 USDC-6 units.
2. Ensure the vault has actionable drift exceeding the minimum order size.
3. A keeper calls rebalance; `_minExecutableCap(100000)` returns 10e6.
4. `_getAssetDelta` applies the promoted 10e6 cap and CoreWriter receives an order larger than the adaptive cap.

RECOMMENDATION

Make sub-minimum adaptive caps non-executable instead of promoting them:

```
function _minExecutableCap(uint256 cap) private pure returns (uint256) {  
    return cap != 0 && cap < MIN_ORDER_NOTIONAL ? 0 : cap;  
}
```

The adaptive caller must then treat a zero result as a blocked limit and skip placing the order, and must distinguish this from a genuinely disabled cap so a blocked limit is not misread as uncapped execution. A more explicit fix is to return a dedicated limiting reason (for example `(0, LIMIT_SPREAD_BLOCKED)`) whenever any post-reduction cap falls below `MIN_ORDER_NOTIONAL`. In the spread-shrink branch, removing `_minExecutableCap` and relying on the normal minimum-order gate also preserves the intended block-or-skip behavior.

RESOLUTION

Ensemble

Commits `d55d854b50af0c8bd47b7f28a6741cf059e832e4` and `ce97c0bf38fa9b36946d50e86c0ffc8aaab21ceb` fix L-23. The L-13 execution policy rejects a positive adaptive cap below the venue minimum instead of promoting the order above that cap. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L23_positiveAdaptiveCapBelowVenueMinimumBlocksRebalance`. Related findings: L-10, L-13.

fixed-PAG

[L-24] Bridge recovery can freeze the vault when funds land in Core spot

Low Fixed

LOCATION

`VaultBridgeOpsLib.prepareRejectedEmergencyBridgeRetry:20-53`

SUMMARY

A failed Core-to-EVM bridge should only stay blocked when the value is truly unrecoverable. The submission path can move funds first, then fail the EVM leg, leaving value recoverable in substance but rejected by the recovery check.

When a keeper triggers a Core-to-EVM refill, `_submitBridgeToEvm()` runs `transferUsdClass()` (perp margin to Core spot) and then `sendAsset()`. If the first leg settles but the second leg does not, the funds sit in Core spot with no EVM USDC received.

`prepareRejectedEmergencyBridgeRetry()` then checks only for an EVM receipt and requires the full rejected amount to remain in perp `withdrawable()`. It never checks Core spot, so this partial-success state fails the predicate and reverts.

The mismatch is notable because `_recoverEmergencyAssets` already knows how to recover `coreSpotUsdc`, but `prepareRejectedEmergencyBridgeRetry()` runs first and blocks that branch from executing.

IMPACT

A legitimate permissionless withdrawal can leave the vault stuck in bridge-recovery state with funds stranded in Core spot.

While `bridgePendingAmount` stays nonzero, deposits, redemptions, fee collection, capital movement, and rebalancing are all blocked. The vault can remain frozen for more than seven days until an owner uses the delayed force-clear or another trusted emergency intervention.

EXPLOIT PATH

1. A keeper calls `withdrawCapital(amount)` while the buffer needs a Core-to-EVM refill; `_submitBridgeToEvm()` submits `transferUsdClass()` then `sendAsset()`.
1. The USD-class transfer settles into Core spot, but no EVM USDC is received.
1. After the timeout, `cancelStaleBridge()` reverses the optimistic accounting and leaves `bridgePendingAmount != 0`.
1. `prepareRejectedEmergencyBridgeRetry()` checks only perp `withdrawable()`, reverts, and normal operations stay blocked until force-clear or another trusted recovery action.

RECOMMENDATION

Include Core spot in the recoverability check and move the value back to the correct class before retrying:

```
uint256 withdrawable = LeveragedVaultOpsLib.withdrawable(precompileTarget);
uint256 coreSpot = LeveragedVaultLib.coreSpotUsdc(precompileTarget, address(this));
if (withdrawable < rejectedAmount && coreSpot < rejectedAmount) {
    revert ILeveragedVaultTypes.BridgeRecoveryPending();
}
if (coreSpot >= rejectedAmount) {
    VaultBridgeOpsLib.startCoreSpotRecovery(
        precompileTarget, coreWriterTarget, rejectedAmount
    );
    return (rejectedAmount, false);
}
```

Alternatively, record the pre-bridge Core spot balance and reconcile the post-failure spot delta, then use that delta together with perp withdrawable as the recovery source.

RESOLUTION

Ensemble

Commit `4a894a8ffd98b6b2172cbf27628287beed7fb918` fixes L-24. Core-to-EVM submission records the scheduled Core spot principal. Settlement reduces that attribution to the live balance, and rejected recovery can move the attributed spot back to perps before retrying the bridge. Funds that land in Core spot therefore remain actionable. Proof case: `test/foundry/unit/CoreSpotAttribution.t.sol::test_L24_coreToEvmSubmissionAttributesPrincipalPlusConfiguredReserve`. Related findings: L-27, L-39, L-46.

fixed-PAG

[L-25] Keeper reward can drain the protocol's minimum reserve buffer

Low

Fixed

LOCATION

`BaseLeveragedVault:confirmBridge/_payBridgeKeeperReward:319-333,635-646`

SUMMARY

When a Core-to-EVM bridge is confirmed, the protocol should reconcile its optimistic accounting to the actual receipt and pay the keeper reward only from the excess of that receipt above `minDeployAmount`. This keeps the minimum buffer intact even when the accepted settlement tolerance produces a partial receipt.

The implementation reconciles the accounting but then discards the actual receipt and sizes the reward cap from the originally requested `bridgeAmount`:

```
reconcileBridgeEvmBalance(usdcToken);
(amount, wasToCore) = clearBridgeState();

uint256 maximumReward = bridgeAmount > minDeployAmount
    ? bridgeAmount - minDeployAmount
    : 0;
```

So a receipt `R` that lands below the requested `A` still yields a reward cap based on `A`. For example, with `minDeployAmount = 100` and a request of `A = 101`, a delivered `R = 99.99` (above the 99% acceptance threshold) still produces a cap of `101 - 100 = 1`, and that shortfall is paid out of the remaining vault buffer.

IMPACT

A permissionless confirmer can receive a reward larger than what is safely available after reconciliation, pushing both the tracked and physical EVM buffer below `minDeployAmount`.

This recurs on every partially settled bridge that still meets the acceptance tolerance, repeatedly eroding the protected minimum reserve. The reward status check only validates accounting and balance sufficiency, so it does not independently preserve `minDeployAmount` once the cap is overstated.

EXPLOIT PATH

1. `minDeployAmount = 100` USDC and a Core-to-EVM withdrawal is initiated for `A = 101` USDC.
2. The bridge delivers `R = 99.99` USDC, above the 99% threshold but below the request.

3. `confirmBridge()` reconciles the tracked balance down to `R`, but `finalizeBridge()` hands back `confirmedAmount = 101`.
4. The reward cap is computed as $101 - 100 = 1$, leaving only `98.99` USDC instead of the intended `100` USDC reserve.

RECOMMENDATION

Make `finalizeBridge()` return the actual credited receipt and size the cap from it:

```
uint256 actualReceived = reconcileBridgeEvmBalance(usdcToken);
clearBridgeState();
return (actualReceived, false, true);
```

Alternatively, compute the cap from the post-reconciliation tracked balance, or have

`reconcileBridgeEvmBalance` return the credited amount, then use:

```
uint256 maximumReward = actualReceived > minDeployAmount
    ? actualReceived - minDeployAmount
    : 0;
```

RESOLUTION

Ensemble

Commit `ea93592843e0d7c00f1de3a606e012ea6118c590` fixes L-25. Core-to-EVM confirmation derives reward capacity from the delivery credited during balance reconciliation, caps over-delivery at the requested principal, and preserves the configured EVM buffer before payment. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L25_bridgeRewardCapsToConfirmedDeliveryAndPreservesDeployPrincipal](#). Related findings: L-16, L-26.

fixed-PAG

[L-26] Emergency keeper rewards can drain funds reserved for a pending bridge recovery

Low Fixed

LOCATION

`LeveragedVaultLib:keeperRewardStatus:266-282`

SUMMARY

The bridge recovery flow keeps enough EVM balance and accounting to verify a late Core-to-EVM receipt against the pre-bridge snapshot. The keeper reward check only treats these funds as reserved while `bridgePending` is set.

When a stale bridge is cancelled, `bridgePending` is cleared but `bridgePendingAmount` and the pre-bridge balance remain in recovery storage. Because `emergencyClose()` does not treat the retained recovery amount as off-limits, it can pay a keeper reward out of the balance meant to back that recovery.

When the bridge eventually settles, `reconfirmBridge()` measures the net balance increase against the original snapshot. The reward already spent reduces that increase, so a receipt `Q` that previously satisfied 99% of the original amount `A` (observed increase `Q-R` instead of `Q`) can fall below the threshold.

IMPACT

A keeper reward can consume EVM USDC needed to prove a late Core-to-EVM settlement, leaving bridge recovery stuck.

Normal vault operations or emergency recovery are delayed until additional USDC is supplied or the slower force-clear path is used.

EXPLOIT PATH

1. Start a Core-to-EVM bridge for `A` USDC and let it reach the stale timeout without visible settlement.
1. Call `cancelStaleBridge()`; the vault reverses optimistic accounting, clears `bridgePending`, but retains `bridgePendingAmount = A`.
1. Call `emergencyClose()` while the vault holds enough physical USDC for the configured keeper reward `R`; `R` is transferred to the emergency keeper.
1. Let the bridge arrive and call `reconfirmBridge()`; the observed increase is `Q-R` rather than `Q`, so a receipt that previously met the 99% threshold now fails.

RECOMMENDATION

Reserve the recovery amount in `keeperRewardStatus` as well as the active pending amount:

```
uint256 spendable =
    !bridgeToCore && (bridgePending || bridgePendingAmount != 0)
    ? uint256(accountedBalance) - bridgePendingAmount
    : accountedBalance;
```

Also make emergency reward payment explicitly defer while a Core-to-EVM recovery is pending:

```
bool unsafeToReward = (!bridgeToCore &&
    (bridgePending || bridgePendingAmount != 0));
```

RESOLUTION

Ensemble

Commit `25e4d12d26d7968b83981e5b24804f0850a66e84` fixes L-26. Emergency rewards use L-16's reserve check for both active Core-to-EVM bridges and retained recovery. Clearing `bridgePending` cannot expose recovery principal to a keeper payment. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L26_emergencyRewardPreservesToEvmRecoveryPrincipal`. Related findings: L-16, L-25.

fixed-PAG

[L-27] Emergency withdrawals can strand vault USDC on HyperCore and socialize the loss

Low Fixed

LOCATION

`VaultAdminOpsLib::_enableEmergencyWithdraw:251-270`

SUMMARY

Emergency withdrawals should distribute all recoverable vault assets pro rata among outstanding share holders. A minimum operational transfer threshold may be used to avoid uneconomic keeper actions, but it must not turn a nonzero protocol-owned balance into an unaccounted one before the shareholder snapshot.

The implementation allows emergency withdrawal activation whenever Core spot is below `minDeployAmount`, then derives `emergencyAvailableUsdc` solely from the EVM token balance. A positive HyperCore spot balance is therefore omitted from the snapshot and from every later claim.

Once shareholders burn their shares through `emergencyWithdraw`, the omitted Core funds are not part of the claim calculation and the emergency lifecycle offers no path to distribute them. For example, with `minDeployAmount` at 10 USDC and 5 USDC left on Core spot, that 5 USDC is treated as below-threshold, never bridged back, and never enters the claim base.

IMPACT

Vault-owned USDC remaining on HyperCore spot can be permanently excluded from emergency withdrawals and socialized as a loss among shareholders.

The omitted amount is bounded by `minDeployAmount`, but that threshold is configurable, so the loss is not limited to a token-rounding dust amount. Shareholders burn their shares without receiving their full proportional entitlement while protocol-owned funds sit stranded on Core.

EXPLOIT PATH

1. Positions are closed and the vault retains 5 USDC on HyperCore spot while `minDeployAmount` is 10 USDC.
2. Emergency recovery treats the 5 USDC as below-threshold and does not bridge it back.
3. `enableEmergencyWithdraw` snapshots only the vault's EVM USDC balance.
4. Users claim and burn all shares, receiving no portion of the 5 Core USDC, which remains stranded outside the claim base.

RECOMMENDATION

Block emergency withdrawals until the Core spot balance is zero:

```
uint256 remainingSpot = LeveragedVaultLib.coreSpotUsdc(precompileTarget, address(this));
if (remainingSpot != 0) {
    revert ILeveragedVaultTypes.EmergencyAssetsNotRecovered(remainingSpot);
}
```

Alternatively, change recovery to process every positive balance, using the threshold only to control keeper economics rather than accounting finality.

RESOLUTION

Ensemble

The L-46 fix in `163e70f` covers this case too. Attributed perp dust below the configured threshold no longer blocks emergency withdrawal enablement or unpause, matching the existing spot treatment. Amounts at or above the threshold must still be recovered. The tests include both the zero-capacity case and an amount exactly at the threshold.

fixed-PAG

[L-28] Emergency withdrawals pay out assets reserved for management fees

Low Fixed

LOCATION

src/vaults/shared/BaseLeveragedVault.sol:enableEmergencyWithdraw:10-14 (via VaultAdminOpsLib) and emergencyWithdraw:30-40

SUMMARY

The normal deposit, redeem, and fee-collection paths checkpoint pending management fees before changing balances or supply. The emergency-withdrawal path skips that checkpoint.

It snapshots the entire physical USDC balance and later distributes it pro rata, while leaving `accruedFees` and `accountedBalance` untouched:

```
uint256 available = IERC20(usdcToken).balanceOf(address(this));
_setEmergencyWithdrawSnapshot(totalSupply_.toUint96(), available.toUint64());

uint256 usdcAmount = (emergencyAvailableUsdc * userShares) / emergencyTotalShares;
_burn(msg.sender, userShares);
IERC20(usdcToken).safeTransfer(msg.sender, usdcAmount);
```

Because the fee liability accrued through the snapshot time is never recognized, fee value is included in the amount distributed to claimants. After the final claim, `accruedFees` remains positive and `accountedBalance` is unchanged even though the physical balance has been reduced.

IMPACT

Treasury can lose management fees that had accrued economically but were not yet checkpointed, since that value is paid out to emergency claimants instead.

The stale `accruedFees` liability and unchanged `accountedBalance` no longer correspond to the physical balance or outstanding share supply, which can distort subsequent NAV calculations and recovery operations.

EXPLOIT PATH

1. Pending management fees exist while the vault still holds USDC and has outstanding shares.
1. The owner completes emergency recovery and enables emergency withdrawals; the full physical USDC balance is stored as `emergencyAvailableUsdc`.
1. Claimants call `emergencyWithdraw()`, which distributes that full snapshot pro rata and burns their shares.
1. After the final claim, `accruedFees` remains positive and `accountedBalance` is unchanged despite the reduced physical balance, leaving treasury fee ownership and NAV accounting inconsistent.

RECOMMENDATION

Accrue pending fees before taking the emergency snapshot and reserve the resulting fee liability:

```
_accrueAndGetNavPerShare();
uint256 feeReserve = accruedFees;
uint256 physical = IERC20(usdcToken).balanceOf(address(this));
```

```
uint256 available = physical > feeReserve ? physical - feeReserve : 0;
_setEmergencyWithdrawSnapshot(totalSupply._.toUint96(), available.toUint64());
```

Alternatively, transfer or segregate accrued fees to the treasury before enabling emergency withdrawals, then snapshot only the remaining shareholder assets and clear the corresponding accounting fields.

RESOLUTION

Ensemble

Commit [97819701fc5a5e10f567f366af060602f46d4ca0](#) fixes L-28. Emergency-withdrawal preparation accrues management fees while NAV reads are available and snapshots only accounted physical USDC after reserving those fees. Donations are excluded. Permanent shutdown remains a separate transaction that needs no external reads, so a later outage cannot block it. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L28_emergencySnapshotAccruesAndReservesFeesWithoutIncludingDonations](#). Related findings: [L-32](#), [L-34](#), [L-35](#), [L-40](#).

fixed-PAG

[L-29] External deposits can block confirmation of a completed rebalance

Low

Fixed

LOCATION

[BaseLeveragedVault:confirmRebalance:409-423](#)

SUMMARY

A rebalance cycle should be judged against the NAV it was submitted with. External capital arriving after submission should not change the completion condition for orders that are already on the book.

Instead, [confirmRebalance\(\)](#) recomputes NAV and all rebalance parameters from current HyperCore state:

```
(uint256 totalNav,) = computeNav();
RebalanceParams memory params = _rebalanceParams(totalNav, rebalanceNonce - 1);
uint256 remaining = LeveragedVaultLib.rebalanceProgressNotional(params);
if (!rebalanceReductionsOnly && remaining != 0) revert RebalanceNotSettled();
```

Because [_grossNav\(\)](#) includes live Core equity and spot balances, a direct third-party deposit while the cycle is pending raises the target. A rebalance whose orders filled completely then appears incomplete, so [remaining](#) is non-zero and confirmation reverts.

IMPACT

A permissionless Core-side donation can make a fully filled, uncapped rebalance fail confirmation.

Confirmation stays blocked until timeout, after which a stale cancellation records a rebalance failure and can activate backoff. This delays subsequent corrective operations and impairs future permissionless rebalancing.

The attacker pays the donation as a griefing cost and gains nothing directly, so this is a liveness and state-consistency issue rather than theft.

EXPLOIT PATH

1. `rebalance()` submits uncapped orders using the vault's current NAV and sets `rebalancePending = true`.
2. The orders fill completely at the original target positions.
3. A third party sends USDC to the vault's HyperCore account, increasing `accountValue` or spot balance while the rebalance is still pending.
4. `confirmRebalance()` derives a larger target from the increased NAV and reverts; a later stale cancellation records a rebalance failure.

RECOMMENDATION

Snapshot the target notionals at submission and use them for confirmation rather than rebuilding them from current NAV:

```
// On submission
rebalanceSubmittedTargets = _targetNotionals(progressParams);

// On confirmation
uint256 remaining = _progressAgainstTargets(
    rebalanceParamsForCurrentReads(), rebalanceSubmittedTargets
);
```

A simpler alternative is to snapshot the submission NAV and build confirmation parameters from that value:

```
rebalanceSubmittedNav = totalNav;
// confirmRebalance
RebalanceParams memory params = _rebalanceParams(
    rebalanceSubmittedNav, rebalanceNonce - 1
);
```

RESOLUTION

Ensemble

Commit `5e3bec019e31e9c4ca17400097af872e88f15a2d` fixes L-29. Rebalance submission records shareholder NAV and an explicit initialized marker. Settlement compares progress to that submission-time value, so a later external deposit cannot change a filled cycle's outcome; an uninitialized upgraded cycle follows the legacy-safe path. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L29_externalDepositCannotBlockFilledRebalanceConfirmation](#). Related findings: L-22, L-41.

fixed-PAG

[L-30] Failed EVM-to-Core bridge can lock the vault in recovery state

Low Fixed

LOCATION

`VaultBridgeOpsLib:prepareRejectedEmergencyBridgeRetry:20-49`

SUMMARY

A stale bridge needs both a terminal rejection transition and a late-settlement transition. The stale-cancellation branch creates a recovery marker by clearing `bridgePending` while retaining `bridgePendingAmount`, and normal vault operations correctly keep treating that marker as blocking state.

The rejection helper only handles Core-to-EVM recovery. It explicitly reverts whenever `bridgeToCore` is true, so an EVM-to-Core bridge that is never later observed as settled cannot be cleared through it. `reconfirmBridge()` also offers no rejection path because it requires a successful bridge outcome.

As a result, a failed or permanently unobservable EVM-to-Core bridge has no ordinary lifecycle transition back to idle. The only available fallback is owner-driven Emergency escalation followed by a delayed force-clear.

IMPACT

A failed or permanently unobservable EVM-to-Core bridge can leave the vault stuck in recovery state indefinitely under normal operation.

This blocks deposits, redemptions, rebalances, fee collection, and other flows until an emergency escalation and a delayed administrative force-clear are performed.

EXPLOIT PATH

1. A keeper calls `deployCapital()` and starts an EVM-to-Core bridge.
2. Settlement stays unobservable until `cancelStaleBridge()` times out, which reverses accounting but leaves `bridgePendingAmount` nonzero.
3. The bridge is rejected or never produces observable Core credit, so `reconfirmBridge()` cannot succeed.
4. `prepareRejectedEmergencyBridgeRetry()` reverts because `bridgeToCore` is true, leaving normal operations blocked.
5. Only Emergency escalation and the delayed `forceClearBridgeRecovery()` path can clear the state.

RECOMMENDATION

Add an ordinary, delayed rejection-clearing branch for to-Core recovery, after verifying that no destination credit was observed:

```
if (_bridgeToCore()) {
    if (verifyBridgeOutcome(precompileTarget, usdcToken)) {
        _reconfirm(precompileTarget, usdcToken);
        clearBridgeState();
        return (rejectedAmount, true);
    }
    clearBridgeState();
    emit RejectedBridgeRecoveryCleared(rejectedAmount);
    return (rejectedAmount, false);
}
```

Make the rejection proof and delay strict enough to avoid clearing a bridge that is merely slow but has not yet settled.

RESOLUTION

Ensemble

Commit `42ae80625b5203ccd6c793d552af5ee23eeae64b` fixes L-30. Retained bridge recovery uses one finality boundary: the greater of 24 hours and the configured bridge timeout. Recovery cannot be force-cleared before that point, but a failed bridge no longer locks the vault indefinitely. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_forceClearBridgeRecoveryHoldsRetainedRecoveryUntilFinality`. Related findings: L-2, L-33, L-36, L-37.

fixed-PAG

[L-31] Bridge withdrawals can freeze when HyperCore fees exceed the hardcoded reserve

Low

Fixed

LOCATION

`LeveragedVaultOpsLib::_submitBridgeToEvm:105-123`

SUMMARY

Every Core-to-EVM bridge submission must reserve enough HyperCore spot USDC to pay the external `sendAsset` gas cost while transferring the requested amount. That cost is determined outside the EVM contract and can vary, so the reserve needs to be fetched dynamically or backed by a guaranteed invariant.

Instead, the implementation uses a fixed `USDC_SPOT_RESERVE = 2,100,000` (2.1 USDC) and tops up only to `amount` plus that reserve before calling `sendAsset`. No code checks the current external fee requirement or updates the reserve when HyperCore behavior changes.

Because the bridge state is stored before the asynchronous action settles, an external rejection leaves optimistic accounting and pending state behind. If HyperCore requires more than 2.1 USDC (for example 3 USDC), the send is rejected, the expected EVM receipt never arrives, confirmation fails, and stale cancellation enters bridge recovery.

IMPACT

Core-to-EVM withdrawals and emergency recovery can become unavailable whenever HyperCore requires more spot USDC than the hardcoded reserve.

The vault optimistically accounts for a receipt that never arrives, then enters bridge recovery. While that state is unresolved, NAV-sensitive operations guarded by `bridgePending` / `bridgePendingAmount` are blocked, so deposits, redemptions, and other normal capital operations freeze until the external fee condition changes or administrative recovery is performed.

EXPLOIT PATH

1. The vault holds sufficient HyperCore collateral, but the current `sendAsset` requirement is 3 USDC, exceeding the hardcoded 2.1 USDC reserve.
2. A keeper calls `withdrawCapital()`, which records the bridge and optimistically credits the expected EVM receipt.
3. The bridge submission supplies only `amount + 2.1 USDC` of spot and is rejected asynchronously by HyperCore.

- Confirmation fails, stale cancellation enters bridge recovery, and normal deposits, redemptions, and capital operations stay blocked until recovery or fee conditions change.

RECOMMENDATION

Replace the fixed reserve with a configurable value that can be updated when the external bridge requirement changes, and require the observed spot balance to cover it:

```
uint256 requiredSpot = amount + coreToEvmReserve;
if (currentSpot < requiredSpot) {
    CoreWriterLib.transferUsdClass(
        coreWriterTarget, (requiredSpot - currentSpot).toUint64(), false
    );
}
```

Preferably, obtain the required reserve from a supported HyperCore fee/quote mechanism and validate it immediately before submission.

Also add a recovery timeout/retry path that can clear residual pending state after verifying no bridge receipt exists, and that retries using the refreshed reserve rather than repeating the same fixed amount.

RESOLUTION

Ensemble

Commit [3e2b41a8872e3f186bd39e0120763569beb02fc0](#) fixes L-31. Governance can set bounded USDC reserve and dust thresholds as protocol costs change. A stored zero selects the protocol default for that value. Proof case: [test/foundry/unit/CoreBridgeConfigAdmin.t.sol::test_L31_ownerPersistsCoreBridgeConfigAndZeroRestoresDefaults](#). Related findings: L-46.

fixed-PAG

[L-32] Keeper rewards can eat into USDC owed to the treasury

Low

Fixed

LOCATION

[BaseLeveragedVault:_payKeeperRewardUpTo:856-874](#)

SUMMARY

Accrued protocol fees are recorded in [accruedFees](#) and excluded from NAV, marking that USDC as owed to the treasury. Keeper rewards are paid from the same physical USDC buffer and reduce [accountedBalance](#), but the reward-eligibility check never reserves the accrued fee amount.

Because of this, a keeper reward can be paid even when the only sufficient tracked balance is the USDC backing [accruedFees](#). The payment leaves [accruedFees](#) unchanged while reducing the physical and accounted funds available to satisfy it.

Afterwards, [collectFees](#) can return less than the recorded [accruedFees](#), because its collectable amount is capped by available USDC and [accountedBalance](#). This leaves a real accounting shortfall for the treasury without needing any malicious privileged actor.

IMPACT

A legitimate permissionless keeper operation can divert USDC that is already represented by `accruedFees` owed to the treasury.

The treasury's recorded fee balance can become partially or wholly uncollectable, with the discrepancy bounded by keeper rewards and the available fee reserve.

EXPLOIT PATH

1. Normal deposits, redeems, or management-fee accrual increase `accruedFees` while the USDC stays in the vault buffer.
1. The vault reaches a state where `accountedBalance` and physical USDC are enough for a keeper reward, but the surplus after reserving `accruedFees` is not.
1. A permissionless keeper calls an eligible operation, causing `_payKeeperRewardUpTo` to invoke `keeperRewardStatus` and transfer the reward.
1. `accountedBalance` decreases but `accruedFees` does not; when the treasury later calls `collectFees`, the reduced accounted balance caps collection and leaves part of the recorded fee claim unpaid.

RECOMMENDATION

Exclude accrued fees from keeper reward spendability:

```
uint256 reservedFees = accruedFees;
uint256 spendable = accountedBalance > reservedFees
    ? uint256(accountedBalance) - reservedFees
    : 0;
if (bridgePending && !bridgeToCore) {
    spendable = spendable > bridgePendingAmount ? spendable - bridgePendingAmount : 0;
}
```

Alternatively, if keeper rewards are meant to be a treasury-funded expense, explicitly charge rewards against `accruedFees` and reduce both the fee claim and the corresponding accounting balance.

RESOLUTION

Ensemble

Commit `d71a6d0ae434c5206d54ed2fb268d0e8df10fdea` fixes L-32. One spendable-EVM calculation caps available funds by both physical and accounted balances, then subtracts accrued treasury fees and the L-16 Core-to-EVM principal reserve with saturation. Keeper rewards cannot spend either liability. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L32_keeperRewardPreservesAccruedTreasuryFees`. Related findings: L-16, L-38, L-40.

fixed-PAG

[L-33] Force-clearing a stuck bridge recovery can overstate the vault's tracked balance

Low Fixed

LOCATION

`VaultBridgeOpsLib:reverseBridgeAccounting:122-134`

SUMMARY

EVM-to-Core bridge submissions pre-debit the vault's tracked EVM balance (`accountedBalance`) because settlement is asynchronous. If a bridge is cancelled as stale before settlement is visible, the recovery path reverses that debit and keeps the pending amount so a later settlement can be reconfirmed.

Reconfirmation via `reconfirmBridge` only succeeds when the observed Core credit reaches 99% of the requested amount. A late partial credit below that threshold leaves the bridge unreconfirmed.

The emergency force-clear path then clears only the pending flags and bridge metadata; it never undoes the earlier `accountedBalance` reversal. For example, a 1,000 USDC bridge that is cancelled stale restores 1,000 USDC to `accountedBalance`, but if only 980 USDC is later credited on Core, the reversal remains in storage even though those tokens are neither in the EVM buffer nor fully credited on Core.

IMPACT

A partial late settlement can strand the vault in `BridgeRecoveryPending`, blocking normal deposits, redeems, fees, rebalancing, and capital movement until emergency intervention.

If emergency force-clear is used to escape that state, `accountedBalance` overstates the tracked EVM assets by the shortfall, leaving NAV accounting inconsistent with the physical funds.

EXPLOIT PATH

1. Start an EVM-to-Core bridge of 1,000 USDC through `deployCapital`.
2. Let the bridge time out without visible settlement and call `cancelStaleBridge`; `accountedBalance` is restored by 1,000 USDC while `bridgePendingAmount` stays nonzero.
3. Allow only 980 USDC-equivalent of Core credit to become visible; `reconfirmBridge` fails because at least 990 USDC is required.
4. Force-clear recovery after the emergency delay; pending state disappears, but `accountedBalance` still reflects the full 1,000 USDC.

RECOMMENDATION

Keep the stale reversal separate from finalized accounting and reconcile it against the observed outcome. Preferably, retain the original debit on stale cancellation and only restore it once settlement is actually confirmed.

If a reversal is retained for operational reasons, force-clear must apply a conservative shortfall adjustment rather than only clearing state:

```
uint256 delivered = observedCoreCredit;
uint256 unrecovered = amount > delivered ? amount - delivered : 0;
_setAccountedBalance((accountedBalance - unrecovered).toUint64());
clearBridgeState();
```

RESOLUTION

Ensemble

Commit [0457c5770b373ce18d7adb0be970c22bb6f14c65](#) fixes L-33. Terminal recovery accounts for the physical EVM receipt separately from the Core source release. It also keeps spot provenance separate from emergency attribution and restores legacy scheduled attribution only for a live retained record bounded by its submission amount. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L33_terminalToCoreChargesOnlyActualCappedEvmDebit](#). Related findings: L-2, L-30, L-36, L-37.

fixed-PAG

[L-34] Management fees can be suppressed indefinitely by frequent vault interactions

Low

Fixed

LOCATION

[BaseLeveragedVault::_accrueAndGetNavPerShare:764-781](#)

SUMMARY

The management fee is meant to accrue continuously over time regardless of how often users interact with the vault, and any fractional accrual below one USDC unit should be preserved so repeated interactions cannot erase it.

Instead, the code floors each interval independently and advances the timestamp even when the computed fee is zero:

```
uint256 elapsed = block.timestamp - lastFeeAccrualTime;
fee = SafeCast.toUint128(
    (totalNav * managementFeeRate * elapsed) / (1e18 * yearSeconds)
);
lastFeeAccrualTime = block.timestamp.toUint40();
if (fee != 0) accruedFees = (uint256(accruedFees) + fee).toUint64();
```

Because [lastFeeAccrualTime](#) is reset on every call, a sequence of valid deposits or redeems that keeps each interval below one fee unit floors the fee to zero each time and discards the fractional remainder. Normal value-moving operations continue while fees never accumulate.

IMPACT

Treasury management fees can be systematically under-collected, especially at low or moderate AUM where the per-interaction fee is below one USDC base unit.

The issue is triggerable through ordinary routed vault operations and requires no privileged access.

EXPLOIT PATH

1. A low-AUM vault has a management fee whose accrual over two minutes is less than 1 USDC base unit.
1. A user performs a valid deposit or redeem just before two minutes elapse, so the fee calculation floors to zero.
1. The vault still updates `lastFeeAccrualTime` to the current block timestamp, discarding the fractional amount.
1. Repeating the operation every two minutes keeps the treasury fee balance at zero while the vault remains active.

RECOMMENDATION

Preserve the fractional remainder explicitly while continuing to update the timestamp:

```
uint256 denominator = 1e18 * yearSeconds;
uint256 numerator = uint256(totalNav) * managementFeeRate * elapsed + feeRemainder;
fee = SafeCast.toUint128(numerator / denominator);
feeRemainder = numerator % denominator;
lastFeeAccrualTime = uint40(block.timestamp);
```

A higher-precision accumulator is preferred over simply skipping the timestamp update on zero-fee intervals, since the latter can cause repeated recomputation and edge-case inaccuracies. If adding storage is undesirable, defer the timestamp update when the calculated fee is zero:

```
if (fee != 0) {
    lastFeeAccrualTime = block.timestamp.toUint40();
    accruedFees = (uint256(accruedFees) + fee).toUint64();
}
```

RESOLUTION

Ensemble

Commit `dda9034fa6b419383c50c57bc542605735309766` fixes L-34. Each accrual stores the exact fractional numerator in the typed `uint96 managementFeeRemainder` field at slot 12 offset 8. Frequent interactions therefore carry fractional fees forward instead of discarding them. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L34_fractionalManagementFeesMatchLumpAccrual`. Related findings: L-35.

fixed-PAG

[L-35] Management fees can be understated during drawdowns

Low Fixed

LOCATION

`LeveragedVaultLib::_pendingManagementFee:1540-1548`

SUMMARY

Management fees are meant to reflect the assets managed across the entire accrual interval. Instead, the whole interval is valued using only the current NAV at the moment fees are crystallized.

`_pendingManagementFee` computes `totalNav * managementFeeRate * elapsed`, and `_accrueAndGetNavPerShare` then resets `lastFeeAccrualTime` to the current timestamp. A deposit or

redeem that happens while NAV is depressed therefore settles a long preceding high-NAV period at the low point.

Because resetting the timestamp discards the high-NAV history and there is no compensating historical accounting, repeated crystallization during drawdowns pays fees based on the momentary low NAV rather than the time-weighted average AUM.

IMPACT

Treasury fee revenue can end up lower than the fee implied by the vault's actual average assets under management.

At the same time, remaining holders see a temporarily overstated net NAV relative to the intended time-weighted fee model.

EXPLOIT PATH

1. A vault holds a high NAV for most of an accrual interval while management fees remain pending.
1. During a temporary drawdown, a holder calls `redeem` with a small share amount, so `_accrueAndGetNavPerShare()` values the whole interval at the depressed current NAV.
1. The same call sets `lastFeeAccrualTime` to the drawdown timestamp, dropping the prior high-NAV period from future accrual.
1. Repeating this at later drawdowns keeps cumulative fees below what the average AUM over those intervals would imply.

RECOMMENDATION

Accrue fees from stored time-weighted NAV checkpoints instead of the final instantaneous NAV. Maintain a checkpoint and integrate each elapsed interval when crystallizing:

```
uint256 intervalAum = storedAumCheckpoint;  
uint256 fee = intervalAum * managementFeeRate * elapsed  
    / (1e18 * yearSeconds);  
accruedFees += fee;  
storedAumCheckpoint = currentTotalNav;  
lastFeeAccrualTime = uint40(block.timestamp);
```

For greater accuracy, use shorter periodic checkpoints or an accumulator updated whenever NAV changes, deposits occur, or redemptions occur.

RESOLUTION

Ensemble

Commit `3db83e0eebb1901f1b8493f11595d2031d77eca7` fixes L-35. Management fees accrue against a stored fee-net opening AUM rather than the lower value observed after a drawdown. A legacy initialization marker distinguishes upgraded state that has not yet recorded its first checkpoint. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L35_drawdownChargesStoredOpeningAum`. Related findings: L-28, L-34.

fixed-PAG

[L-36] A single failed bridge can freeze the entire vault for over a week

Low Fixed

LOCATION

`BaseLeveragedVault:cancelStaleBridge:392-415; VaultBridgeOpsLib:forceClearBridgeRecovery:112-130`

SUMMARY

A pending bridge is meant to either confirm once the asynchronous HyperCore state becomes visible, or be safely cancelled after its timeout, returning the vault to a usable normal state with optimistic accounting restored.

Instead, `cancelStaleBridge()` reverses the accounting but preserves `bridgePendingAmount` as recovery state. The normal-state guard rejects every NAV-sensitive operation while that amount is nonzero, and the only unconditional cleanup, `forceClearBridgeRecovery()`, is guarded by an Emergency-state check.

A never-settling bridge therefore has no normal-state terminal transition. There is no bounded, non-emergency path to clear the retained `bridgePendingAmount`.

This is especially visible for conclusively rejected Core-to-EVM bridges, where `reconfirmBridge` cannot clear the state because it is only a later-success path.

IMPACT

A single failed asynchronous bridge can block deposits, redemptions, fee collection, capital management, and rebalancing for the entire vault.

Because the only unconditional cleanup requires Emergency state and is subject to a recovery delay, this can cause a vault-wide freeze exceeding seven days until the owner changes the vault state and the delayed emergency recovery path completes.

Permissionless stale cancellation makes this worse: a merely delayed bridge can be pushed into this blocking recovery state immediately after timeout, so liveness then depends on a later reconciliation or governance action.

EXPLOIT PATH

1. A keeper calls `deployCapital()` while the vault is Normal, causing `_startBridge()` to set `bridgePendingAmount` and `bridgePending`.
2. HyperCore never produces the expected settlement observations.
3. After the timeout, any caller invokes `cancelStaleBridge()`; `finalizeBridge()` returns false, `reverseBridgeAccounting()` runs, and `markBridgeRecoveryPending()` leaves `bridgePendingAmount` nonzero.
4. All normal lifecycle operations revert with `BridgeRecoveryPending`, while `forceClearBridgeRecovery()` stays unavailable until Emergency state and its recovery delay are satisfied.

RECOMMENDATION

When a bridge failure can be proven conclusively, the pending amount should be cleared together with the pending flag rather than requiring indefinite recovery-state retention.

Add a final stale-recovery operation that clears the retained amount without requiring Emergency, after a conservative finality period:

```
function clearFinalBridgeRecovery() external nonReentrant {
    if (bridgePending || bridgePendingAmount == 0) revert BridgeNotPending();
    if (block.timestamp < bridgeSubmitTimestamp + FINAL_RECOVERY_DELAY) {
        revert StaleTimeoutNotReached(...);
    }
    VaultBridgeOpsLib.clearBridgeState();
}
```

Alternatively, make `forceClearBridgeRecovery()` callable from Normal after the same finality delay, while retaining the existing Emergency-only path for immediate administrative recovery.

RESOLUTION

Ensemble

Commit [85014b73c51d052ca86e5e6791bae89ae7283ce6](#) fixes L-36. After the shared finality boundary, anyone may reconfirm retained recovery and fall back to terminal reconciliation, even when the vault is in Normal state. A failed bridge therefore cannot freeze the vault for another administrative timeout. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L36_permissionlessRetainedRecoveryIsTerminalAtSharedFinality](#). Related findings: L-2, L-30, L-33, L-37.

fixed-PAG

[L-37] Partial bridge settlement can permanently freeze the vault

Low Fixed

LOCATION

[LeveragedVaultOpsLib::reconfirmBridgeAccountedBalance:120-140](#)

SUMMARY

The bridge recovery flow only has a clean exit when a late settlement is nearly complete. After a stale cancellation, `cancelStaleBridge` reverses the optimistic accounting but keeps `bridgePendingAmount` set, and `reconfirmBridge` only clears it when the observed settlement reaches 99% of the pending amount.

```
uint256 expectedAssets = (amount * 99) / 100;
if (received < expectedAssets) return (false, accountedBalance);
return (true, (uint256(accountedBalance) + received).toUint64());
```

A permanently short or partial transfer stays below the threshold, so `reconfirmBridge` returns false without ever clearing the recovery state. Since `_requireNormalNoPending` rejects any nonzero `bridgePendingAmount`, the vault is left frozen in Normal state with no trustless exit. For example, a 100 USDC bridge that settles at only 98 USDC falls below the 99 USDC threshold and can never be finalized by ordinary users.

IMPACT

A short or partial external bridge settlement can leave the vault stuck in Normal mode with `bridgePendingAmount` set indefinitely.

Deposits, redemptions, capital deployment and withdrawal, rebalancing, fee collection, and new bridge operations all revert until an administrative emergency recovery path is used. There is no trustless liveness path for ordinary users or keepers to unblock the vault.

EXPLOIT PATH

1. Start a Core-to-EVM bridge for 100 USDC via `withdrawCapital(100e6)`, setting `bridgePendingAmount` to 100e6.
2. After the timeout, call `cancelStaleBridge()` before settlement is visible; the vault clears `bridgePending` but retains the recovery amount.
3. The bridge later delivers only 98 USDC, below the 99 USDC threshold, so `reconfirmBridge()` fails.
4. `deposit()`, `redeem()`, `deployCapital()`, `withdrawCapital()`, `rebalance()`, and `collectFees()` all revert because `bridgePendingAmount` remains nonzero.

RECOMMENDATION

Add a time-gated short-settlement finalization that credits only the observed amount and clears the recovery marker, so the shortfall is reflected explicitly in accounting rather than silently treated as a full bridge:

```
function finalizeShortRecovery() external {
    if (bridgePending != false || bridgePendingAmount == 0) revert InvalidState(...);
    if (block.timestamp < recoveryDeadline) revert TooEarly();
    uint256 received = _lateBridgeEvmReceipt(usdcToken, bridgePendingAmount, preBridgeEvmBalance);
    accountedBalance = (uint256(accountedBalance) + received).toUint64();
    clearBridgeState();
}
```

Alternatively, allow the existing emergency force-clear mechanism after a timeout while automatically recording the unrecovered amount as a loss, rather than requiring a separate owner-triggered transition to Emergency.

RESOLUTION

Ensemble

Commit [5f3fcdfe31bb0c2cfbaea8b4d2d35b0c50ce4f5a](#) fixes L-37. Terminal reconciliation credits only delivery that can be observed independently. It records the requested and delivered amounts for every terminal outcome, accounts for any shortfall, and then clears the retained state. Proof case:

`test/foundry/unit/`

`VaultLifecycle.t.sol::test_L37_terminalToCoreReportsObservedDeliveryAgainstRequest`. Related findings: L-2, L-30, L-33, L-36.

fixed-PAG

[L-38] Permissionless capital deployment can drain accrued fee reserves into the strategy

Low

Fixed

LOCATION

`BaseLeveragedVault:deployCapital:407-433; LeveragedVaultLib:_bufferDeployAmount:377-397`

SUMMARY

`deployCapital` should move only user-attributable excess buffer assets into HyperCore while preserving the backing for accrued treasury fees. Accrued fees are excluded from total NAV, so they should stay reserved and collectable rather than treated as strategy collateral.

The deployment calculation uses the full tracked EVM buffer against a target derived from fee-excluded NAV, without subtracting `accruedFees` from either `accountedBalance` or the physical buffer:

```
uint256 targetBuffer = (totalNav * bufferBps) / bpsDenominator;
uint256 currentBuffer = IERC20(usdcToken).balanceOf(address(this));
if (currentBuffer > accountedBalance) currentBuffer = accountedBalance;
if (currentBuffer > targetBuffer) deployAmount = currentBuffer - targetBuffer;
```

Because the fee backing is not reserved, a permissionless keeper can bridge accrued fee reserves.

`accountedBalance` is reduced by the bridged amount, but `accruedFees` stays unchanged, so the protocol records a fee claim whose backing has been moved into the leveraged strategy and is no longer immediately collectable.

IMPACT

Accrued treasury fees can be moved from the EVM buffer into HyperCore collateral, where they become unavailable for collection and economically exposed to strategy losses or gains.

The fee liability remains recorded even though its backing has been deployed alongside user strategy capital.

EXPLOIT PATH

1. Fees accrue to 100 USDC while the vault holds 115 USDC in its EVM buffer and net user NAV is 100 USDC.
2. The 15% buffer target from net user NAV is 15 USDC.
3. A permissionless keeper calls `deployCapital()`, and the formula selects 100 USDC to deploy.
4. The bridge pre-debits `accountedBalance` by 100 USDC, but `accruedFees` stays at 100 USDC, so the treasury reserve is now exposed in HyperCore and cannot be collected from the depleted EVM buffer.

RECOMMENDATION

Subtract accrued fees from the deployable tracked balance before computing the current buffer. Pass the fee reserve into the helper or compute a reserved balance at the call site.

```
uint256 reserved = accruedFees;
uint256 deployableAccounted = accountedBalance > reserved
    ? accountedBalance - reserved
    : 0;
uint256 excess = LeveragedVaultLib.capitalDeployAmount(
    usdcToken, deployableAccounted, totalNav, bufferBps,
    minDeployAmount, _rebalanceParams(totalNav, rebalanceNonce), address(this)
);
```

Ensure the helper and related keeper previews use the same reserved-balance model so execution and read paths stay consistent.

RESOLUTION**Ensemble**

Commit `6489e43d8fd24f51e82b20c77ec3812213c277d0` fixes L-38. Capital-deployment and margin-recovery sizing use the L-32 spendable-EVM calculation. Neither path can bridge physical or accounted USDC already owed to the treasury. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L38_deployCapitalPreservesAccruedTreasuryFees`. Related findings: L-32, L-40.

fixed-PAG**[L-39] Anyone can indefinitely block vault unpause and emergency withdrawals****Low****Fixed****LOCATION**`VaultAdminOpsLib::_unpause:128-154`**SUMMARY**

The intended recovery flow lets the owner unpause a paused vault or enable pro-rata emergency withdrawals once positions are closed and bridge operations settle. Only protocol-attributable assets still awaiting reconciliation should block those transitions.

In practice the checks inspect the vault address's entire live HyperCore balance. A nonzero withdrawable balance always blocks `_unpause`, and Core spot holdings at or above the configured minimum also block it; `_enableEmergencyWithdraw` applies the same policy. Neither function verifies that the balance came from a vault bridge operation or is reflected in vault accounting.

Because HyperCore balances attach to the vault address, a third party can add funds without touching vault storage. Recovery can clear the balance, but the attacker can restore it before the owner retries, producing a repeatable lock.

IMPACT

Any permissionless actor who can transfer USDC to the vault's HyperCore account can repeatedly prevent unpause or emergency-withdraw activation.

This keeps ordinary redemptions and emergency claims unavailable for an indefinite period, since the attacker can re-apply the lock after every recovery attempt.

EXPLOIT PATH

1. The owner has closed all positions and is ready to call `adminUpdate(OP_UNPAUSE, ...)` or `adminUpdate(OP_ENABLE_EMERGENCY_WITHDRAW, ...)`.
2. An attacker transfers `M >= minDeployAmount` USDC to the vault's HyperCore account.
3. The owner's transition call reads the new withdrawable or Core spot balance and reverts before changing vault state.
4. After recovery removes the balance, the attacker repeats the transfer before the next transition call, sustaining the lock.

RECOMMENDATION

Gate recovery transitions on protocol-tracked bridge state rather than on all live Core balances. Remove unsolicited balances from the blocking condition and keep only checks proving pending protocol operations have settled:

```
if (bridgePending || bridgePendingAmount != 0) {  
    revert ILeveragedVaultTypes.BridgeInFlight();  
}  
_setVaultState(VaultState.Normal);
```

Alternatively, snapshot and attribute only balances produced by the vault's own bridge operations, and treat any excess Core balance as surplus that cannot prevent unpause or emergency-withdraw activation.

RESOLUTION

Ensemble

Commit [7ca854feb3506dfb2a29bb5363417c22560d8885](#) fixes L-39. Unpause and emergency-withdrawal checks count only protocol-attributed spot and perp balances that the next recovery action can move. An outside donation cannot create an unrecoverable amount that blocks either transition. Proof case: [test/foundry/unit/AttributedRecoveryPreview.t.sol::test_L39_donatedCoreSpotIsNeitherAdvertisedNorRecovered](#). Related findings: L-24, L-27, L-46.

fixed-PAG

[L-40] Keeper rewards skim management fees owed to the treasury

Low

Fixed

LOCATION

`BaseLeveragedVault:confirmRebalance`

SUMMARY

Every state-changing operation that alters the vault's tracked assets is supposed to crystallize management fees through the current NAV checkpoint first, then pay keeper rewards from the remaining tracked balance so historical yield attributable to the treasury is unaffected.

`confirmRebalance` instead pays the reward right after clearing the rebalance state. The reward helper reduces `accountedBalance` while the fee accumulator is left unchanged:

```
_clearPendingRebalance();  
if (remainingNotional == 0) {  
    VaultExecutionPolicyLib.resetRebalanceFailures();  
}  
_payKeeperRewardUpTo(achievedProgress);
```

If time-based yield has accrued since the previous fee checkpoint, the next fee calculation sees the lower post-reward balance, so the treasury loses the management-fee portion tied to the reward amount.

The same ordering bug affects both bridge directions in `confirmBridge`, since bridge finalization also changes tracked assets before the reward is paid.

IMPACT

Each successful rebalance (or bridge) confirmation permanently under-accrues management fees by the fee amount attributable to the keeper reward.

Repeated confirmations compound the discrepancy, transferring value that should have been recorded as treasury fees to keepers instead.

EXPLOIT PATH

1. A vault has a positive `accountedBalance` and a `lastFeeAccrualTime` far enough in the past for management fees to have accrued.
2. A keeper submits a rebalance cycle and waits for it to make measurable progress.
3. The keeper calls `confirmRebalance()`, which invokes `_payKeeperRewardUpTo` and decreases `accountedBalance` without updating `accruedFees`.
4. The next NAV-sensitive operation calculates fees from the reduced balance, permanently omitting the fee that should have accrued before the reward payment.

RECOMMENDATION

Accrue fees immediately before paying the reward:

```
_clearPendingRebalance();  
if (remainingNotional == 0) {  
    VaultExecutionPolicyLib.resetRebalanceFailures();  
}  
_accrueAndGetNavPerShare();  
_payKeeperRewardUpTo(achievedProgress);
```

Alternatively, centralize keeper-reward payment in a helper that always calls `_accrueAndGetNavPerShare` before changing `accountedBalance`, and use that helper for every reward path including `confirmBridge`.

RESOLUTION

Ensemble

Commit [681a6e3d67b6976eebcae88aaf154a5e45889005](#) fixes L-40. Deploy, rebalance confirmation, stale cancellation, and bridge reward paths accrue the current management fee before calculating or paying a keeper reward. Proof case: [test/foundry/unit/](#)

[VaultLifecycle.t.sol::test_L40_rebalanceConfirmAccruesManagementFeesBeforeReward](#). Related findings: L-28, L-32, L-38.

fixed-PAG

[L-41] Rebalance confirmation can freeze the vault or accept an unfilled rebalance

Low

Fixed

LOCATION

`BaseLeveragedVault:confirmRebalance:563-589`

SUMMARY

Confirmation is meant to settle the asynchronous orders generated for the target that existed when the rebalance was submitted. Instead of persisting that submission target, the code stores only progress metadata and recomputes NAV and the target at confirmation time.

```
(uint256 totalNav,) = computeNav();
RebalanceParams memory params = _rebalanceParams(totalNav, rebalanceNonce - 1);
uint256 remainingNotional = LeveragedVaultLib.rebalanceProgressNotional(params);
if (!rebalanceReductionsOnly && params.maxRebalanceOrderNotional == 0
    && params.autoRebalanceCapBps == 0 && remainingNotional != 0) {
    revert RebalanceNotSettled();
}
```

Because mark-priced exposure and NAV-derived target notionals both move during asynchronous settlement, natural price or equity changes between submission and confirmation shift the target from the original T1 to a new T2. Confirmation then checks remaining drift against T2 rather than T1, even on the ordinary uncapped path.

IMPACT

A rebalance that fully executed against its submission-time target can still fail confirmation and stay pending. While pending, the vault blocks deposits, redeems, capital movement, and further rebalances until confirmation eventually succeeds or stale cancellation clears the cycle.

Conversely, an unfilled rebalance can be accepted when later market movement makes the current target appear satisfied. This does not require any third-party deposit; ordinary market movement between submission and confirmation is enough to cause either outcome.

EXPLOIT PATH

1. With NAV N1 and target T1, a keeper calls rebalance; the vault submits IOC orders and records only position-size and progress information.
2. Before confirmation, a market move changes account equity and NAV to N2, producing a different target T2.
3. confirmRebalance builds fresh parameters from N2 and checks remaining drift against T2 instead of T1.
4. Confirmation reverts despite full T1 execution, leaving rebalancePending active and freezing NAV-sensitive operations until stale cancellation.

RECOMMENDATION

Persist the submission-time NAV (or target notional) and pass it into the confirmation calculation:

```
// storage
uint64 internal rebalanceSubmittedNav;

// in rebalance()
rebalanceSubmittedNav = totalNav.toUint64();

// in confirmRebalance()
RebalanceParams memory params =
    _rebalanceParams(rebalanceSubmittedNav, rebalanceNonce - 1);
```

Clear rebalanceSubmittedNav in _clearPendingRebalance. A more precise variant stores each asset's signed submission target, which avoids reconstructing targets from any later configuration or NAV change.

RESOLUTION

Ensemble

Commit [41c7f58279156b13128d17d456b7839dc5b8626f](#) fixes L-41. One submission-relative predicate checks the direction and filled size of every submitted asset delta. A capped or phased rebalance cannot confirm until every nonzero submitted leg reaches its recorded target. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L41_cappedPartialFillCannotConfirm](#). Related findings: L-22, L-47.

fixed-PAG

[L-42] Stale dust positions can block emergency liquidation of the whole vault

Low

Fixed

LOCATION

[VaultPortfolioLib:applyTransition:45-72](#)

SUMMARY

During a basket transition, removed dust assets are added back into the active arrays rather than kept separate. `applyTransition()` combines `dustAssets` with `finalAssets`, and `emergencyClose()` later passes that combined array to `closeAllPositions()`.

For a nonzero retained dust position, `_tryEmergencyCloseLimitPx()` performs a strict BBO read before it can return a per-asset skip result:

```
(uint64 rawBestBid, uint64 rawBestOffer) =
    PrecompileLib.bbo(p.precompileTarget, uint64(assetId));
uint64 rawReferencePx = isBuy ? rawBestOffer : rawBestBid;
if (rawReferencePx == 0) {
    rawReferencePx = _emergencyAnchorPx(p.precompileTarget, assetId, isBuy);
}
```

If that read fails, the whole transaction reverts instead of skipping only the obsolete asset. Since all close submissions are part of the reverted transaction, current-basket positions stay open.

Separately, zero-weight dust entries continue to count toward transition basket capacity, so previously removed residual markets can push a later otherwise-valid basket rotation past `MAX_ASSETS`.

IMPACT

A stale dust position in a removed market can make the vault's emergency-close transaction revert whenever that market's BBO read is unavailable.

This leaves live positions open during margin stress and defeats the intended permissionless risk-reduction path.

EXPLOIT PATH

1. A removed BTC or HYPE market retains a sub-minimum nonzero position when the owner calls `applyTransition()`.
2. The transition stores the removed asset alongside the new basket with weight zero.
3. The removed market later has a failed or unavailable BBO read, while another live-basket position makes `emergencyClose()` authorized.

4. `emergencyClose()` reaches the retained position, `bbo()` reverts, and the transaction rolls back all close orders, leaving the live position exposed.

RECOMMENDATION

Keep transition dust outside the active liquidation basket and handle it through a separate recovery mechanism:

```
_replace(_activeAssets(), _activeWeights(), finalAssets, finalWeights);
_replace(_dustAssets(), _dustSizes(), dustAssets, dustSizes);
```

Alternatively, make emergency pricing failure-tolerant per asset so one obsolete market cannot revert the entire close operation:

```
(bool priced, uint64 limitPx) = _tryEmergencyCloseLimitPx(...);
if (!priced) {
    emit EmergencyCloseAssetSkipped(assetId, isBuy);
    return;
}
_placeEmergencyCloseOrder(p, assetId, isBuy, limitPx, posSize);
```

RESOLUTION

Ensemble

We removed the early return in `8cf0d69`, as suggested. The vault attempts active assets first, then retired assets on every call. An unreadable or unfilled active position no longer skips the retired pass. Recovery still requires every position to be flat. Tests exercise both failure cases and check that active orders remain first.

fixed-PAG

[L-43] Redemptions can reopen while a stale rebalance order is still live

Low

Fixed

LOCATION

`src/vaults/shared/BaseLeveragedVault.sol:cancelStaleRebalance:550-579`

SUMMARY

A timed-out rebalance should only be cleared once the asynchronous CoreWriter orders it submitted can no longer change the vault's position. Clearing the local pending flag must be tied to venue cancellation or a settlement proof.

The zero-progress branch instead clears only local state and returns, without cancelling the deterministic CLOIDs used by the submitted orders:

```
if (LeveragedVaultLib.rebalanceProgressNotional(params) == 0) {
    _clearPendingRebalance();
    VaultExecutionPolicyLib.resetRebalanceFailures();
    _payKeeperRewardUpTo(achievedProgress);
    emit RebalanceConfirmed(cancelNonce, msg.sender);
    return;
}
```

The branch treats zero current drift as sufficient completion, but an asynchronous venue order may still be executable. `redeem()` uses `rebalancePending` as its only rebalance gate, so once the flag is cleared a shareholder can receive current-buffer USDC before the later fill lands.

IMPACT

A stale rebalance order can execute after redemptions have reopened, letting some shareholders exit before the resulting position change or loss is reflected in NAV.

The adverse performance is then concentrated among the shareholders who remain in the vault.

EXPLOIT PATH

1. A rebalance submits an IOC/CoreWriter order that HyperCore has not yet finalized or cancelled.
1. After the stale timeout, `cancelStaleRebalance()` sees zero current drift and clears `rebalancePending` without calling `cancelOrdersByAssets()`.
1. A shareholder redeems through the configured circuit breaker and receives the current calculated USDC amount.
1. The still-live order executes later, changing exposure or realizing a loss borne by the remaining shareholders.

RECOMMENDATION

Cancel the cycle's deterministic orders before clearing local state in the zero-progress branch:

```
if (LeveragedVaultLib.rebalanceProgressNotional(params) == 0) {
    LeveragedVaultLib.cancelOrdersByAssets(coreWriterTarget, _perpAssets, cancelNonce);
    _clearPendingRebalance();
    VaultExecutionPolicyLib.resetRebalanceFailures();
    _payKeeperRewardUpTo(achievedProgress);
    emit RebalanceConfirmed(cancelNonce, msg.sender);
    return;
}
```

Alternatively, keep a dedicated order-recovery flag after the timeout and keep redemptions blocked until venue cancellation or a later settlement check proves the old orders can no longer execute.

RESOLUTION

Ensemble

Commit [940424f1bce193e95db42664b5b82453419b8ae2](#) fixes L-43. Cancellation submission records its block and timestamp and keeps the rebalance pending through the timed observation period. Redemptions cannot reopen while stale orders may still be live. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L43_staleCancellationRemainsPendingUntilTimedObservation](#). Related findings: M-10, L-44.

fixed-PAG

[L-44] Stale rebalance cancellation can turn a successful partial rebalance into a recorded failure

Low Fixed

LOCATION

`BaseLeveragedVault:cancelStaleRebalance:506-535`

SUMMARY

Capped and reductions-only rebalance cycles are meant to settle once they achieve the configured minimum position-size progress, even if residual drift remains because the cap or phase gating intentionally defers further exposure changes.

`confirmRebalance` applies this progress-based rule, but the stale-cancellation path does not. It computes `achievedProgress` and then ignores it when deciding success:

```
uint256 achievedProgress = LeveragedVaultLib.positionSizeProgressNotional(params, submittedSizes);
if (LeveragedVaultLib.rebalanceProgressNotional(params) == 0) {
    _clearPendingRebalance();
    VaultExecutionPolicyLib.resetRebalanceFailures();
    _payKeeperRewardUpTo(achievedProgress);
    emit RebalanceConfirmed(cancelNonce, msg.sender);
    return;
}
```

Because the stale path only treats a cycle as successful when remaining progress is exactly zero, a cycle that met the progress threshold but left expected residual drift is treated as a failure. This divergence is especially visible in capped and reductions-only modes, where residual drift is expected and should not by itself imply failure.

IMPACT

A successful partial capped rebalance can be recorded as a failure: it increments `rebalanceFailureCount`, activates backoff, shrinks later order capacity, and cancels still-open orders.

Repeated occurrences can impair the vault's ability to correct its exposure.

EXPLOIT PATH

1. A keeper calls `rebalance()` on a vault with `maxRebalanceOrderNotional` configured; the vault stores submitted position sizes and sets `rebalancePending`.
2. The submitted IOC order partially fills, producing at least `MIN_REBALANCE_PROGRESS_NOTIONAL` of measured progress while residual drift remains.
3. Before a normal confirmation, any caller waits for `rebalanceConfirmTimeout` and calls `cancelStaleRebalance()`.
4. Since residual drift is nonzero, the function records a rebalance failure and cancels orders instead of applying the progress-based success rule that `confirmRebalance()` would accept.

PREREQUISITES

The cancel path is permissionless, so any caller can wait out the timeout and deliberately choose it to convert a progress-qualified cycle into a recorded failure. This creates a timeout race against normal confirmation.

RECOMMENDATION

Mirror the capped/reductions-only completion rule in `cancelStaleRebalance`:

```
uint256 remaining = LeveragedVaultLib.rebalanceProgressNotional(params);
if (
```

```

    (params.maxRebalanceOrderNotional != 0 || params.autoRebalanceCapBps != 0 ||
    rebalanceReductionsOnly)
    && achievedProgress >= MIN_REBALANCE_PROGRESS_NOTIONAL
  ) {
    _clearPendingRebalance();
    VaultExecutionPolicyLib.resetRebalanceFailures();
    _payKeeperRewardUpTo(achievedProgress);
    emit RebalanceConfirmed(cancelNonce, msg.sender);
    return;
  }

```

Alternatively, factor the completion predicate into one shared internal/library helper and call it from both `confirmRebalance` and `cancelStaleRebalance`.

RESOLUTION

Ensemble

Commit [25a08af851f0547898d99c63e5db727e242420df](#) fixes L-44. The first stale-cancellation observation and ordinary confirmation use the same submitted-size completion check, including capped and reductions-only cycles. Work already completed before cancellation is recorded as success rather than failure. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L44_completedCappedFillIsSuccessAtFirstStaleCall](#). Related findings: M-10, L-43.

fixed-PAG

[L-45] One unreadable price can revert the entire emergency close

Low Fixed

LOCATION

`LeveragedVaultLib::_tryEmergencyCloseLimitPx:1460-1488`

SUMMARY

Emergency close is meant to fail closed per asset: if an asset has no usable price it should be skipped while the vault stays paused and the other configured positions continue through the close loop.

`_closePosition` is built around this, checking the `priced` boolean returned by `_tryEmergencyCloseLimitPx`.

The problem is that the price helper performs strict reads before it can ever return that boolean. The initial BBO read reverts immediately on failure, and the zero-BBO or spread-bounded paths call strict mark and oracle reads inside `_emergencyAnchorPx` that also revert on failure. Any of these reverts bubbles up through `closeAllPositions` and bypasses the intended skip branch entirely.

A single failing read therefore reverts the whole `emergencyClose` transaction. Because the transaction is atomic, the vault's state transition (pause and timestamp) and any close orders already submitted for earlier assets in the loop are all rolled back.

IMPACT

A price-read failure for just one open configured asset can block every emergency-close attempt, leaving all positions exposed during exactly the condition where emergency mitigation is needed.

The permissionless emergency workflow cannot advance until the underlying read (an external market or precompile outage) recovers, so the entire portfolio stays leveraged.

EXPLOIT PATH

1. A vault with an open position enters the emergency margin condition and a keeper calls `emergencyClose()`.
1. `closeAllPositions` reaches the affected asset and `_tryEmergencyCloseLimitPx` calls `PrecompileLib.bbo` or `_emergencyAnchorPx`.
1. The strict precompile read reverts before the helper can return `priced = false`, so the revert propagates through `emergencyClose`.
1. The pause, timestamp, and all earlier close orders are rolled back, and keepers remain unable to execute `emergencyClose` until the failing read recovers.

RECOMMENDATION

Wrap every price read used by `_tryEmergencyCloseLimitPx` in a non-reverting emergency-specific helper that returns `(0, false)` on failure, so both the BBO path and the anchor path (mark and oracle reads) classify a bad read as an unpriced, skippable asset rather than reverting. The wrapper should also treat malformed returndata as unpriced.

```
function _tryEmergencyAnchorPx(...) private view returns (uint64 px, bool ok) {
    try this.markPx(...) returns (uint64 mark) { /* ... */ }
    catch { return (0, false); }
}
```

Alternatively, catch the complete helper call in `_closePosition`, skip the asset on failure, and continue the loop while preserving the paused state.

```
try this._emergencyLimit(...) returns (uint64 limitPx, bool priced) {
    if (!priced) return;
    _placeEmergencyCloseOrder(...);
} catch {
    emit EmergencyCloseAssetSkipped(assetId, isBuy);
}
```

RESOLUTION

Ensemble

Commit [b4e21e1c0b313195d6492a040aeaff8ce7d3b033](#) fixes L-45. Position, BBO, mark, and oracle reads return status instead of reverting the full emergency close. An unreadable asset is skipped with telemetry, readable siblings still close, and no reward is paid while any liability remains. Proof case: [test/foundry/unit/VaultLifecycle.t.sol::test_L45_unreadableMarkSkipsOnlyAffectedEmergencyAsset](#). Related findings: [M-9](#), [L-42](#), [L-48](#).

fixed-PAG

[L-46] Residual HyperCore dust can permanently block emergency withdrawals

Low

Fixed

LOCATION

`VaultAdminOpsLib:_recoverEmergencyAssets:190-219`

SUMMARY

The emergency flow is meant to recover all meaningful HyperCore collateral, confirm positions are flat, then enable pro-rata withdrawals for share holders. Small residual amounts that cannot independently fund a Core-to-EVM bridge should not be able to block that whole process.

The recovery path rejects this state. `_safeBridgeToEvmCapacity` subtracts the required spot reserve and returns zero when the reserve deficit consumes the entire withdrawable amount, while

`_recoverEmergencyAssets` rejects zero capacity whenever withdrawable is nonzero:

```
uint256 withdrawableAmount =
    LeveragedVaultOpsLib.withdrawable(ctx.precompileTarget);
if (withdrawableAmount == 0) return;
uint256 amount = LeveragedVaultLib.safeBridgeToEvmCapacity(...);
if (amount == 0) {
    revert ILeveragedVaultTypes.InsufficientWithdrawable(
        withdrawableAmount, 0
    );
}
```

The only alternative, `_enableEmergencyWithdraw`, requires `remainingWithdrawable == 0`. A positive amount at or below the reserve deficit therefore creates a closed loop: recovery cannot bridge it, and emergency withdrawals cannot begin.

For example, with HyperCore withdrawable equal to 1 USDC and Core spot below the 2.1 USDC reserve requirement, capacity computes to zero, recovery reverts, and activation reverts.

IMPACT

A small residual HyperCore withdrawable balance can freeze the emergency withdrawal lifecycle and prevent all share holders from claiming recovered USDC until an external HyperCore state change or administrative intervention clears the dust. The blocked funds can include the entire already-recovered EVM buffer belonging to all share holders.

The same stuck state also blocks `_unpause`, since unpause and emergency-withdraw activation both inherit the exact-zero withdrawable requirement once recovery can no longer progress.

EXPLOIT PATH

1. A vault enters emergency recovery with positions closed, EVM buffer recovered, HyperCore withdrawable equal to 1 USDC, and Core spot below the 2.1 USDC reserve requirement. A small external Core deposit can also induce this dust state.
1. The owner calls `recoverAllEmergencyFunds()`; `_safeBridgeToEvmCapacity` returns zero capacity because the spot deficit is at least 1 USDC.
1. `_recoverEmergencyAssets` reverts instead of recording or resolving the residual amount.
1. `enableEmergencyWithdraw()` also reverts on the nonzero withdrawable balance, so users cannot claim their recovered EVM USDC.

RECOMMENDATION

Permit emergency activation when the remaining withdrawable amount is below the minimum recoverable bridge threshold, after a timeout and explicit dust accounting:

```
uint256 withdrawableAmount =
    LeveragedVaultOpsLib.withdrawable(ctx.precompileTarget);
uint256 capacity = LeveragedVaultLib.safeBridgeToEvmCapacity(...);
if (capacity == 0 && withdrawableAmount <= _dustThreshold()) {
    emit EmergencyRecoveryComplete();
    return;
}
```

Use the same dust threshold in `_enableEmergencyWithdraw`:

```
if (remainingWithdrawable > _dustThreshold()) {
    revert ILeveragedVaultTypes.EmergencyAssetsNotRecovered(
        remainingWithdrawable
    );
}
```

RESOLUTION**Ensemble**

`163e70f` applies the spot dust rule to attributed perp residuals as well. Recovery, withdrawal enablement, unpause, and the Lens all use `amount < effectiveDust`. A residual below the threshold no longer blocks completion when bridge capacity is zero. Equality still blocks withdrawal enablement and unpause.

We also wired up `EmergencyCoreResidualExcluded` to report the excluded amounts. Tests check zero capacity, the exact threshold, event values, and agreement between the Lens and execution.

fixed-PAG

[L-47] Stuck rebalance can block all deposits and redeems until stale cancellation

Low

Fixed

LOCATION

`BaseLeveragedVault:confirmRebalance:430-456`

SUMMARY

A rebalance cycle is meant to finish once every remaining exposure is either corrected or too small to produce a valid HyperCore order. Submission and preview logic enforce this by applying lot rounding and the minimum-notional check before counting an order.

The uncapped confirmation path uses a different rule. It counts raw residual deltas without applying the same placeability test:

```
uint256 absDelta = delta >= 0 ? uint256(delta) : uint256(-delta);
if (absDelta >= MIN_ORDER_NOTIONAL) {
    totalProgressNotional += absDelta;
}
```

A residual can be unplaceable after rounding yet still nonzero at confirmation time. The vault then rejects confirmation even though no valid corrective order remains, keeping `rebalancePending` set.

IMPACT

Partially filled uncapped rebalances can stay pending when no further valid order can ever be placed.

Deposits and redeems remain blocked until stale cancellation, and the cycle may be recorded as a rebalance failure.

EXPLOIT PATH

1. A 10,000,000-unit rebalance delta is submitted as an IOC order for an asset whose lot rounding removes enough size to make a near-threshold residual unexecutable.
1. The IOC partially fills, leaving a raw residual of 10,100,000 units.
1. `_countPlaceableRebalanceOrders()` builds the rounded order and returns zero because its executable notional is below 10,000,000.
1. `confirmRebalance()` calls `rebalanceProgressNotional()`, obtains 10,100,000, reverts, and leaves deposits and redeems blocked until stale cancellation.

RECOMMENDATION

Centralize the residual test and reuse the actual order-placement predicate during confirmation:

```
function _hasPlaceableRemaining(RebalanceParams memory p) private view returns (bool) {
    return _countPlaceableRebalanceOrders(p, totalTargetNotional, address(this), false) != 0;
}
```

Then settle an uncapped cycle when no placeable residual remains:

```
if (!hasPlaceableRemaining(params)) {
    _clearPendingRebalance();
} else {
    revert RebalanceNotSettled();
}
```

RESOLUTION

Ensemble

Commit `f7a21a0951b12c87c886c35c9b5b61d007176d78` fixes L-47. A residual blocks settlement only if it can still produce a valid order under the same book-readability, oracle-band, lot, minimum-notional, and adaptive-cap rules used for placement. Work that cannot be placed no longer freezes deposits and redemptions. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L47_subminimumSubmittedResidualDoesNotBlockConfirmation`. Related findings: L-41.

fixed-PAG

[L-48] Unpriceable positions can permanently lock all withdrawals during emergency recovery

Low

Fixed

LOCATION

`LeveragedVaultLib:_closePosition:1450-1471`

SUMMARY

Emergency close is meant to pause the vault and submit reduce-only orders for every open configured position, so the owner can later recover collateral and enable pro-rata withdrawals.

When an asset cannot produce a valid close price, the implementation skips it and returns without placing an order:

```
(bool priced) = _tryEmergencyCloseLimitPx(p, assetId, isBuy, _perpSizeDecimals(assetId));
if (!priced) {
    emit EmergencyCloseAssetSkipped(assetId, isBuy);
    return;
}
```

The vault stays paused, but recovery later requires exact flatness through `ensurePositionsClosed`. Because no order was submitted for the skipped position, that check keeps reverting and `emergencyTotalShares` can never be initialized.

A zero or otherwise unusable computed emergency limit price triggers the same skip path, so the gap is not limited to pure market-data outages.

IMPACT

All holders can lose access to both normal redemption and emergency withdrawal while an open configured position remains unpriceable.

Normal redemption is unavailable during the paused/emergency recovery flow, so the lock can persist indefinitely if the external market or read dependency does not recover.

EXPLOIT PATH

1. An open configured position exists while its BBO, mark, and oracle values are zero or unreadable.
1. A caller invokes `emergencyClose`; the vault moves to Paused and emits `EmergencyCloseAssetSkipped` without submitting a close order.
1. The owner invokes `recoverPausedEmergencyFunds` or `enableEmergencyWithdraw`, but `ensurePositionsClosed` detects the unchanged nonzero position and reverts.
1. Until the external price dependency recovers or a separate recovery mechanism is introduced, normal redemption and emergency withdrawal stay unavailable.

RECOMMENDATION

Add an explicit recovery mode for skipped positions rather than requiring the normal exact-flatness check forever:

```
if (unpriceablePosition[assetId]) {
    _handleUnpriceablePosition(assetId, posSize);
} else {
    _placeEmergencyCloseOrder(p, assetId, isBuy, limitPx, posSize);
}
```

Alternatively, permit emergency-withdraw activation once all priceable positions are closed and snapshot the unresolved position as a separately accounted liability, with clear authorization and audit events.

RESOLUTION

Ensemble

Commit `d6354c83918ba445a5de842c9ef79164534e99c4` fixes L-48. After cancellation drain completes, the VaultAdmin owner may submit a manual-price close for a configured live position while the vault is Paused or Emergency. The order remains IOC and reduce-only, giving recovery a bounded path when market prices are unreadable. Proof case: `test/foundry/unit/VaultLifecycle.t.sol::test_L48_ownerManualPriceRecoveryClosesUnpriceablePausedPositionRepeatably`. Related findings: L-42, L-45.

fixed-PAG

[L-49] Vault is missing standard ERC-4626 preview and conversion functions

Low

Fixed

LOCATION

`BaseLeveragedVault:ERC4626InterfaceSurface:asset()-missing-conversion-selectors`

SUMMARY

An ERC-4626-style vault is expected to expose conversion and preview functions on the vault address so integrators can quote deposits and redemptions before calling the state-changing methods.

BaseLeveragedVault only exposes `asset()`, `deposit()`, and `redeem()`. The conversion and preview helpers live on `VaultLens`, which is a separate address and does not receive calls sent to the vault.

As a result, standard calls such as `vault.previewDeposit/assets)` and `vault.convertToShares/assets)` have no implementation on the vault and revert.

IMPACT

ERC-4626 aggregators, wrappers, and routers that rely on conversion or preview functions cannot integrate with the vault. They may reject or disable deposits and redemptions even when the vault's custom route would otherwise work, breaking composability.

EXPLOIT PATH

1. Configure an ERC-4626 wrapper to use a deployed BULL vault as its target.
2. The wrapper calls `vault.previewDeposit(10000000)` to calculate expected shares.
3. The call reaches BaseLeveragedVault, which has no `previewDeposit` selector, and reverts.
4. The wrapper aborts the operation or marks the vault incompatible because the required standard quote cannot be obtained.

RECOMMENDATION

Add the missing ERC-4626 conversion and preview functions directly to BaseLeveragedVault, mirroring the actual fee and rounding behavior.

```
function convertToShares(uint256 assets)
    external view returns (uint256)
{
    (, uint256 pps) = computeNav();
    return (assets * SHARE_SCALE) / pps;
}
```



```
function previewRedeem(uint256 shares)
    external view returns (uint256)
{
    (, uint256 pps) = computeNav();
    return LeveragedVaultLib.computeRedeemAmounts(
        shares, pps, withdrawalFeeRate, SHARE_SCALE
    ).netAssets;
}
```

Implement the remaining required selectors with the same source-of-truth math, or explicitly drop the ERC-4626-style interface claim.

RESOLUTION

Ensemble

In `3734d01`, `totalAssets`, `convertToShares`, and `convertToAssets` catch HyperCore precompile failures and use the existing fee-net AUM checkpoint. Previews and transactions still require live reads. If an upgraded vault has no checkpoint yet, the fallback uses accounted EVM assets minus fees. The rollout initializes the checkpoint before returning the vault to service.

Tests cover failures of both Core reads, the unset-checkpoint case, and unrelated errors, which still propagate. We also removed the unused `OnlyCircuitBreaker` error.

fixed-PAG